

necesito desarrollar un proyecto en django para un servicio de otp

Desarrollar un servicio de **OTP (One-Time Password)** en Django es una excelente decisión para fortalecer la seguridad de cualquier aplicación. Básicamente, necesitas un sistema que genere un código aleatorio, lo almacene temporalmente y lo valide contra la entrada del usuario.

Aquí tienes una hoja de ruta técnica para construirlo desde cero:

1. Arquitectura del Proyecto

Para que el sistema sea robusto, te sugiero dividir la lógica en tres componentes principales:

1. **Generación:** Crear un código numérico aleatorio (normalmente de 6 dígitos).
2. **Transporte:** Enviar el código vía Email (usando el core de Django) o SMS (usando servicios como Twilio o Vonage).
3. **Validación:** Comparar el código ingresado por el usuario con el almacenado, verificando que no haya expirado.

2. Modelado de Datos

No es recomendable guardar el OTP directamente en el modelo de `User`. Lo ideal es una tabla separada para manejar la expiración.

Python



```
from django.db import models
from django.conf import settings
import uuid

class OTPVerification(models.Model):
    user = models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=models.CASCADE)
    code = models.CharField(max_length=6)
    created_at = models.DateTimeField(auto_now_add=True)
    is_verified = models.BooleanField(default=False)

    # Podrías usar un campo para definir cuándo expira
    def is_expired(self):
```

```
# Lógica para comparar created_at con el tiempo actual
pass
```

3. Herramientas y Librerías Recomendadas

Si quieres evitar "reinventar la rueda" y seguir estándares de seguridad (como TOTP), estas librerías son las líderes en el ecosistema Django:

- **django-otp**: La librería estándar. Es modular y permite agregar dispositivos (SMS, Email, aplicaciones de autenticación como Google Authenticator).
- **django-two-factor-auth**: Una solución completa que ya trae las vistas y formularios listos para usar con `django-otp`.
- **PyOTP**: Si prefieres construir tu propia lógica, esta librería genera y verifica códigos compatibles con aplicaciones de autenticación.

4. El Flujo de Seguridad

Es vital considerar la **fuerza bruta**. Si un atacante intenta adivinar el código, debes tener un límite de intentos.

Tip de experto: Nunca envíes el código OTP en la respuesta de la API. El código solo debe viajar hacia el canal de comunicación (Email/SMS). En la respuesta de la API, solo confirma que el código fue enviado.

5. Ejemplo de Generación Simple

Si decides hacerlo manual, puedes usar el módulo `secrets` de Python para mayor seguridad criptográfica:

Python



```
import secrets

def generate_otp():
    return "".join([str(secrets.randbelow(10)) for _ in range(6)])
```

¿Tienes pensado usar este servicio para el inicio de sesión (sustituyendo la contraseña) o como un **segundo factor (2FA)** después de que el usuario ya ingresó sus credenciales?

Implementar un segundo factor (2FA) es el estándar de oro hoy en día. En este flujo, el usuario primero se identifica con sus credenciales tradicionales y, solo si estas son correctas, el sistema le solicita el código OTP.

Aquí te detallo cómo estructurar la lógica específicamente para un **segundo factor** en Django:

1. El Flujo de Autenticación

Es crucial que el usuario no esté "logueado" completamente hasta que pase el segundo paso. Normalmente se maneja una sesión intermedia o un estado de "pre-autenticación".

2. Implementación Paso a Paso

A. Almacenamiento Temporal

En lugar de autenticar al usuario inmediatamente con `login(request, user)`, puedes guardar el ID del usuario en la sesión de Django de forma temporal:

Python



```
# En tu vista de Login inicial
if user.check_password(password):
    request.session['pre_auth_user_id'] = user.pk
    # Aquí disparas el envío del OTP
    send_otp_to_user(user)
    return redirect('verify-otp')
```

B. Verificación del OTP

Crea una vista que reciba el código. Si es correcto, recuperas al usuario de la sesión y completas el login oficial.

Python



```
def verify_otp_view(request):
    user_id = request.session.get('pre_auth_user_id')
```

```

if not user_id:
    return redirect('login')

user = User.objects.get(pk=user_id)
input_code = request.POST.get('otp_code')

if validate_otp(user, input_code): # Tu lógica de validación
    from django.contrib.auth import login
    login(request, user)
    del request.session['pre_auth_user_id']
    return redirect('dashboard')
else:
    # Manejar error de código incorrecto

```

3. Consideraciones de Seguridad para 2FA

Característica	Recomendación
Ventana de Tiempo	El OTP debe expirar en 3 a 5 minutos .
Limpieza	Una vez usado el código, debe marcarse como utilizado o eliminarse para evitar ataques de reutilización (<i>replay attacks</i>).
Throttling	Si el usuario falla 3 o 5 veces, bloquea el proceso de OTP por 15 minutos para evitar ataques de fuerza bruta.
Backup Codes	Considera generar códigos de un solo uso (estáticos) por si el usuario pierde acceso a su teléfono o email.

...

4. Recomendación de Librería (2FA Nativo)

Si buscas algo profesional y mantenido, te recomiendo encarecidamente usar `django-two-factor-auth`.

1. **Instalación:** `pip install django-two-factor-auth[phonenumbers]`
2. **Ventaja:** Maneja por ti la interfaz de usuario, los códigos de respaldo, y es compatible con aplicaciones como **Google Authenticator** o **Authy** mediante códigos QR (TOTP), lo cual te ahorra costos de envío de SMS.

¿Prefieres que profundicemos en cómo integrar una librería como **django-two-factor-auth** o prefieres seguir construyendo una solución **manual y personalizada** para tener control total?

este código pienso utilizarlo solo para la firma digital de documentos

Entendido. Cuando el OTP se usa para **firma digital de documentos**, el nivel de criticidad aumenta. Ya no solo estás validando un acceso, estás vinculando la identidad del usuario a una acción legal o administrativa.

En este contexto, el OTP actúa como una **Firma Electrónica Simple** (o avanzada, dependiendo de tu legislación local), donde el código sirve como evidencia de que el firmante tuvo el control del proceso en un momento específico.

Aquí tienes cómo adaptar tu proyecto Django para este caso de uso:

1. Flujo de Trabajo (Workflow)

A diferencia del login, aquí el proceso suele dispararse por un evento sobre un archivo:

1. **Visualización:** El usuario revisa el documento en la plataforma.
2. **Solicitud de Firma:** El usuario hace clic en "Firmar".
3. **Desafío (Challenge):** El sistema genera el OTP y lo envía (Email/SMS).
4. **Verificación y Sellado:** El usuario ingresa el código. Si es correcto, el sistema marca el documento como firmado y genera un **log de auditoría**.

2. El Log de Auditoría (Crucial)

Para que una firma con OTP sea válida en una auditoría, no basta con borrar el código y ya. Debes guardar una prueba de la transacción.

Modelo de Auditoría de Firma

Python



```
class DocumentSignature(models.Model):  
    document = models.ForeignKey('Document', on_delete=models.CASCADE)
```

```
signer = models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=models.CASCADE)
timestamp = models.DateTimeField(auto_now_add=True)
ip_address = models.GenericIPAddressField()
otp_method = models.CharField(max_length=20) # 'email' o 'sms'
metadata = models.JSONField() # Para guardar User-Agent, etc.
integrity_hash = models.CharField(max_length=64) # Hash del documento al momento
```

3. Implementación Técnica Recomendada

A. No uses la sesión para el estado

En firmas de documentos, es mejor usar un **Token de Transacción**. Cuando el usuario pide firmar, generas un registro temporal en la base de datos que vincula: **Documento + Usuario + OTP**.

B. Generación y Envío

Te sugiero usar **PyOTP** para generar códigos basados en tiempo (TOTP), pero enviándolos tú manualmente para que el usuario no necesite una app externa.

Python



```
import pyotp

# Generar una clave secreta única para esta transacción de firma
secret = pyotp.random_base32()
totp = pyotp.TOTP(secret, interval=300) # Válido por 5 minutos
otp_code = totp.now()

# Guardar 'secret' temporalmente en tu DB para verificarlo luego
```

4. Consideraciones Legales y de Seguridad

- **Integridad del Documento:** Al momento de la firma, calcula el hash (SHA-256) del archivo original y guárdalo en el registro de la firma. Si el documento cambia después, la firma queda invalidada.
- **No Reutilización:** Una vez que el código se usa para firmar el documento A, ese mismo código no debe servir para el documento B, incluso si se intenta en el mismo minuto.
- **Prueba de Entrega:** Si usas un proveedor de SMS (como Twilio) o Email (como SendGrid), guarda el **Message ID** que ellos te devuelven como prueba de que el código fue enviado al destino correcto.

5. Ejemplo de Vista de Firma

Python



```
def sign_document_view(request, doc_id):
    if request.method == 'POST':
        user_otp = request.POST.get('otp')
        stored_transaction = get_object_or_404(SignatureTransaction, doc_id=doc_id, u

        if stored_transaction.is_valid(user_otp):
            # 1. Marcar documento como firmado
            # 2. Generar certificado de firma (PDF con los datos de auditoría)
            # 3. Borrar la transacción temporal
            return JsonResponse({'status': 'signed'})

        return JsonResponse({'error': 'Código inválido'}, status=400)
```

¿El documento que van a firmar es un PDF generado dinámicamente en Django o es un archivo que los usuarios suben a la plataforma?

queremos separar en proyecto distintos la visualización de documento, el servicio de OTP, la firma del documento y el envío de correo

Separar estas funcionalidades en microservicios o proyectos independientes es una decisión de arquitectura muy inteligente, especialmente para un sistema de firma digital. Esto te permite escalar el servicio de OTP de forma independiente (por si tienes picos de tráfico) y protege la integridad de los documentos.

Aquí tienes cómo estructurar esta **Arquitectura Orientada a Servicios (SOA)**:

1. Mapa de la Arquitectura

Cada "proyecto" o servicio tendrá una responsabilidad única y se comunicará mediante **APIs REST** o **Webhooks**.

A. Servicio de Visualización (Frontend/Portal)

- **Responsabilidad:** Interfaz de usuario. Carga el documento desde un almacenamiento seguro (S3, Azure Blob) y lo renderiza en el navegador.
- **Tecnología:** Django (como orquestador) + PDF.js o React/Vue.

B. Servicio de OTP (El "Validador")

- **Responsabilidad:** Generar secretos únicos, almacenarlos temporalmente y validarlos.
- **Lógica:** No conoce nada de documentos. Solo recibe un `user_id` o `transaction_id`, genera un código de 6 dígitos y lo guarda (Redis es ideal aquí por su auto-expiración).
- **Endpoint clave:** `POST /generate-otp/` y `POST /validate-otp/`.

C. Servicio de Firma (El "Core")

- **Responsabilidad:** Es el único que tiene permisos de **escritura** sobre los documentos originales.
- **Acción:** Recibe la confirmación de que el OTP fue válido, toma el documento, le aplica el sello digital (puedes usar librerías como `PyMuPDF` o `ReportLab` para estampar la firma) y genera el log de auditoría.

D. Servicio de Notificaciones (Email/SMS)

- **Responsabilidad:** Un "worker" dedicado solo a enviar mensajes.
- **Ventaja:** Si el servidor de correos falla o es lento, no detiene el proceso de firma del usuario. Puedes usar **Celery** con **RabbitMQ/Redis** para manejar estas colas.

2. Flujo de Comunicación entre Proyectos

Para que esto funcione, los servicios deben hablar entre sí de forma segura (usando **API Keys** o **JWT**).

1. **Usuario** hace clic en "Firmar" en el **Servicio de Visualización**.
2. El **Servicio de Firma** crea una transacción y solicita al **Servicio de OTP** que genere un código.
3. El **Servicio de OTP** genera el código y envía una señal al **Servicio de Email** para que lo entregue al usuario.
4. El **Usuario** ingresa el código en la interfaz; esta se envía al **Servicio de OTP** para validación.
5. Si es `True`, el **Servicio de Firma** procede a sellar el documento y notifica el éxito.

3. Matriz de Responsabilidades Técnicas

Servicio	Almacena Datos	Dependencias Principales
Visualización	No (solo sesión)	API Gateway
OTP	Sí (Redis / Cache)	PyOTP
Firma	Sí (PostgreSQL)	PyMuPDF, Hashlib
Email	No (Logs de envío)	SendGrid / Amazon SES
...		

4. Consideración de Seguridad: El "Idempotency Key"

Como tienes servicios separados, corres el riesgo de que una petición falle a mitad de camino. Te recomiendo que cuando el **Servicio de Firma** le pida algo al **Servicio de OTP**, envíe un `UUID` único. Así, si la red falla y reintentas la petición, el servicio de OTP sabrá que es la misma transacción y no generará dos códigos distintos.

¿Estás planeando desplegar estos servicios en contenedores independientes (como **Docker/Kubernetes**) o prefieres mantenerlos como aplicaciones Django separadas en un mismo servidor por ahora?

en esta primera discusion quiero detallar mas el servicio de OTP

Perfecto, pongamos la lupa sobre el **Servicio de OTP**. Al ser un componente independiente en tu arquitectura de microservicios, debe funcionar como una "caja negra" de validación: tú le envías una identidad y él te responde si el código es válido o no.

Aquí tienes el diseño técnico detallado para este servicio específico en Django:

1. Responsabilidades del Microservicio

El servicio de OTP no debe saber qué documento se está firmando. Solo debe gestionar:

- **Generación:** Crear el secreto y el código.

- **Persistencia temporal:** Almacenar el código con un tiempo de vida (TTL).
- **Intento y Bloqueo:** Controlar cuántas veces se ha equivocado el usuario.
- **Verificación:** Confirmar la validez y marcarlo como usado.

2. Definición de la API (Contrato)

Al ser un servicio separado, interactuarás con él mediante endpoints REST.

POST /api/otp/generate/

Cuerpo de la petición:

JSON



```
{
  "transaction_id": "uuid-doc-123",
  "user_identifier": "usuario@email.com",
  "ttl_minutes": 5
}
```

Respuesta:

JSON



```
{
  "otp_id": "internal-uuid",
  "status": "generated",
  "expires_at": "2026-04-20T15:30:00Z"
}
```

POST /api/otp/verify/

Cuerpo de la petición:

JSON



```
{
  "transaction_id": "uuid-doc-123",
  "code": "123456"
}
```

Respuesta:

JSON



```
{
  "valid": true,
  "message": "OTP verified successfully"
}
```

3. Stack Tecnológico Sugerido

Para este servicio, la velocidad es clave. Te sugiero esta combinación:

- **Django + Django REST Framework (DRF):** Para la estructura de la API.
- **Redis:** Como base de datos principal. ¿Por qué? Porque los OTP son efímeros. Redis permite usar `SETEX` (set with expiration), lo que hace que el código desaparezca solo después de N segundos sin que tengas que limpiar la base de datos manualmente.
- **PyOTP:** Para la lógica de generación de tokens (basados en tiempo o contador).

4. Lógica de Seguridad Interna

A. El Algoritmo (TOTP vs HOTP)

Para firma de documentos, te recomiendo **HOTP (HMAC-based OTP)** o un esquema basado en un **Secreto Aleatorio** único por transacción, en lugar de tiempo puro. Esto asegura que el código solo sirva para esa firma específica.

B. Anti-Fuerza Bruta (Throttling)

Debes implementar un contador de intentos en Redis:

1. Por cada fallo en `verify`, incrementa un contador vinculado al `transaction_id`.
2. Si el contador llega a 3, invalida el OTP y bloquea nuevos intentos por 10 minutos.

C. Consumo Único (Burn on use)

Inmediatamente después de una validación exitosa, el servicio debe borrar el código de Redis. Un OTP no puede ser usado dos veces, ni siquiera dentro de su ventana de tiempo de validez.

5. Integración con el Servicio de Correo

Como quieres separar el envío de correos, el flujo sería:

1. **Servicio de Firma** → llama a → **Servicio de OTP**.
2. El **Servicio de OTP** genera el código y publica un evento en una cola (ej. **Celery** + **RabbitMQ**).
3. El **Servicio de Email** escucha esa cola y envía el correo al usuario.

6. Seguridad de la Comunicación (M2M)

Como este servicio será atacado si alguien lo descubre, la comunicación entre tus microservicios debe estar protegida:

- **API Keys:** Solo el Servicio de Firma tiene una llave válida para pedir OTPs.
- **Firmas HMAC:** Las peticiones entre servicios pueden ir firmadas para asegurar que nadie alteró el `user_identifier` en el camino.

¿Quieres que veamos cómo sería el modelo de datos en Redis para manejar los intentos fallidos, o prefieres definir primero cómo el Servicio de Firma recibirá la confirmación?

si me gustaria ver como seria el modelo de datos

Al tratarse de un servicio de OTP independiente y de alto rendimiento, el "modelo de datos" no tiene por qué vivir exclusivamente en una base de datos relacional (como PostgreSQL). Lo más eficiente es un modelo híbrido: **Redis** para la volatilidad y **PostgreSQL** para la auditoría histórica.

Aquí tienes el detalle de ambos:

1. Almacenamiento Volátil (Redis)

Redis es perfecto porque gestiona la expiración automáticamente. No guardamos objetos complejos, sino estructuras **Key-Value** optimizadas.

Estructura de Llaves

Usaremos prefijos para organizar los datos y evitar colisiones:

- **El Código:** `otp:code:[transaction_id]`
 - **Valor:** El código de 6 dígitos (hasheado por seguridad).
 - **TTL:** 300 segundos (5 min).
- **Intentos Fallidos:** `otp:attempts:[transaction_id]`

- **Valor:** Entero (ej. 1, 2, 3).
- **Lógica:** Se incrementa en cada fallo. Si llega al límite, se bloquea.
- **Metadata Técnica:** `otp:meta:[transaction_id]`
 - **Valor:** JSON con el `user_id` y el `email` al que se envió.

2. Modelo de Datos en Django (PostgreSQL)

Aunque el código muera en 5 minutos, el **Servicio de Firma** necesita un rastro histórico de qué OTP se emitió y qué pasó con él para fines legales.

Python



```
from django.db import models
import uuid

class OTPTransaction(models.Model):
    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    user_identifier = models.CharField(max_length=255, db_index=True)

    # Estados del OTP
    STATUS_CHOICES = [
        ('pending', 'Pendiente'),
        ('verified', 'Verificado'),
        ('failed', 'Fallido (Exceso de intentos)'),
        ('expired', 'Expirado'),
    ]
    status = models.CharField(max_length=10, choices=STATUS_CHOICES, default='pending')

    # Auditoría técnica
    created_at = models.DateTimeField(auto_now_add=True)
    verified_at = models.DateTimeField(null=True, blank=True)
    ip_address = models.GenericIPAddressField()

    # Relación con el sistema externo
    external_reference = models.CharField(max_length=100, help_text="ID del documento")

    class Meta:
        verbose_name = "Transacción OTP"
```

3. Lógica de Interacción (Pseudo-código)

Cuando el **Servicio de OTP** recibe una petición de generación, el flujo de datos es este:

1. **DB Relacional:** Crea un registro en `OTPTransaction` con estado `pending`.

2. **Redis:** * `SETEX otp:code:uuid 300 "valor_hash"`
 1. `SETEX otp:attempts:uuid 300 0`
3. **Evento:** Envía a la cola de mensajería (RabbitMQ/Celery) la instrucción para que el **Servicio de Email** dispare el correo.

4. ¿Por qué hashear el OTP en Redis?

Incluso si Redis es una base de datos interna, es una buena práctica no guardar el código de 6 dígitos en texto plano. Si alguien logra acceso a la memoria de Redis, no podrá ver los códigos activos.

- Usa algo simple como `hashlib.sha256(code + salt).hexdigest()`.

5. El "Check" de Validación

Cuando llega el código del usuario:

1. Buscas en Redis `otp:code:[uuid]`. Si no existe → **Expirado**.
2. Si existe, comparas hashes.
3. Si falla: incrementas `otp:attempts:[uuid]`. Si llega a 3, borras la llave del código y marcas la DB como `failed`.
4. Si acierta: borras la llave de Redis y marcas la DB como `verified`.

¿Ves viable integrar **Redis** en tu infraestructura actual o prefieres que adaptemos este modelo para que funcione **100% sobre la base de datos SQL** tradicional?

me gustaria que me aclares en que momento utilizo el envio de correo o sms en mi servicio de OTP

Esta es una pregunta clave de arquitectura. En un sistema de microservicios, el **Servicio de OTP** es el "cerebro" que conoce el código, pero no debería ser necesariamente el "brazo" que lo entrega.

El envío del correo o SMS debe ocurrir **inmediatamente después de que el código ha sido generado y persistido** en tu almacenamiento temporal (Redis).

Aquí te detallo el flujo exacto paso a paso:

El Momento Exacto: "Generar y Disparar"

El envío no es un paso previo, ni un paso muy posterior; es parte de la **transacción de creación**.

Flujo Lógico en el Servicio de OTP:

1. **Recepción:** El Servicio de Firma pide un OTP para el `Usuario X`.
2. **Cálculo:** El Servicio de OTP genera el código (ej: `529301`).
3. **Persistencia:** El Servicio de OTP guarda ese código en Redis con su tiempo de expiración. **Este paso es vital:** si envías el correo antes de guardar el código y la base de datos falla, el usuario tendrá un código en su bandeja de entrada que el sistema no podrá validar.
4. **Disparo (El Envío):** Una vez que Redis confirma que el código está guardado, el Servicio de OTP envía una instrucción al **Servicio de Correo/SMS**.

¿Cómo se comunican el Servicio de OTP y el de Envío?

Tienes dos caminos principales dependiendo de la carga de trabajo que esperes:

Opción A: Comunicación Sincrónica (HTTP)

El Servicio de OTP hace una petición `POST` al Servicio de Email.

- **Pros:** Sencillo de implementar.
- **Contras:** Si el servicio de Email está lento, el Servicio de OTP se queda "esperando", bloqueando la respuesta al usuario.

Opción B: Comunicación Asincrónica (Colas de Mensajería) - Recomendado

El Servicio de OTP publica un mensaje en un "Broker" (como Redis o RabbitMQ) y el Servicio de Email lo procesa en segundo plano.

Python



```
# Ejemplo de lo que el Servicio de OTP envía a la cola:
{
  "task": "send_email_otp",
  "recipient": "usuario@email.com",
  "body": "Tu código de firma es: 529301",
  "template_id": "firma_documento_otp"
}
```

¿Por qué hacerlo así?

1. **Aislamiento de Errores:** Si el proveedor de SMS (ej. Twilio) se cae, tu Servicio de OTP sigue funcionando perfectamente. El mensaje se queda en la cola y se reintenta automáticamente cuando el proveedor vuelva.
2. **Seguridad de la Información:** El Servicio de Email solo necesita saber a quién enviar y qué texto enviar. No necesita saber qué documento se está firmando ni tener acceso a la lógica de validación.
3. **Experiencia de Usuario:** Tu API de OTP responde "Código enviado" en milisegundos, mientras que el proceso de conectar con los servidores de Google o Outlook para mandar un correo puede tardar 2 o 3 segundos.

Resumen del proceso de firma completo:

Orden	Acción	Servicio Responsable
1	Usuario da clic en "Firmar"	Visualización / Frontend
2	Crea transacción de firma y solicita OTP	Servicio de Firma
3	Genera código y lo guarda en Redis	Servicio de OTP
4	Toma el código y lo manda al usuario	Servicio de Email/SMS
5	Usuario ingresa el código	Visualización / Frontend
6	Valida si el código coincide	Servicio de OTP
7	Procede con el sellado del PDF	Servicio de Firma

...

¿Te hace sentido que el Servicio de OTP sea quien "mande la orden" al de correos, o habías pensado que el Servicio de Firma controlara ambos?

me gustariar evaaluar las 2 opciones integrar Redis o adaptarlo a que funcione 100 sobre la base de datos SQL tradicional

Evaluar si usar **Redis** o una base de datos **SQL (PostgreSQL/MySQL)** para el servicio de OTP es una decisión que depende del balance entre **rendimiento, complejidad de infraestructura y volumen de firmas** que esperas manejar.

Aquí tienes una comparativa detallada para ayudarte a decidir:

Opción 1: Arquitectura 100% SQL (PostgreSQL)

En este modelo, el OTP se guarda en una tabla con una columna de "expiración".

- **Cómo funciona:** Insertas una fila con el código y un `timestamp`. Para validar, buscas la fila y compruebas si `now() < expiration_time`.
- **Pros:**
 - **Simplicidad:** No necesitas instalar ni mantener otro servidor (Redis). Menos piezas que pueden fallar.
 - **Consistencia:** Tienes todo en un solo lugar. La auditoría y el código activo están en la misma transacción.
 - **Consultas complejas:** Es más fácil sacar reportes inmediatos de "cuántos OTP fallaron en la última hora".
- **Contras:**
 - **Basura en la DB:** Debes crear un proceso (un *Cron job* o *Celery beat*) que borre los registros viejos cada pocos minutos, o tu tabla crecerá infinitamente con códigos inútiles.
 - **Rendimiento:** Escribir y leer en disco (SQL) es más lento que en memoria (Redis). Para miles de firmas simultáneas, la base de datos podría sufrir.

Opción 2: Arquitectura Híbrida (Redis + SQL)

Redis maneja el "código vivo" y SQL guarda el "historial".

- **Cómo funciona:** El código se guarda en Redis con un `TTL` (Time To Live). Al expirar, Redis lo borra solo. En SQL solo guardas que "se intentó una firma".
- **Pros:**
 - **Velocidad extrema:** Redis responde en microsegundos. La experiencia de usuario es instantánea.

- **Auto-limpieza:** No tienes que programar nada para borrar códigos viejos; Redis lo hace por ti de forma nativa.
- **Escalabilidad:** Separa la carga de "validación rápida" de la carga de "almacenamiento persistente".
- **Contras:**
 - **Infraestructura:** Debes configurar, asegurar y pagar por un servidor de Redis.
 - **Volatilidad:** Si el servidor de Redis se reinicia y no tiene persistencia activada, todos los OTP activos en ese segundo se pierden (el usuario tendría que pedir otro).

Cuadro Comparativo para Toma de Decisión

Característica	SQL Tradicional (PostgreSQL)	Redis (In-Memory)
Dificultad de Implementación	Baja (Ya lo tienes)	Media (Configuración extra)
Gestión de Expiración	Manual (Requiere limpieza)	Automática (Nativa)
Velocidad de Respuesta	Media	Muy Alta
Costo de Servidor	Bajo	Adicional
Uso ideal	Pocas firmas / Prototipos / MVP	Alta concurrencia / Producción masiva

...

Mi Recomendación Profesional

1. **Si estás en una fase inicial (MVP):** Empieza con **SQL**. Django maneja muy bien las bases de datos relacionales. Puedes usar un campo `expires_at` y un pequeño script de limpieza. Evitarás la complejidad de gestionar Redis mientras validas el modelo de negocio.
2. **Si el sistema es para una entidad financiera o de gran escala:** Ve directamente por **Redis**. La capacidad de bloquear intentos de fuerza bruta y manejar la expiración sin "ensuciar" tu base de datos principal es una ventaja técnica enorme a largo plazo.

Un punto medio: Si usas Django, puedes usar el sistema de **Caché de Django** configurado para usar la base de datos. Así escribes código de "caché" (que es fácil de migrar a Redis luego) pero por ahora los datos se guardan en una tabla oculta de SQL.

¿Qué volumen de documentos estimas que se firmarán al mes en este sistema? Esto podría inclinar la balanza definitivamente.

la cantidad de documento no es muy alta

Si el volumen de documentos no es muy alto, la balanza se inclina definitivamente hacia la **base de datos SQL tradicional (PostgreSQL)**.

Para tu caso de uso, introducir Redis sería añadir una "pieza móvil" más a la infraestructura que probablemente no justifique el esfuerzo de mantenimiento. Con SQL tendrás una arquitectura más limpia, centralizada y fácil de respaldar.

Aquí tienes cómo estructurar este modelo en Django para que sea eficiente y seguro:

1. El Modelo de Datos (SQL)

En lugar de una tabla de caché genérica, crearemos una tabla de **Tokens de Firma** que nos permita tener una auditoría clara.

Python



```
from django.db import models
from django.utils import timezone
from datetime import timedelta
import secrets

class OTPSignatureToken(models.Model):
    # Relación con el usuario y la referencia del documento del otro servicio
    user_identifier = models.EmailField()
    document_id = models.UUIDField()

    # El código OTP (se guarda el hash para mayor seguridad)
    otp_code_hash = models.CharField(max_length=128)

    # Gestión de expiración e intentos
    created_at = models.DateTimeField(auto_now_add=True)
```

```

expires_at = models.DateTimeField()
attempts = models.IntegerField(default=0)
is_used = models.BooleanField(default=False)

def save(self, *args, **kwargs):
    if not self.expires_at:
        # Definimos 5 minutos de validez por defecto
        self.expires_at = timezone.now() + timedelta(minutes=5)
    super().save(*args, **kwargs)

@property
def is_valid(self):
    return not self.is_used and timezone.now() < self.expires_at and self.attempt

```

2. Flujo de Implementación en el Servicio de OTP

A. Generación

Cuando el Servicio de Firma solicite un código:

1. Generas un número aleatorio de 6 dígitos usando `secrets.choice`.
2. Hasheas el código (usando `make_password` de Django o `hashlib`).
3. Guardas el registro en la base de datos.
4. **Disparas el envío del correo** (pasándole el código en texto plano al servicio de Email).
5. Retornas al Servicio de Firma el `id` del token para que sepa cuál validar después.

B. Validación

Cuando el usuario ingresa el código:

1. Buscas el registro por su `id`.
2. Verificas si `is_valid` es verdadero.
3. Comparas el código ingresado con el hash.
4. **Si falla:** Incrementas el contador de `attempts`.
5. **Si acierta:** Marcas `is_used = True` y devuelves éxito.

3. Estrategia de "Limpieza" (The Cleanup)

Como mencionamos, el "contra" de SQL es que los registros expirados se quedan ahí. Para un volumen bajo, tienes dos opciones:

1. **Lazy Cleanup:** Cada vez que alguien pida un OTP nuevo, borras los que ya expiraron: `OTPSignatureToken.objects.filter(expires_at__lt=timezone.now()).delete`

()

2. **Management Command:** Un pequeño script de Django que ejecutas una vez al día con un cronjob para limpiar la tabla.

4. Ventajas para tu Proyecto de Firma

- **Trazabilidad Total:** Si en un año alguien cuestiona una firma, tienes en tu base de datos exactamente a qué hora se generó el OTP, cuántos intentos tomó y cuándo se validó, todo vinculado al ID del documento.
- **Consistencia:** No hay riesgo de que el OTP se pierda por un reinicio de servidor (cosa que sí pasaría con un Redis mal configurado).
- **Fácil Debugging:** Puedes entrar al admin de Django y ver qué está pasando con los códigos en tiempo real.

Resumen de la Arquitectura SQL

¿Te parece bien este enfoque para comenzar? Si estás de acuerdo, lo siguiente sería definir cómo el **Servicio de OTP** notificará al **Servicio de Email** de forma sencilla.

desarrollo de una funcion para consultar un registro desde una api rest en python django

Para desarrollar una función que consulte un registro de OTP desde tu servicio independiente, lo ideal es utilizar la librería `requests` si estás haciendo la llamada desde otro microservicio (como el de Firma), o implementar un **ViewSet** en el servicio de OTP si lo que quieres es exponer el endpoint.

Asumiendo que estás en el **Servicio de Firma** y necesitas consultar la validez de un OTP al **Servicio de OTP**, aquí tienes cómo estructurar esa lógica de consulta:

1. Función de Consulta (Consumidor de API)

Esta función se encargaría de enviar el código que el usuario ingresó hacia el microservicio de OTP para verificar si es correcto.

```

import requests
from django.conf import settings

def consultar_validez_otp(transaction_id, codigo_usuario):
    """
    Consulta al microservicio de OTP si un código es válido.
    """
    url = f"{settings.OTP_SERVICE_URL}/api/otp/verify/"
    payload = {
        "transaction_id": transaction_id,
        "code": codigo_usuario
    }

    try:
        response = requests.post(url, json=payload, timeout=5)
        response.raise_for_status() # Lanza error si el status es 4xx o 5xx

        data = response.json()
        return data.get('valid', False)

    except requests.exceptions.RequestException as e:
        # Log del error (importante para auditoría de firma)
        print(f"Error de conexión con servicio OTP: {e}")
        return False

```

2. Endpoint de Consulta (En el Servicio de OTP)

En el lado del **Servicio de OTP** (el que usa la base de datos SQL), necesitas una vista que reciba esa petición y busque en la tabla que definimos antes.

Python



```

from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework import status
from .models import OTPSignatureToken
from django.contrib.auth.hashers import check_password

class VerifyOTPView(APIView):
    def post(self, request):
        transaction_id = request.data.get('transaction_id')
        input_code = request.data.get('code')

        try:
            # 1. Buscar el registro en SQL

```

```

otp_record = OTPSignatureToken.objects.get(id=transaction_id)

# 2. Validar reglas de negocio (Expiración, intentos, ya usado)
if not otp_record.is_valid:
    return Response({"valid": False, "message": "OTP expirado o inválido"}

# 3. Verificar el hash del código
if check_password(input_code, otp_record.otp_code_hash):
    otp_record.is_used = True
    otp_record.save()
    return Response({"valid": True})
else:
    otp_record.attempts += 1
    otp_record.save()
    return Response({"valid": False, "message": "Código incorrecto"}, status=400)

except OTPSignatureToken.DoesNotExist:
    return Response({"valid": False, "message": "Transacción no encontrada"}, status=404)

```

3. Puntos clave de esta implementación

Manejo de Seguridad

- **Hasheo:** Nota que usamos `check_password`. Esto es porque en la base de datos SQL no guardamos el "123456", sino su versión cifrada.
- **Timeouts:** En la función de consulta siempre define un `timeout`. No quieres que tu proceso de firma se quede congelado si el servicio de OTP tarda en responder.

Idempotencia

Al marcar `is_used = True` inmediatamente después de un acierto, garantizas que ese mismo registro no pueda ser consultado con éxito una segunda vez.

Recomendación para el envío de correo

Dado que mencionaste que quieres los proyectos separados, cuando la función de **Generación** cree el registro en SQL, debe disparar otra petición similar a esta pero hacia tu **Servicio de Email**, enviando el código en texto plano (solo en ese momento) para que el usuario lo reciba.

¿Te gustaría que viéramos cómo estructurar el JSON que el Servicio de OTP le enviaría al de Email para completar el círculo?

Perfecto. Para cerrar el círculo de comunicación entre tus microservicios, el **Servicio de OTP** debe actuar como un "emisor de órdenes" hacia el **Servicio de Email**.

Aquí tienes el diseño del contrato (JSON) y la lógica para que el código llegue al usuario final sin que el Servicio de OTP se cargue con la responsabilidad técnica de configurar servidores SMTP o plantillas de diseño.

1. El Contrato JSON (OTP → Email)

Este es el objeto que el Servicio de OTP enviará al Servicio de Email. Es minimalista para mantener la independencia:

JSON



```
{
  "service_key": "otp_system_secret_token",
  "recipient": "usuario@ejemplo.com",
  "template": "signature_otp_delivery",
  "context": {
    "otp_code": "529301",
    "expires_in": "5",
    "user_name": "Francisco"
  }
}
```

2. Implementación de la función de disparo

Dentro de tu vista de **Generación de OTP** en Django, después de guardar el registro en la base de datos SQL, ejecutarías esta lógica:

Python



```
import requests
import logging

logger = logging.getLogger(__name__)

def disparar_envio_email(email_destino, codigo, nombre_usuario):
```



```

"""
Notifica al microservicio de Email que debe enviar un código.
"""

url = "https://tu-servicio-email.com/api/send/"
payload = {
    "recipient": email_destino,
    "template": "otp_template",
    "context": {
        "otp_code": codigo,
        "user_name": nombre_usuario
    }
}

try:
    # Usamos un timeout corto para no bloquear el flujo
    response = requests.post(url, json=payload, timeout=3)
    response.raise_for_status()
    return True
except requests.exceptions.RequestException as e:
    # Importante: Si el correo falla, debemos decidir si cancelamos
    # la transacción o si permitimos un reintento manual.
    logger.error(f"Fallo al contactar servicio de Email: {e}")
    return False

```

3. El Flujo de Datos Completo

Para que visualices cómo se mueven los datos en esta arquitectura de 4 servicios:

1. **Servicio de Firma:** Recibe la orden de firmar y genera un `UUID` de transacción.
2. **Servicio de OTP:** Crea el registro en SQL, genera el código aleatorio y lo hashea.
3. **Servicio de Email:** Recibe el código en texto plano, lo inyecta en una plantilla HTML y lo envía vía SMTP/API (SendGrid, Mailgun, etc.).
4. **Usuario:** Recibe el correo y lo escribe en la interfaz del **Servicio de Visualización**.

4. Recomendaciones Finales de Seguridad

- **Tokens de Servicio:** El Servicio de Email debe validar que la petición viene realmente de tu Servicio de OTP (usando un `Authorization: Bearer <token>` en el header).
- **Logs de Auditoría:** En tu base de datos SQL del Servicio de OTP, te sugiero añadir un campo `email_notified_at` (`DateTimeField`). Actualízalo solo cuando el Servicio de Email te responda un `200 OK`. Así sabrás si el código no se validó porque el usuario no quiso o porque el correo nunca salió.
- **Uso de Celery:** Si en el futuro notas que el envío de correos ralentiza la generación del OTP, puedes mover la función `disparar_envio_email` a una tarea de **Celery** para que ocurra de forma asíncrona.

Con esta estructura, tienes un sistema de grado profesional, escalable y con responsabilidades claramente separadas. ¿Hay alguna parte del proceso de firma (el sellado del documento, por ejemplo) en la que te gustaría profundizar ahora?

quiero migrar mi servicio OTP de sqlite3 a postgres

Migrar de **SQLite3** a **PostgreSQL** es el paso natural cuando buscas profesionalizar tu servicio de OTP. PostgreSQL maneja mucho mejor la concurrencia (varias firmas al mismo tiempo) y permite una integridad de datos superior.

Aquí tienes la guía técnica para realizar la migración de forma segura en Django:

1. Preparación del Entorno

Primero, necesitas instalar el adaptador de base de datos para que Python pueda hablar con Postgres:

Bash



```
pip install psycopg2-binary
```

2. Configuración en `settings.py`

Debes cambiar la configuración de `DATABASES`. En lugar de apuntar a un archivo local, apuntarás al servidor de base de datos (que puede estar en tu mismo servidor o en un servicio como AWS RDS).

Python



```
DATABASES = {  
    'default': {  
        'ENGINE': 'django.db.backends.postgresql',  
        'NAME': 'otp_service_db',  
        'USER': 'tu_usuario',  
        'PASSWORD': 'tu_password',  
        'HOST': 'localhost', # O la IP de tu servidor Postgres  
        'PORT': '5432',
```

}

}

3. El Proceso de Migración de Datos

Si ya tienes registros en SQLite que no quieres perder (aunque en un servicio de OTP los datos suelen ser efímeros), sigue estos pasos:

A. Volcar los datos actuales (Dump)

Con la configuración de SQLite activa:

Bash



```
python manage.py dumpdata --exclude auth.permission --exclude contenttypes > datos_ot
```

B. Aplicar el esquema en Postgres

Cambia a la configuración de Postgres en `settings.py` y ejecuta:

Bash



```
python manage.py migrate
```

C. Cargar los datos (Load)

Bash



```
python manage.py loaddata datos_otp.json
```

4. Optimizaciones Específicas para el Servicio de OTP

Al pasar a Postgres, puedes aprovechar características que SQLite no tiene:

- **Índices GIN o B-Tree:** Asegúrate de que el campo `transaction_id` (o el ID que uses para buscar) tenga un índice. Postgres los maneja de forma muy eficiente.
- **Constraints de Tiempo:** Puedes usar `CheckConstraint` a nivel de base de datos para asegurar que `expires_at` sea siempre mayor que `created_at`.

- **Seguridad de Conexión:** Utiliza variables de entorno para las credenciales. Nunca las dejes en texto plano en el código.

5. Limpieza Automática (Cleanup) en Postgres

Como mencionamos que el volumen de documentos no es masivo, pero quieres orden, puedes usar un comando de administración de Django para limpiar los OTPs viejos.

Crea el archivo `management/commands/clear_expired_otps.py`:

Python



```
from django.core.management.base import BaseCommand
from django.utils import timezone
from your_app.models import OTPSignatureToken

class Command(BaseCommand):
    help = 'Borra los tokens de OTP que ya han expirado'

    def handle(self, *args, **options):
        count, _ = OTPSignatureToken.objects.filter(expires_at__lt=timezone.now()).delete()
        self.stdout.write(f'Se eliminaron {count} registros expirados.')
```

Luego, puedes programar un **Cron job** en tu servidor Linux que ejecute este comando una vez al día o cada hora:

Bash



```
0 * * * * python /ruta/al/proyecto/manage.py clear_expired_otps
```

6. Consideración para AWS

Ya que utilizas **AWS**, si decides usar **RDS (Relational Database Service)** para Postgres, recuerda configurar los **Security Groups** para permitir el tráfico en el puerto **5432** desde la instancia donde corre tu servicio de OTP.

¿Ya tienes instalado PostgreSQL en tu servidor o planeas utilizar un servicio gestionado como RDS?

por ahora voy a utilizar mi servicio en mi ambiente local

Excelente. Mantener el desarrollo en un ambiente local con **PostgreSQL** es una práctica recomendada (el famoso *Dev-Prod Parity*), ya que te asegura que lo que funcione en tu máquina se comporte igual cuando lo subas a AWS.

Aquí tienes los pasos exactos para configurar PostgreSQL localmente y conectar tu servicio de OTP:

1. Instalación de PostgreSQL (Local)

Si aún no lo tienes, el método más sencillo depende de tu sistema operativo:

- **Linux (Ubuntu/Debian):**

Bash



```
sudo apt update
sudo apt install postgresql postgresql-contrib
```

- **macOS:** Usando Homebrew: `brew install postgresql`
- **Windows:** Descargando el instalador oficial de [EnterpriseDB](#).

2. Crear la Base de Datos y el Usuario

Una vez instalado, debes entrar a la consola de Postgres (terminal) para crear el espacio para tu proyecto:

Bash



```
# Entrar como el usuario administrador de postgres
sudo -u postgres psql

# Dentro de la consola de psql:
CREATE DATABASE otp_service_db;
CREATE USER otp_user WITH PASSWORD 'tu_password_seguro';
GRANT ALL PRIVILEGES ON DATABASE otp_service_db TO otp_user;
\q
```

3. Configurar Django (settings.py)

Ahora, apunta Django a tu Postgres local. Es muy recomendable usar **variables de entorno** incluso en local, para que no subas tus contraseñas a GitHub/GitLab.

Python



```
import os

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': 'otp_service_db',
        'USER': 'otp_user',
        'PASSWORD': 'tu_password_seguro',
        'HOST': '127.0.0.1', # O 'localhost'
        'PORT': '5432',
    }
}
```

4. Ejecutar las Migraciones

Como estás cambiando de motor de base de datos (de un archivo **.sqlite3** a un servidor real), necesitas crear las tablas en Postgres:

Bash



```
python manage.py makemigrations
python manage.py migrate
```

5. Tip: Uso de **pgAdmin** o **DBeaver**

Para el desarrollo local de tu servicio de OTP, te sugiero instalar una herramienta visual como **pgAdmin** o **DBeaver**.

- Te permitirá ver la tabla **OTPSignatureToken** en tiempo real.
- Podrás verificar manualmente si los hashes se están guardando bien y si el campo **expires_at** tiene la zona horaria correcta (recuerda que Django usa UTC por defecto).

6. Siguiente Paso: Prueba de Integración

Ahora que tienes Postgres listo, lo ideal es que pruebes el flujo completo:

1. Llamar al endpoint de **Generar OTP**.
2. Ver en tu base de datos que el registro existe.
3. Intentar validar con un código erróneo y ver cómo aumenta el contador de `attempts`.

¿Quieres que te ayude a configurar un archivo `.env` para que el cambio entre tu ambiente local y el de producción en AWS sea automático en el futuro?

```
CommandError: Unable to serialize database: 'utf-8' codec  
can't decode byte 0xab in position 96: invalid start byte
```

Este error es muy común al migrar de **SQLite3** a **PostgreSQL** y suele ocurrir porque SQLite es muy permisivo con la codificación de caracteres, mientras que Postgres y Django esperan un `utf-8` estricto. El byte `0xab` indica que hay algún carácter especial (posiblemente una comilla latina «, una tilde o un símbolo de grado) grabado en una codificación distinta (como Latin-1 o Windows-1252).

Como tu servicio es de **OTP**, lo más probable es que el problema esté en los registros de auditoría o en nombres con tildes que se guardaron antes de la migración.

Aquí tienes los pasos para solucionar este bloqueo:

1. Localizar el dato corrupto (Opcional pero recomendado)

Si quieres saber exactamente dónde está el problema antes de limpiar, puedes intentar ejecutar el dump tabla por tabla hasta encontrar la que falla:

Bash



```
python manage.py dumpdata app_name.OTPSignatureToken --indent 2 > test.json
```

Si falla en esa, ya sabes que el problema está en tus registros de OTP.

2. Solución A: Limpiar y empezar de cero (Recomendada)

Dado que los códigos OTP son **temporales** y los que tienes en SQLite probablemente ya expiraron, la forma más rápida y limpia de migrar a Postgres local es **no migrar los datos**

viejos, sino solo la estructura.

1. Borra el archivo JSON que generaste antes (el que dio error).
2. Crea un dump solo de la configuración necesaria (usuarios/admin), ignorando los registros de OTP:

Bash



```
python manage.py dumpdata --exclude app_name.OTPSignatureToken --exclude content
```

3. Migra a Postgres como lo planeamos y carga solo ese archivo limpio.

3. Solución B: Reparar el archivo con Python

Si realmente necesitas esos datos, puedes usar un pequeño script para forzar la decodificación del archivo generado:

Python



```
# repara_json.py
import json

try:
    with open('datos_otp.json', 'rb') as f:
        content = f.read()

    # Intentamos decodificar ignorando errores o usando latin-1
    decoded_content = content.decode('latin-1')

    with open('datos_otp_fixed.json', 'w', encoding='utf-8') as f:
        f.write(decoded_content)
    print("Archivo reparado con éxito.")
except Exception as e:
    print(f"Error: {e}")
```

4. Configuración Preventiva en Postgres

Para evitar que esto vuelva a pasar en tu ambiente local con Postgres, asegúrate de que tu base de datos esté creada con `UTF8`. Puedes verificarlo en tu terminal de Postgres (`psql`):

SQL



```
\l -- Lista las bases de datos y su 'Encoding'
```

Si por alguna razón no dice `UTF8`, puedes recrearla así:

SQL



```
DROP DATABASE otp_service_db;  
CREATE DATABASE otp_service_db WITH ENCODING 'UTF8';
```

¿Por qué pasó esto?

Es muy probable que en alguna prueba previa, al ingresar un nombre o un campo de texto en el servicio de OTP (como el nombre del usuario para el correo), se haya pegado un carácter especial desde una fuente externa. SQLite lo aceptó "en bruto", pero al intentar convertirlo a JSON para la migración, Python falló al no reconocer el formato.

Mi consejo: Como es un servicio de OTP, simplemente ejecuta el `migrate` en Postgres y deja que la base de datos comience vacía. Los registros de SQLite ya no tienen utilidad técnica para validar firmas nuevas.

¿Prefieres que intentemos rescatar los datos de SQLite o te parece bien empezar con la base de datos de Postgres limpia?

CommandError: No installed app with label 'app_name'.

Este error ocurre porque en el comando de `dumpdata` que te sugerí, utilicé `'app_name'` como un marcador de posición (placeholder), pero Django necesita el nombre real de la carpeta donde están tus modelos.

Para solucionarlo, solo debes identificar el nombre de tu aplicación dentro del proyecto.

1. Identifica el nombre de tu App

Busca en tu estructura de carpetas el nombre del directorio donde está el archivo `models.py` de tu servicio de OTP.

- Si tu carpeta se llama `otp_service`, entonces el comando debe usar ese nombre.
- También puedes verificarlo en el archivo `apps.py` dentro de esa carpeta, en la línea que dice `name = 'nombre_de_tu_app'`.

2. Ejecuta el comando corregido

Si decides seguir mi recomendación de **no migrar los datos viejos** (para evitar el error de codificación UTF-8) y solo quieres migrar los usuarios administrativos, usa este comando:

Bash



```
python manage.py dumpdata auth.user --indent 2 > usuarios.json
```

Si prefieres intentar migrar todo, sustituye `app_name` por el nombre real (por ejemplo, si tu app se llama `core`):

Bash



```
python manage.py dumpdata core.OTPSignatureToken --indent 2 > datos_otp.json
```

3. El camino más rápido (Sin Dump/Load)

Dado que estás en un ambiente **local** y los códigos OTP de prueba en SQLite ya no te sirven, lo más sano para tu desarrollo es:

1. Asegurarte de que `settings.py` ya apunta a **PostgreSQL**.
2. Ejecutar las migraciones desde cero:

Bash



```
python manage.py migrate
```

3. Crear un nuevo superusuario para entrar al panel de administración en Postgres:

Bash



```
python manage.py createsuperuser
```

¿Por qué es mejor esto?

Al ser un servicio de microservicio para firmas, lo que más te importa es que la estructura de la base de datos sea idéntica a la de producción. Los registros de "prueba" que tenías en SQLite solo podrían traerte problemas de integridad o errores de codificación (como el que vimos antes) al pasar a un sistema más estricto como Postgres.

4. Verificación en Postgres

Una vez que ejecutes el `migrate`, puedes entrar a tu terminal de Postgres y verificar que las tablas se crearon correctamente:

Bash



```
sudo -u postgres psql -d otp_service_db
# Una vez dentro:
\dt # Esto listará todas las tablas creadas por Django
```

¿Lograste identificar el nombre de tu aplicación en el archivo `apps.py`?

Traceback (most recent call last):

```
File "C:\transtecnia\proyectos\transtecnia_otp\manage.py",
line 22, in <module>
    main()
```



Este error es un poco engañoso porque parece ser el mismo de antes, pero ahora ocurre al **momento de intentar conectar Django con PostgreSQL**. El culpable suele ser un carácter especial en la **contraseña**, en el **nombre de la base de datos** o incluso en la ruta de instalación de Python que Postgres no logra interpretar bajo `utf-8`.

Aquí tienes los pasos para limpiar la conexión y solucionar este `UnicodeDecodeError`:

1. Revisa las credenciales en `settings.py`

El byte `0xab` (que representa el carácter «) a menudo aparece cuando se copian y pegan contraseñas o nombres desde documentos (como Word o PDFs) que usan comillas tipográficas en lugar de comillas simples estándar.

- Asegúrate de que el `USER`, `PASSWORD` y `NAME` de la base de datos no tengan caracteres como tildes, comillas especiales o la letra "ñ".

- **Prueba esto:** Cambia temporalmente la contraseña de tu usuario en Postgres a algo ultra simple (ej: `password123`) y actualiza tu `settings.py` para descartar que sea el origen.

2. Verifica la codificación de la base de datos en Postgres

Si la base de datos se creó con una codificación distinta a UTF8 (como `WIN1252`), la conexión fallará al intentar negociar el protocolo.

Entra a tu terminal de Postgres y ejecuta:

SQL



```
-- Esto te mostrará la codificación de tus bases de datos
\l
```

Si tu base de datos `transtecnia_otp` (o el nombre que le hayas puesto) no dice **UTF8** en la columna "Encoding", bórrala y créala de nuevo:

SQL



```
DROP DATABASE transtecnia_otp;
CREATE DATABASE transtecnia_otp ENCODING 'UTF8';
```

3. Problema con variables de entorno o rutas

Veo en tu Traceback que estás trabajando en una ruta de Windows:

```
C:\transtecnia\proyectos\...
```

Si estás usando un archivo `.env`, asegúrate de que esté guardado con codificación **UTF-8** sin BOM. Si el archivo se guardó como `ANSI`, Python 3.13 (que es el que estás usando según el log) lanzará ese error al intentar leer la configuración.

4. El "Fix" de Psycopg2 en Windows

A veces, `psycopg2` intenta decodificar mensajes de error que vienen directamente de Windows en el idioma local (español) y falla.

Intenta forzar el idioma de los mensajes de Postgres a inglés añadiendo esto en tu `settings.py`, dentro de la configuración de la base de datos:

Python



```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': 'tu_db',
        'USER': 'tu_user',
        'PASSWORD': 'tu_password',
        'HOST': '127.0.0.1',
        'PORT': '5432',
        'OPTIONS': {
            'client_encoding': 'UTF8',
        },
    },
}
```

5. ¿Por qué ocurre en `makemigrations` ?

El comando `makemigrations` en las versiones modernas de Django intenta conectarse a la base de datos para verificar el historial de migraciones (`check_consistent_history`). Por eso, aunque no estés guardando datos aún, el error salta al intentar establecer el túnel de comunicación.

Resumen de acción inmediata:

1. Verifica que tu contraseña en `settings.py` no tenga caracteres extraños.
2. Asegúrate de que la base de datos en Postgres sea realmente **UTF8**.
3. Si persiste, intenta ejecutar `python manage.py migrate --run-syncdb` para ver si el error cambia.

¿Qué contraseña o nombre de base de datos estás intentando usar? (No pongas la real, solo dime si tiene algún símbolo especial).

quiero implementar un servicio de envío de correo al momento de generar el código otp, ese servicio está en django también y su endpoint es `http://localhost:7000/api/v1/send/?=` con un json como el 

Para integrar tu servicio de OTP con tu microservicio de correos en `localhost:7000`, debes realizar una petición **POST** desde la lógica de generación.

Considerando que estás trabajando en tu ambiente local, aquí tienes el código necesario para conectar ambos servicios de forma limpia.

1. Función de Envío (Integración)

Te recomiendo colocar esta función en un archivo de utilidades (ej. `utils.py`) dentro de tu proyecto de OTP. Esta función mapea tus datos al formato JSON exacto que espera tu microservicio.

Python



```
import requests
import logging

logger = logging.getLogger(__name__)

def enviar_correo_otp(email_destino, nombre_usuario, codigo_otp, referencia_externa):
    """
    Consume el microservicio de correos en el puerto 7000.
    """
    url = "http://localhost:7000/api/v1/send/" # Ajustado según tu endpoint

    payload = {
        "to_email": email_destino,
        "subject": "Tu Código de Firma Digital",
        "template_name": "base_email",
        "context_data": {
            "nombre": nombre_usuario,
            "mensaje_principal": f"Tu código OTP para la firma del documento es: {codigo_otp}"
        },
        "external_reference": referencia_externa
    }

    try:
        response = requests.post(url, json=payload, timeout=5)
        response.raise_for_status()
        return True
    except requests.exceptions.RequestException as e:
        logger.error(f"Error al conectar con el microservicio de correo: {e}")
        return False
```

2. Implementación en la Vista de Generación

En tu archivo `views.py` del servicio de OTP, invoca la función justo después de guardar el registro en PostgreSQL.

```
from rest_framework.views import APIView
from rest_framework.response import Response
from .utils import enviar_correo_otp
from .models import OTPSignatureToken
import secrets

class GenerarOTPView(APIView):
    def post(self, request):
        email = request.data.get('email')
        nombre = request.data.get('nombre')
        doc_id = request.data.get('document_id') # Referencia externa

        # 1. Generar código numérico
        codigo = "".join([str(secrets.randbelow(10)) for _ in range(6)])

        # 2. Guardar en Postgres (hasheado)
        otp_record = OTPSignatureToken.objects.create(
            user_identifier=email,
            document_id=doc_id,
            # Aquí deberías hashear el código antes de guardar
            otp_code_hash=make_password(codigo)
        )

        # 3. Disparar el correo
        exito_envio = enviar_correo_otp(email, nombre, codigo, str(otp_record.id))

        if exito_envio:
            return Response({"message": "OTP generado y enviado correctamente", "tran
        else:
            return Response({"error": "OTP generado pero fallo el envío de correo"},
```

3. Consideraciones Técnicas Importantes

- **El Endpoint:** En tu prompt incluiste `?=`. Normalmente, los endpoints de microservicios terminan en `/` y el JSON va en el cuerpo (body). Asegúrate de que el puerto 7000 esté aceptando peticiones `POST`.
- **Seguridad en Local:** Al estar en `localhost`, no tendrás problemas de certificados SSL, pero asegúrate de que ambos servicios estén corriendo simultáneamente.
- **Tratamiento del Código:** Fíjate que el código OTP solo existe en "texto plano" durante un segundo: cuando se genera y cuando se envía al microservicio de correo. Después de eso, el servicio de OTP solo conoce el hash (gracias a `make_password`), lo cual protege el código de accesos no autorizados a la base de datos.

4. Flujo de Comunicación en Local

¿Ya tienes corriendo el microservicio de correos en el puerto 7000 para hacer la primera prueba de envío?

si ya lo tengo corriendo, pero quiero saber donde coloco el archivo `utils.py`

El archivo `utils.py` debe ir dentro de la carpeta de la **aplicación** que maneja la lógica de los OTP (donde tienes tus `models.py` y `views.py`).

Asumiendo que tu proyecto se llama `transtecnia_otp` y tu aplicación se llama `otp_engine` (o similar), la estructura debería verse así:

1. Ubicación en la Estructura de Carpetas

Plaintext



```
transtecnia_otp/
├── manage.py
├── transtecnia_otp/           # Configuración global (settings.py, urls.py)
│   ├── settings.py
│   └── ...
└── otp_engine/              # Tu aplicación de OTP
    ├── migrations/
    ├── __init__.py
    ├── admin.py
    ├── apps.py
    ├── models.py
    ├── tests.py
    ├── utils.py              <-- AQUÍ LO CREAS
    └── views.py              # Aquí importarás las funciones de utils.py
```

2. Cómo importarlo en tu `views.py`

Una vez que hayas creado el archivo en esa ubicación, para usar la función en tus vistas debes importarla usando la notación de punto de Python.

Si estás dentro de `views.py` de la misma aplicación, se importa así:

Python



```
# En otp_engine/views.py
from .utils import enviar_correo_otp

class GenerarOTPView(APIView):
    def post(self, request):
        # ... tu lógica ...
        exito = enviar_correo_otp(email, nombre, codigo, referencia)
        # ...
```

3. ¿Por qué ponerlo ahí?

- **Modularidad:** Mantienes tus `views.py` limpios. La vista se encarga de recibir la petición y la "utilidad" se encarga de la comunicación externa.
- **Reutilización:** Si mañana creas otra vista (por ejemplo, una para re-enviar el código), solo tienes que llamar a la misma función en `utils.py`.
- **Mantenibilidad:** Si cambias de microservicio de correos o el endpoint cambia de puerto, solo modificas un archivo (`utils.py`) y no todas tus vistas.

4. Recomendación Final (Configuración)

Para evitar tener la URL `"http://localhost:7000/api/v1/send/"` escrita directamente en el código de `utils.py`, te sugiero moverla a tu `settings.py`:

En `settings.py`:

Python



```
EMAIL_SERVICE_URL = "http://localhost:7000/api/v1/send/"
```

En `utils.py`:

Python



```
from django.conf import settings

def enviar_correo_otp(...):
    url = settings.EMAIL_SERVICE_URL
    # ... resto del código
```

Esto hará que cuando pases de tu ambiente local a AWS, solo tengas que cambiar una línea en la configuración.

¿Quieres que revisemos el código de la vista (`views.py`) para asegurar que la integración con la base de datos Postgres y el envío de correo funcionen juntos?

me dio el siguiente error ==> Internal Server Error:
/api/v1/otp/generate/
Traceback (most recent call last):
File

El error `AttributeError: 'NoneType' object has no attribute 'id'` indica que la variable `otp_record` es **None** al momento de intentar acceder a su ID. Esto sucede porque el método `.create()` no devolvió una instancia del objeto, o falló silenciosamente antes de llegar a esa línea.

Revisando tu Traceback y el flujo que llevamos, aquí están las causas más probables y cómo corregirlas:

1. El problema: `otp_record` es None

En Django, `Model.objects.create(...)` normalmente devuelve la instancia creada. Si es `None`, es probable que estés usando un bloque `try/except` que captura un error y no asigna nada a la variable, o que el guardado falló.

2. Corrección en `views.py`

Asegúrate de que la creación del registro ocurra correctamente y que los nombres de las variables coincidan. Según tu log, estás pasando 5 argumentos a `enviar_correo_otp`, pero la función que definimos antes recibía 4.

Ajusta tu `views.py` de la siguiente forma:

Python



```
from django.contrib.auth.hashers import make_password
# ... otros imports ...

def post(self, request):
    # Obtener datos de los parámetros de la URL (Query Params) o del Body
```

```

email = request.query_params.get('email')
id_orig = request.query_params.get('origin_id')

# 1. Generar el código
import secrets
codeotp = "".join([str(secrets.randbelow(10)) for _ in range(6)])

# 2. Crear el registro en la base de datos
# USAMOS UN TRY PARA CAPTURAR ERRORES DE POSTGRES
try:
    otp_record = OTPSignatureToken.objects.create(
        user_identifier=email,
        document_id=id_orig, # 0 el campo que corresponda
        otp_code_hash=make_password(codeotp)
    )
except Exception as e:
    return Response({"error": f"Error al guardar en base de datos: {str(e)}"}, status=500)

# 3. Disparar el envío (Verifica que otp_record existe aquí)
if otp_record:
    # Aquí enviamos str(otp_record.id) como referencia externa
    exito_envio = enviar_correo_otp(email, "Estimado", codeotp, str(otp_record.id))

    if exito_envio:
        return Response({"message": "OTP generado", "id": otp_record.id}, status=200)

return Response({"error": "Fallo el proceso de OTP"}, status=500)

```

3. Ajuste en `utils.py` (La Firma de la Función)

El error también sugiere una discrepancia en cuántos datos envías. Asegúrate de que en `utils.py` la función acepte los argumentos que le pasas:

Python



```

# utils.py
def enviar_correo_otp(email_destino, nombre_usuario, codigo_otp, referencia_externa):
    # ... tu lógica de requests.post ...

```

4. Una observación sobre tu URL

En el log veo: `/api/v1/otp/generate/?email=faponte280426@yopmail.com&origin_id=ServicioFirma`.

Estás enviando los datos por URL (GET params) pero el método es POST.

- Si usas `request.data.get('email')` y el dato viene en la URL, te dará `None`.
- Para datos en la URL usa: `request.query_params.get('email')`.
- Para datos en el cuerpo JSON usa: `request.data.get('email')`.

¿Cómo lo corregimos rápido?

Verifica si en tu `views.py`, justo antes de `OTPSignatureToken.objects.create`, tienes los valores de `email` e `id_orig`. Si esos valores son `None` porque los estás buscando en el lugar equivocado, PostgreSQL podría dar un error de integridad (si los campos son obligatorios), haciendo que `otp_record` no se cree.

Error al conectar con el microservicio de correo: 401 Client Error: Unauthorized for url: http://localhost:7000/api/v1/send/

El error **401 Unauthorized** significa que tu microservicio de correos (el que corre en el puerto 7000) está recibiendo la petición, pero la está rechazando porque **falta una credencial de autenticación** o la que envías no es válida.

La mayoría de los microservicios de envío de correos requieren un token (como un `API Key` o un `Bearer Token`) para evitar que cualquier persona que conozca la URL pueda enviar correos masivos.

Aquí tienes cómo solucionarlo:

1. Identifica qué autenticación requiere tu Microservicio de Correo

Revisa el código o la configuración del proyecto en el puerto 7000. Lo más probable es que esté usando una de estas dos:

- **Token en el Header (Lo más común):** Espera un encabezado llamado `Authorization`.
- **API Key en el JSON:** Espera un campo como `"api_key": "xxx"` dentro del cuerpo que envías.

2. Actualiza tu función en `utils.py`

Si tu microservicio de correo usa una `API Key` sencilla en los encabezados (que es lo estándar), ajusta tu función así:

```
import requests
import logging

logger = logging.getLogger(__name__)

def enviar_correo_otp(email_destino, nombre_usuario, codigo_otp, referencia_externa):
    url = "http://localhost:7000/api/v1/send/"

    # AGREGA AQUÍ LAS CABECERAS DE AUTENTICACIÓN
    headers = {
        "Authorization": "Api-Key TU_TOKEN_SECRETO_AQUI", # Verifica el formato que e
        "Content-Type": "application/json"
    }

    payload = {
        "to_email": email_destino,
        "subject": "Tu Código de Firma Digital",
        "template_name": "base_email",
        "context_data": {
            "nombre": nombre_usuario,
            "mensaje_principal": f"Tu código OTP es: {codigo_otp}"
        },
        "external_reference": referencia_externa
    }

    try:
        # Pasamos los 'headers' en la petición
        response = requests.post(url, json=payload, headers=headers, timeout=5)
        response.raise_for_status()
        return True
    except requests.exceptions.RequestException as e:
        logger.error(f"Error al conectar con el microservicio de correo: {e}")
        return False
```

3. Si el microservicio de correo es tuyo y usa Django

Si el proyecto del puerto 7000 también es un Django con Django REST Framework, revisa la variable `DEFAULT_AUTHENTICATION_CLASSES` en su archivo `settings.py`.

- Si tiene `IsAuthenticated` en los permisos, **necesitas enviarle un token válido**.
- **Prueba rápida:** Si solo quieres probar localmente sin seguridad por ahora, en el microservicio de correo (puerto 7000), podrías cambiar el permiso de esa vista específica a `AllowAny`, aunque no es recomendable para producción.

4. Cómo probarlo rápidamente

Puedes usar **Postman** o **Insomnia** para enviar la misma petición al puerto 7000.

1. Si al enviar el JSON te da 401, confirma en la pestaña **Auth** de Postman qué tipo de llave necesita.
2. Una vez que logres que Postman te devuelva un `200 OK`, copia esos mismos encabezados a tu código de Python.

¿Sabes qué tipo de seguridad tiene implementada el microservicio de la API de correos? (Por ejemplo: SimpleJWT, TokenAuthentication, o una API Key personalizada).

me puedes ayudar a crear un manual de uso para este servicio

Claro que sí. Un buen manual técnico para un microservicio debe permitir que cualquier otro desarrollador entienda cómo integrarse con él sin necesidad de leer todo el código fuente.

Aquí tienes una estructura profesional para el **Manual de Integración del Servicio de OTP**.

Manual de Integración: Servicio de OTP (Transtecnia)

1. Descripción General

El **Servicio de OTP** es un microservicio independiente encargado de la generación, almacenamiento temporal y validación de códigos de un solo uso (One-Time Password) para procesos de firma digital.

El servicio utiliza **PostgreSQL** para la persistencia de auditoría y se comunica con un **Servicio de Email** externo para la entrega del código al usuario.

2. Flujo de Operación

1. **Solicitud:** Un sistema externo (ej. Servicio de Firma) solicita un OTP.
2. **Generación:** El servicio genera un código de 6 dígitos, guarda su hash en la DB y define una expiración de 5 minutos.
3. **Entrega:** El servicio notifica automáticamente al microservicio de correo (Puerto 7000) para enviar el código al usuario.

4. **Validación:** El usuario ingresa el código y el sistema externo consulta la validez del mismo.

3. Especificación de la API

A. Generar OTP

Crea una nueva transacción de firma y dispara el envío del correo.

- **URL:** `/api/v1/otp/generate/`
- **Método:** `POST`
- **Parámetros de URL (Query Params):**
 - `email` (string): Correo del destinatario.
 - `origin_id` (string): Identificador del documento o proceso de origen.

Ejemplo de Petición:

```
POST http://localhost:8000/api/v1/otp/generate/?  
email=usuario@ejemplo.com&origin_id=DOC-12345
```

Respuesta Exitosa (201 Created):

JSON



```
{  
  "message": "OTP generado y enviado correctamente",  
  "transaction_id": "550e8400-e29b-41d4-a716-446655440000"  
}
```

B. Verificar OTP

Valida si el código ingresado por el usuario es correcto y no ha expirado.

- **URL:** `/api/v1/otp/verify/`
- **Método:** `POST`
- **Cuerpo (JSON):**

JSON



```
{  
  "transaction_id": "550e8400-e29b-41d4-a716-446655440000",  
}
```

```
"code": "123456"
```

```
}
```

Respuestas Posibles:

- **200 OK:** `{"valid": true}` (El código es correcto y se marca como usado).
- **401 Unauthorized:** `{"valid": false, "message": "Código incorrecto"}` (Aumenta el contador de intentos).
- **400 Bad Request:** `{"valid": false, "message": "OTP expirado o ya utilizado"}`.

4. Reglas de Negocio y Seguridad

- **Vigencia:** Los códigos expiran automáticamente a los **5 minutos** de su creación.
- **Intentos:** Se permite un máximo de **3 intentos fallidos**. Al tercer error, el OTP se invalida permanentemente.
- **Uso Único:** Una vez que un OTP es validado con éxito, queda invalidado para futuros usos (*Burn on use*).
- **Almacenamiento:** Los códigos nunca se guardan en texto plano; se utiliza el algoritmo de hash de Django (`PBKDF2`).

5. Configuración del Entorno (Variables de Entorno)

Para que el servicio funcione correctamente, deben estar configuradas las siguientes variables en el servidor:

Variable	Descripción	Ejemplo
<code>DB_NAME</code>	Nombre de la DB Postgres	<code>transtecnia_otp</code>
<code>EMAIL_SERVICE_URL</code>	Endpoint del servicio de correos	<code>http://localhost:7000/api/v1/send/</code>
<code>EMAIL_SERVICE_TOKEN</code>	Token de autorización para el puerto 7000	<code>Tu-Token-Aqui</code>
...		

6. Mantenimiento (Limpieza de Base de Datos)

Para evitar que la tabla de tokens crezca indefinidamente, existe un comando de limpieza:

Bash



```
python manage.py clear_expired_otps
```

Se recomienda programar este comando mediante un CRON Job cada 24 horas.

¿Te gustaría que añadiera una sección sobre cómo manejar los errores más comunes (como el 401 que vimos antes) o prefieres incluir un ejemplo de cómo llamarlo desde JavaScript/Postman?

si me gustaria añadir una seccion de como manejar los errores mas comunes y tambien un ejemplo de como llamarlo desde otro proyecto en django o postman

Excelente idea. Completar el manual con estas secciones lo convierte en una herramienta "autosuficiente" para cualquier desarrollador que se sume al proyecto.

Aquí tienes las secciones adicionales para tu manual:

7. Manejo de Errores Comunes

En un entorno de microservicios, las fallas suelen ocurrir en la comunicación. Aquí cómo identificar y resolver los problemas más frecuentes:

Error HTTP	Significado	Causa Probable	Solución
401 Unauthorized	Error de Autenticación	El token de servicio en <code>utils.py</code> no coincide con el del puerto 7000.	Verifica el header <code>Authorization</code> en el microservicio correos.
404 Not Found	Recurso no encontrado	El <code>transaction_id</code> no existe en la base de datos PostgreSQL.	Verifica que el ID enviado en el request sea el que debes <code>generate</code> .

Error HTTP	Significado	Causa Probable	Solución
500 Internal Error	Error del Servidor	Falla en la conexión a la DB Postgres o error de codificación UTF-8.	Revisa los logs del servicio (Traza) y verifica que los datos estén en UTF8.
ConnectionError	Servicio Caído	El microservicio de correo (puerto 7000) no está encendido.	Asegúrate de que ambos servicios estén corriendo en el entorno de desarrollo.
...			

8. Guía de Uso (Clientes Externos)

A. Desde Postman

Para probar el flujo manualmente sin programar:

- Generar:** * Método: `POST`
 - URL: `http://localhost:8000/api/v1/otp/generate/`
 - Params: `email=tu@correo.com`, `origin_id=PRUEBA_01`
- Validar:**
 - Método: `POST`
 - URL: `http://localhost:8000/api/v1/otp/verify/`
 - Body (Raw JSON):

JSON



```
{
  "transaction_id": "ID_RECIBIDO",
  "code": "123456"
}
```

B. Desde otro Proyecto Django (Servicio de Firma)

Si quieres llamar al OTP desde otro de tus proyectos, usa la librería `requests`. Es la forma más limpia de hacerlo:

```
import requests

def solicitar_firma_otp(email_usuario, id_documento):
    """
    Llamada desde el Servicio de Firma hacia el Servicio de OTP
    """
    # 1. Solicitar generación
    url_gen = "http://localhost:8000/api/v1/otp/generate/"
    params = {'email': email_usuario, 'origin_id': id_documento}

    try:
        response = requests.post(url_gen, params=params, timeout=5)
        if response.status_code == 201:
            return response.json().get('transaction_id')
        return None
    except requests.exceptions.RequestException:
        return "Error de conexión"

def validar_codigo_usuario(uuid_transaccion, codigo_ingresado):
    """
    Valida el código que el usuario escribió en el frontend
    """
    url_val = "http://localhost:8000/api/v1/otp/verify/"
    payload = {
        "transaction_id": uuid_transaccion,
        "code": codigo_ingresado
    }

    response = requests.post(url_val, json=payload)
    return response.json().get('valid', False)
```

9. Monitoreo Rápido (Consola)

Si necesitas verificar qué está pasando "detrás de cámaras" en tu ambiente local:

- **Ver logs de Django:** Mira la terminal donde corres `python manage.py runserver`.
- **Ver correos enviados:** Si tu microservicio de correo tiene un panel de logs, verifícalo ahí.
- **Consultar la DB:** ```sql`
-- Ver últimos 5 OTPs generados
`SELECT user_identifier, created_at, attempts, is_used`
`FROM api_otp_otpsignaturetoken`
`ORDER BY created_at DESC LIMIT 5;`

Con esto, tu manual está completo y listo para ser entregado. Cubre desde la arquitectura hasta la resolución de problemas técnicos. ¿Hay algún otro detalle de la lógica de negocio (como aumentar el tiempo de expiración o cambiar el formato del código) que quieras ajustar?

tambien me puedes ayudar con manual tecnico

Claro que sí, Francisco. Mientras que el manual de uso está orientado a quien **consume** el servicio, el **Manual Técnico** está diseñado para el administrador o desarrollador que debe **mantener, desplegar o modificar** el código.

Aquí tienes la estructura para el Manual Técnico de tu servicio de OTP:

Manual Técnico: Microservicio de OTP (Backend)

1. Stack Tecnológico

- **Lenguaje:** Python 3.13
- **Framework:** Django 5.x / Django REST Framework (DRF)
- **Base de Datos:** PostgreSQL (Producción/Local)
- **Librerías Clave:** * `psycopg2-binary` (Conector Postgres)
 - `requests` (Comunicación entre microservicios)
 - `django-environ` (Gestión de variables de entorno)

2. Arquitectura de Datos

El modelo principal es `OTPSignatureToken`. A diferencia de un sistema estándar, este no utiliza `id` numérico secuencial por seguridad, sino `UUID`.

Diccionario de Datos: Tabla `api_otp_otpsignaturetoken`

Campo	Tipo	Descripción
<code>id</code>	UUID	Clave primaria. Identificador único de la transacción.
<code>user_identifier</code>	Email	Correo electrónico asociado a la firma.
<code>otp_code_hash</code>	String (128)	Hash del código (PBKDF2). Nunca texto plano.
<code>document_id</code>	String	Referencia externa del documento a firmar.
<code>attempts</code>	Integer	Contador de fallos (Máx. 3).
<code>is_used</code>	Boolean	Flag de redención única.
<code>expires_at</code>	DateTime	Marca de tiempo para invalidación automática.
...		

3. Lógica de Seguridad (Criptografía)

El servicio no almacena el código generado. Utiliza la función `make_password` de Django:

- Se genera un `SECRET_KEY` aleatorio de 6 dígitos.
- Se aplica un *salt* y se hashea antes de guardar en Postgres.
- La validación se realiza mediante `check_password(input_code, stored_hash)`.

4. Configuración del Ambiente de Desarrollo

Instalación de Dependencias

Bash



```
pip install -r requirements.txt
```

Configuración de Postgres en Windows

Es imperativo que la base de datos esté creada con codificación **UTF-8** para evitar el error `UnicodeDecodeError` durante la migración de esquemas:

SQL



```
CREATE DATABASE transtecnia_otp ENCODING 'UTF8';
```

5. Endpoints Técnicos y Serialización

El servicio utiliza `APIView` para un control total sobre la lógica de negocio sin depender de la magia de los `ModelViewSet`.

- **Validación de Datos:** Se recomienda que los campos `email` y `origin_id` sean saneados antes de la inserción para evitar inyecciones.
- **Headers de Respuesta:** Todas las respuestas exitosas devuelven un `Content-Type: application/json`.

6. Procedimiento de Despliegue (Deployment)

Para mover este servicio a **AWS (Ubuntu)**:

1. **Configurar el entorno virtual:** `python -m venv venv`.
2. **Variables de Entorno:** Configurar un archivo `.env` fuera del control de versiones (Git).
3. **Migraciones:** `python manage.py migrate`.
4. **Servidor de Aplicaciones:** Se recomienda el uso de **Gunicorn** detrás de un proxy inverso como **Nginx**.

7. Troubleshooting (Diagnóstico de Fallas)

- **Error 401 en Envío de Email:** Revisar la función `enviar_correo_otp` en `utils.py`. Verificar que el token de autorización coincida con el requerido por el microservicio en el puerto 7000.
- **Lentitud en Generación:** Si el servicio de correo tarda mucho en responder, la API de OTP se bloqueará. Se recomienda pasar el envío a una tarea asíncrona (Celery) si el volumen de usuarios aumenta.
- **Error de Conexión DB:** Verificar que el servicio de PostgreSQL esté iniciado (`services.msc` en Windows).

Con estos dos manuales (Uso y Técnico), tu proyecto de microservicio de OTP está debidamente documentado para estándares profesionales. ¿Necesitas que detalle alguna configuración específica de **AWS** o de **Nginx** para cuando decidas subirlo a producción?

quiero agregar una capa de seguridad antes de generar un código OTP, me deben de enviar un token, este token debe de venir en un json y hay que decodificarlo, en este token viene el `client_id`, scopes y fecha de expiración, me puedes

Para incorporar esta capa de seguridad, lo más profesional es utilizar **JWT (JSON Web Tokens)**. Esto te permite decodificar el token, verificar su integridad (que no haya sido manipulado) y comprobar la fecha de expiración de forma automática.

Necesitarás instalar la librería **PyJWT** :

Bash



```
pip install pyjwt
```

Aquí tienes el código actualizado. He añadido una función de utilidad para la decodificación y la lógica dentro de tu `post` :

1. Actualización de la Vista **GenerateOTP**

Python



```
import jwt # Necesitas instalar PyJWT
from django.conf import settings
from datetime import datetime

# ... tus otros imports ...

@method_decorator(csrf_exempt, name='dispatch')
class GenerateOTP(APIView):
    authentication_classes = []
    permission_classes = [HasClientApiKey]
```

```

def post(self, request):
    # 1. CAPA DE SEGURIDAD: Validación del Token recibido en el JSON
    token_recibido = request.data.get('auth_token') # Asumimos que el JSON trae e

    if not token_recibido:
        return Response({"error": "Se requiere un token de seguridad para esta op

    try:
        # Decodificamos el token usando tu SECRET_KEY de Django
        # 'exp' se valida automáticamente si está presente en el payload
        payload = jwt.decode(token_recibido, settings.SECRET_KEY, algorithms=['HS

        # Extraemos los datos del token
        client_id_token = payload.get('client_id')
        scopes = payload.get('scopes')

        # Validación extra: Verificar si el scope permite generar OTP
        if 'otp_write' not in scopes:
            return Response({"error": "El token no tiene permisos suficientes (sc

    except jwt.ExpiredSignatureError:
        return Response({"error": "El token de seguridad ha expirado"}, status=40
    except jwt.InvalidTokenError:
        return Response({"error": "Token de seguridad inválido o corrupto"}, stat

    # --- CONTINÚA TU LÓGICA ORIGINAL ---
    cliente_solicitante = request.authorized_client
    leng_cod_otp = settings.OTP_CONFIG['CODE_LENGTH']

    serializer = OTPGenerateSerializer(data=request.data)
    if serializer.is_valid():
        email = serializer.validated_data['email'].lower().strip()
        id_orig = serializer.validated_data['app_source']
        ip_address = request.META.get('REMOTE_ADDR')

        # Verificación de bloqueo (Tu lógica existente)
        otp_record = OTPTransaction.objects.filter(
            user_email=email,
            app_source=id_orig,
            status='blocked'
        ).order_by('-blocked_until').first()

        if otp_record and timezone.now() < otp_record.blocked_until:
            return Response({"error": "Este correo esta suspendido temporalmente,

        # Generación de código
        caracteres = string.ascii_letters + string.digits
        codeotp = ''.join(random.choices(caracteres, k=leng_cod_otp))

```



```

nuevo_otp = OTPTransaction.objects.create(
    client=cliente_solicitante,
    user_email=email,
    otp_code=make_password(codeotp),
    app_source=id_orig,
    ip_address=ip_address,
    status='pending'
)

# Disparar el correo
exito_envio = enviar_correo_otp(email, "Estimado", codeotp, id_orig)

if exito_envio:
    return Response({
        "message": f"OTP generado, correo enviado exitosamente a {nuevo_o",
        "otp_code": codeotp,
        "created_at": nuevo_otp.created_at.strftime("%Y-%m-%d %H:%M:%S"),
        "status OTP": nuevo_otp.status,
        "status": "Ok"
    }, status=status.HTTP_201_CREATED)
else:
    return Response({"error": "OTP generado pero fallo el envío de correo

return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

```

2. Manual Técnico (Actualización)

Debes agregar esta sección a tu manual para que los otros desarrolladores sepan cómo enviarte el token.

Sección: Seguridad por Token de Aplicación (Payload)

Para consumir el endpoint de generación, el cuerpo de la petición debe incluir un campo `auth_token` generado con el algoritmo **HS256**.

Estructura del Payload del Token:

JSON



```

{
  "client_id": "nombre_app_firma",
  "scopes": ["otp_write", "signature_read"],
  "exp": 1714935547
}

```

Ejemplo de Petición (Body):

JSON



```
{  
  "email": "faponte@yopmail.com",  
  "app_source": "ServicioFirma",  
  "auth_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."  
}
```

Puntos Clave de la Mejora:

- **Aislamiento:** Ahora, aunque alguien tenga la API Key (de `HasClientApiKey`), necesita poseer un token dinámico con los `scopes` correctos.
- **Integridad:** Si alguien intenta cambiar el `client_id` dentro del token, la firma fallará y el servicio devolverá un 401.
- **Validación de Tiempo:** `jwt.decode` verifica automáticamente que la fecha actual sea menor a `exp`.

Nota de Seguridad: Asegúrate de que el microservicio que genera este token use el mismo `settings.SECRET_KEY` que tu servicio de OTP, o definan una llave compartida específica en el `settings.py`.

me dio este error `ModuleNotFoundError: No module named 'core'`

Este error es muy común en Django y ocurre porque en el código que implementamos, intentamos importar algo desde una carpeta llamada `core`, pero en tu proyecto esa carpeta tiene otro nombre o simplemente no existe.

Normalmente sucede en la línea donde intentas importar el modelo `OTPTransaction` o el serializador.

1. La Solución Directa

Debes cambiar la palabra `core` por el nombre real de tu aplicación (la carpeta donde están tus modelos). Según tus mensajes anteriores, parece que tu aplicación se llama `api_otp`.

Busca en la parte superior de tu archivo `views.py` y corrige las importaciones:

En lugar de:

Python



```
from core.models import OTPTransaction # <--- Esto da el error
from core.serializers import OTPGenerateSerializer
```

Cámbialo por:

Python



```
from api_otp.models import OTPTransaction
from api_otp.serializers import OTPGenerateSerializer
```

2. ¿Cómo saber el nombre correcto?

Si no estás seguro del nombre, mira la estructura de tus carpetas en la terminal o en tu editor:

- Busca la carpeta que contiene el archivo `models.py`.
- Ese nombre de carpeta es el que debes usar en el `from`.

Si los archivos están en la **misma carpeta** que tu `views.py`, puedes usar el punto para una importación relativa:

Python



```
from .models import OTPTransaction
from .serializers import OTPGenerateSerializer
```

3. Checklist de Verificación

Si después de cambiar el nombre el error persiste, revisa lo siguiente:

1. `settings.py`: Asegúrate de que el nombre de tu aplicación esté listado en `INSTALLED_APPS`.

Python



```
INSTALLED_APPS = [  
    ...  
    'api_otp',  
]
```

2. **Archivo `__init__.py`** : Verifica que dentro de la carpeta de tu aplicación exista un archivo llamado `__init__.py` (aunque esté vacío). Esto le dice a Python que esa carpeta es un módulo.

Resumen del cambio en tu código

En el bloque que agregamos para la seguridad del Token, asegúrate de que los imports de JWT también estén instalados:

Bash



```
pip install pyjwt
```

Y en tu `views.py`, la parte superior debería verse algo así:

Python



```
import jwt  
import string  
import random  
from django.conf import settings  
from django.utils import timezone  
from django.contrib.auth.hashers import make_password  
from rest_framework.views import APIView  
from rest_framework.response import Response  
from rest_framework import status  
  
# Usa el nombre real de tu app aquí  
from api_otp.models import OTPTransaction  
from api_otp.serializers import OTPGenerateSerializer  
from api_otp.utils import enviar_correo_otp
```

¿Cuál es el nombre de la carpeta donde tienes el archivo `models.py`? Con eso te puedo dar el import exacto.

Perfecto, entonces el nombre de tu aplicación es `api_otp`. El error ocurría porque Python buscaba una carpeta llamada `core` que no existe en tu estructura.

Aquí tienes el bloque de importaciones corregido y cómo debe quedar la lógica del token integrada en tu vista.

1. Importaciones Corregidas (`views.py`)

Asegúrate de que la parte superior de tu archivo se vea exactamente así:

Python



```
import jwt
import string
import random
from django.conf import settings
from django.utils import timezone
from django.utils.decorators import method_decorator
from django.views.decorators.csrf import csrf_exempt
from django.contrib.auth.hashers import make_password
from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework import status

# Importaciones desde tu app real
from api_otp.models import OTPTransaction
from api_otp.serializers import OTPGenerateSerializer
from api_otp.utils import enviar_correo_otp
```

2. Implementación de la Capa de Seguridad

He integrado la decodificación del JSON dentro de tu lógica. El sistema ahora espera que en el cuerpo del POST envíes una llave llamada `auth_token`.

Python



```
@method_decorator(csrf_exempt, name='dispatch')
class GenerateOTP(APIView):
    authentication_classes = []
```

```

permission_classes = [HasClientApiKey]

def post(self, request):
    # --- VALIDACIÓN DEL TOKEN DE SEGURIDAD ---
    token_recibido = request.data.get('auth_token')

    if not token_recibido:
        return Response({"error": "Falta token de seguridad (auth_token)"}, status=400)

    try:
        # Se decodifica usando la SECRET_KEY de tu settings.py
        payload = jwt.decode(token_recibido, settings.SECRET_KEY, algorithms=['HS256'])

        # Verificación de Scopes (Permisos específicos dentro del token)
        scopes = payload.get('scopes', [])
        if 'otp_write' not in scopes:
            return Response({"error": "El token no tiene permisos para generar OTP"}, status=403)

    except jwt.ExpiredSignatureError:
        return Response({"error": "El token de seguridad ha expirado"}, status=401)
    except jwt.InvalidTokenError:
        return Response({"error": "Token de seguridad inválido"}, status=401)

    # --- LÓGICA DE GENERACIÓN ---
    cliente_solicitante = request.authorized_client
    leng_cod_otp = settings.OTP_CONFIG['CODE_LENGTH']

    serializer = OTPGenerateSerializer(data=request.data)
    if serializer.is_valid():
        email = serializer.validated_data['email'].lower().strip()
        id_orig = serializer.validated_data['app_source']
        ip_address = request.META.get('REMOTE_ADDR')

        # Verificar si el correo está bloqueado
        otp_record = OTPTransaction.objects.filter(
            user_email=email,
            app_source=id_orig,
            status='blocked'
        ).order_by('-blocked_until').first()

        if otp_record and timezone.now() < otp_record.blocked_until:
            return Response({"error": "Correo suspendido temporalmente"}, status=403)

        # Generar código alfanumérico
        caracteres = string.ascii_letters + string.digits
        codeotp = ''.join(random.choices(caracteres, k=leng_cod_otp))

        nuevo_otp = OTPTransaction.objects.create(
            client=cliente_solicitante,

```

```

        user_email=email,
        otp_code=make_password(codeotp),
        app_source=id_orig,
        ip_address=ip_address,
        status='pending'
    )

    # Envío vía microservicio de correo (Puerto 7000)
    exito_envio = enviar_correo_otp(email, "Estimado", codeotp, id_orig)

    if exito_envio:
        return Response({
            "message": f"OTP enviado a {nuevo_otp.user_email}",
            "otp_code": codeotp, # Solo para pruebas en local
            "transaction_id": nuevo_otp.id,
            "status": "Ok"
        }, status=status.HTTP_201_CREATED)

    return Response({"error": "Error en el servicio de correo"}, status=500)

return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

```

3. Prueba Técnica (Postman)

Para probar que esto funciona, debes enviar un JSON como este:

JSON



```

{
  "email": "tu@correo.com",
  "app_source": "ServicioFirma",
  "auth_token": "PEGAR_AQUI_EL_JWT_GENERADO"
}

```

Tip Pro: Para generar un token de prueba rápido y validar que tu código decodifica bien, puedes entrar a jwt.io, pegar tu `SECRET_KEY` en la sección de firma y poner el payload de `scopes` y `client_id`.

¿Te gustaría que creáramos un pequeño script de Python para generar estos tokens de forma automática para tus pruebas?

estoy tratando de correr mi proyecto en mi servidor de aws y me da el siguiente error ==> Traceback (most recent call last):
File "/home/ubuntu/repos/otp/venv/lib/python3.13/site-

Este es el mismo síntoma que tuvimos en local, pero ahora en tu servidor AWS. El

`ModuleNotFoundError: No module named 'core'` en producción suele ocurrir por una de estas dos razones: o el código que subiste todavía tiene referencias a `core`, o la estructura de carpetas en el servidor no coincide con lo que Django espera.

Como ya sabemos que tu aplicación se llama `api_otp`, vamos a cazar ese "fantasma" de `core` en tu servidor Ubuntu:

1. El culpable más probable: `wsgi.py` o `asgi.py`

Cuando despliegas en un servidor, Django utiliza estos archivos para arrancar. Es muy probable que dentro de la carpeta de configuración de tu proyecto (donde está el `settings.py`), estos archivos intenten importar algo de `core`.

¿Qué hacer?

Revisa los archivos en `/home/ubuntu/repos/otp/nombre_de_tu_proyecto/`:

- `wsgi.py`
- `asgi.py`
- `manage.py`

Busca cualquier línea que diga `os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'core.settings')` y cámbiala por el nombre correcto de tu carpeta de configuración.

2. Buscar referencias antiguas con `grep`

En lugar de buscar archivo por archivo, usa el poder de la terminal de Linux para encontrar dónde se está escondiendo la palabra `core`:

Ejecuta este comando parado en `/home/ubuntu/repos/otp/`:

Bash



```
grep -r "core" .
```

Esto te listará todos los archivos y líneas donde aparece "core". Si ves que aparece en `INSTALLED_APPS`, en `ROOT_URLCONF` o en importaciones de modelos, ahí tienes el

problema.

3. Verificar `INSTALLED_APPS` en el servidor

A veces, al subir el código a AWS, el archivo `settings.py` del servidor no es el mismo que el de local. Asegúrate de que en el servidor diga:

Python



```
# settings.py en AWS
INSTALLED_APPS = [
    ...
    'api_otp', # Asegúrate de que NO diga 'core' aquí
]
```

4. La estructura de carpetas en AWS

Revisa que la carpeta de tu aplicación se llame igual que en local.

Ejecuta `ls -F` en `/home/ubuntu/repos/otp/`. Deberías ver algo así:

Plaintext



```
api_otp/      <-- Tu aplicación
nombre_proy/  <-- Carpeta de configuración (donde está settings.py)
manage.py
venv/
```

Si por alguna razón la carpeta en AWS se sigue llamando `core`, pero en tu código pusiste `api_otp`, cámbiale el nombre a la carpeta físicamente:

Bash



```
mv core api_otp
```

5. Limpiar archivos `.pyc`

A veces Python guarda versiones "compiladas" de los archivos viejos y eso confunde al servidor. Borra todos los residuos:

Bash



```
find . -name "*.pyc" -delete  
find . -name "__pycache__" -delete
```

Consejo de oro para AWS:

Si estás usando un servidor de aplicaciones como **Gunicorn**, recuerda que debes reiniciarlo cada vez que hagas un cambio en el código, ya que Gunicorn se queda con la versión vieja en memoria:

Bash



```
sudo systemctl restart gunicorn # 0 como lo tengas configurado
```

¿Lograste ver con el comando `grep` en qué archivo se quedó grabada la palabra `core` ?

el mismo proyecto me esta funcionando en mi equipo local
pero no en mi maquina de aws

Es frustrante cuando el famoso "en mi máquina funciona" se rompe al subir a la nube. El error `ModuleNotFoundError: No module named 'core'` en **AWS** (mientras en local funciona) indica casi siempre una **discrepancia de rutas** o de **variables de entorno** en el servidor Ubuntu.

Aquí tienes los 4 puntos críticos que debes revisar en tu instancia de AWS para solucionar esto:

1. Revisa el archivo `manage.py` en AWS

Cuando ejecutas `python manage.py runserver`, Django lee este archivo para saber dónde están los ajustes.

Abre el archivo en AWS (`nano manage.py`) y busca esta línea:

Python



```
os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'nombre_de_tu_proyecto.settings')
```

Si ahí dice `'core.settings'`, cámbialo al nombre real de la carpeta que contiene tu `settings.py`. En AWS, un simple error de dedo en esta ruta detiene todo el sistema.

2. El problema del PYTHONPATH

En Windows (local), el sistema a veces es más flexible con las rutas. En Linux (AWS), si ejecutas el comando desde la carpeta equivocada o si la estructura de carpetas cambió al clonar el repo, Python no encontrará los módulos.

Prueba esto en la terminal de AWS:

Bash



```
export PYTHONPATH=$PYTHONPATH:/home/ubuntu/repos/otp
python manage.py runserver
```

Si esto funciona, significa que necesitas agregar el directorio raíz a tu configuración de entorno.

3. Diferencias en settings.py (ALLOWED_HOSTS)

Tu Traceback menciona un error justo cuando Django intenta leer `settings.ALLOWED_HOSTS`.

En tu equipo local, probablemente tienes `DEBUG = True`. En AWS, si pusiste `DEBUG = False`, Django se vuelve estricto. Asegúrate de que en el `settings.py` de AWS tengas la IP de tu instancia:

Python



```
# settings.py en AWS
DEBUG = False # 0 True si estás probando apenas
ALLOWED_HOSTS = ['tu-ip-de-aws', 'localhost', '127.0.0.1']
```

4. ¿Cómo subiste el código? (Archivos ocultos)

Si usaste `git clone`, es posible que el archivo `.env` (donde guardamos las variables de entorno) no se haya subido porque suele estar en el `.gitignore`.

Si tu código intenta importar algo basado en una variable de entorno que está vacía o apunta a `core`, fallará.

- Revisa si existe el archivo `.env` en `/home/ubuntu/repos/otp/`.
- Si no existe, créalo manualmente con los datos correctos de Postgres y los nombres de las apps.

5. Verificación de la estructura real

Ejecuta este comando en la terminal de AWS y compáralo con lo que tienes en local:

Bash



```
find . -maxdepth 2 -not -path '*/.*'
```

Si ves una carpeta llamada `core/` en el listado de AWS que no debería estar ahí (o viceversa), ahí tienes el conflicto de nombres.

Un último consejo: Como estás usando `pyenv` y un `venv`, asegúrate de que el entorno virtual esté **activado** antes de correr el servidor:

Bash



```
source venv/bin/activate
```

¿Qué nombre tiene la carpeta que contiene el archivo `settings.py` en tu servidor AWS? (A veces le ponemos un nombre al repositorio y otro a la carpeta del proyecto Django).

transtecnia_otp

Entendido, el nombre de la carpeta de configuración es `transtecnia_otp`.

Si en local funciona pero en AWS no, y el error sigue mencionando `core`, es porque hay una referencia "quemada" (hardcoded) en los archivos de arranque de Django en el servidor. En Linux, la sensibilidad a mayúsculas/minúsculas y las rutas son más estrictas.

Sigue estos pasos exactos en tu terminal de AWS para corregirlo:

1. Corregir `manage.py`

Este archivo es el que lanzas para correr el servidor. Si apunta a `core.settings`, nunca encontrará tu configuración.

1. Ejecuta: `nano /home/ubuntu/repos/otp/manage.py`
2. Busca la línea:
`os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'core.settings')`
3. Cámbiala por:
`os.environ.setdefault('DJANGO_SETTINGS_MODULE',
'transtecnia_otp.settings')`
4. Guarda (Ctrl+O) y sal (Ctrl+X).

2. Corregir `wsgi.py` y `asgi.py`

Aunque estés usando `runserver` para probar, estos archivos también deben estar correctos.

1. Entra a la carpeta: `cd /home/ubuntu/repos/otp/transtecnia_otp/`
2. Edita **ambos** archivos (`nano wsgi.py` y `nano asgi.py`).
3. Cambia cualquier mención de `'core.settings'` por `'transtecnia_otp.settings'`.

3. Verificar `ROOT_URLCONF` en `settings.py`

Si Django logra cargar el archivo de configuración pero este dice que las URLs están en `core.urls`, fallará de inmediato.

1. Abre tu configuración: `nano /home/ubuntu/repos/otp/transtecnia_otp/settings.py`
2. Busca la variable `ROOT_URLCONF`.
3. Asegúrate de que diga:
`ROOT_URLCONF = 'transtecnia_otp.urls'`
4. Revisa también `WSGI_APPLICATION`:
`WSGI_APPLICATION = 'transtecnia_otp.wsgi.application'`

4. Limpiar la caché de Python en el Servidor

A veces el `ModuleNotFoundError` persiste porque Python mantiene archivos compilados `.pyc` que aún referencian a la estructura vieja. Borrémoslos todos:

Bash



```
cd /home/ubuntu/repos/otp/  
find . -name "*.pyc" -delete
```

```
find . -name "__pycache__" -delete
```

5. Prueba de ejecución

Asegúrate de tener tu entorno virtual activo y prueba correrlo:

Bash



```
source /home/ubuntu/repos/otp/venv/bin/activate  
python manage.py runserver 0.0.0.0:8000
```

¿Por qué en local te funciona?

Es muy probable que en tu máquina Windows tengas configurada una variable de entorno global o que el IDE (como PyCharm o VS Code) esté "ayudando" a Django a encontrar las rutas. En AWS, dependes exclusivamente de lo que esté escrito en el código.

Si después de estos cambios el error persiste, ejecuta `grep -r "core" .` dentro de la carpeta del proyecto y dime qué archivos aparecen. ¡Eso nos dará la ubicación exacta del intruso!

me esta dando este error al tratar de leer el token ==>
TypeError: 'HttpHeaders' object is not callable

Ese error `TypeError: 'HttpHeaders' object is not callable` ocurre porque estás intentando acceder al encabezado de la petición como si fuera una función o un método, cuando en **Django REST Framework** los headers se acceden como un diccionario o a través del objeto `request`.

Probablemente estás haciendo algo como `request.headers('Authorization')` (con paréntesis), lo cual genera el error.

Aquí tienes la forma correcta de extraer el token de seguridad dependiendo de cómo lo estés enviando:

1. Si el token viene en el JSON (como lo planeamos antes)

Si el token viene dentro del cuerpo del mensaje (body), debes usar `.get()` sobre `request.data`:

Python



```
# FORMA CORRECTA
token_recibido = request.data.get('auth_token')
```

2. Si el token viene en los Headers (Recomendado)

Si estás enviando el token en los encabezados de la petición (por ejemplo, en `Authorization`), la sintaxis correcta en Django es:

Python



```
# FORMA CORRECTA (Usando corchetes o .get, NO paréntesis)
token_recibido = request.headers.get('Authorization')

# Si el token viene precedido por "Bearer ", debes limpiarlo:
if token_recibido and token_recibido.startswith('Bearer '):
    token_recibido = token_recibido.split(' ')[1]
```

3. Aplicando la corrección a tu vista `GenerateOTP`

Ajusta el inicio de tu método `post` de esta manera para evitar el error:

Python



```
def post(self, request):
    # Opción A: Si lo mandas en el JSON
    token_recibido = request.data.get('auth_token')

    # Opción B: Si lo mandas en el Header (descomenta si prefieres esta)
    # auth_header = request.headers.get('Authorization')
    # token_recibido = auth_header.split(' ')[1] if auth_header else None

    if not token_recibido:
        return Response({"error": "No se proporcionó el token de seguridad"}, status=401)

    try:
        # Aquí continúa tu lógica de jwt.decode...
        payload = jwt.decode(token_recibido, settings.SECRET_KEY, algorithms=['HS256'])
        # ... rest of the code
```

¿Por qué falló en AWS y no en local?

Es muy probable que en tu máquina local estés usando una versión de Python o de Django que maneja los objetos de forma ligeramente distinta, o que la petición que hiciste desde Postman a local fuera diferente a la que estás lanzando contra AWS.

Resumen del error:

- **Mal:** `request.headers('Authorization')` ← Los objetos `HttpHeaders` no se pueden "llamar" como funciones.
- **Bien:** `request.headers.get('Authorization')` o `request.headers['Authorization']`.

¿Cómo estás enviando el token desde tu cliente (Postman o el otro microservicio), en el JSON o en los Headers?

no esta decodificado el token recibido con la siguiente instrccion ==> `payload = jwt.decode(token_recibido, settings.SECRET_KEY_AUTH, algorithms=['HS256'])`

Si la instrucción `jwt.decode` no está devolviendo el payload esperado o parece no estar "haciendo nada", generalmente se debe a una discrepancia entre cómo se firmó el token y cómo se está intentando leer.

Aquí tienes los puntos clave para asegurar que la decodificación funcione correctamente:

1. Verifica la Clave Secreta (`SECRET_KEY_AUTH`)

Para que `jwt.decode` funcione, la clave debe ser **exactamente la misma** que se usó para firmar el token.

- Si el token fue generado por otro microservicio, asegúrate de que ambos compartan el mismo valor de texto.
- Si estás usando `settings.SECRET_KEY_AUTH`, verifica en tu archivo `settings.py` de AWS que esa variable tenga el valor correcto y no sea `None` o una cadena vacía.

2. El tipo de dato de `token_recibido`

A veces, el token llega con el prefijo `Bearer`. Si le pasas la cadena completa a `jwt.decode`, fallará o no decodificará.

Asegúrate de limpiar el token antes de la instrucción:

Python



```
# Si el token viene de un Header Authorization: "Bearer xyz..."
if token_recibido.startswith('Bearer '):
    token_recibido = token_recibido.split(' ')[1]

# Ahora sí, decodifica
payload = jwt.decode(token_recibido, settings.SECRET_KEY_AUTH, algorithms=['HS256'])
```

3. Prueba de impresión (Debug)

Para saber qué está fallando, envuelve la instrucción en un bloque `try/except` y pide que te muestre el error exacto en la consola de AWS:

Python



```
try:
    payload = jwt.decode(token_recibido, settings.SECRET_KEY_AUTH, algorithms=['HS256'])
    print(f"DEBUG: Payload decodificado con éxito: {payload}")
except jwt.ExpiredSignatureError:
    print("ERROR: El token ya expiró.")
except jwt.InvalidSignatureError:
    print("ERROR: La firma no coincide. ¿Es correcta la SECRET_KEY_AUTH?")
except Exception as e:
    print(f"ERROR inesperado: {str(e)}")
```

4. ¿El token es realmente un JWT?

Puedes verificar si el token es válido (sin importar la firma por un momento) pegándolo en jwt.io.

- Si en la página web se ve el JSON (client_id, scopes, etc.), el token está bien construido.
- Si la página dice "Invalid Signature", confirma que la clave que pegas en la web sea la misma que tienes en tu `settings.py`.

5. Algoritmos

Confirmaste que usas `algorithms=['HS256']`. Asegúrate de que el servicio que genera el token no esté usando **RS256** (que requiere una llave pública `.pem` en lugar de una

cadena de texto).

Pregunta rápida: ¿El error que te da es `InvalidSignatureError` o simplemente el objeto `payload` te aparece vacío?

me acabo de dar cuenta que en el token al momento de recibirlo le agrego al inicio del token la Bearer

¡Ahí está el detalle! Ese es el comportamiento estándar de los encabezados de autenticación, pero la librería `PyJWT` no sabe qué hacer con esa palabra; ella solo espera la cadena de caracteres del token (el revoltijo de letras y puntos).

Cuando le pasas `"Bearer eyJhbG..."`, la función `decode` falla porque intenta procesar la palabra "Bearer" como parte del código criptográfico.

Aquí tienes la forma limpia y segura de removerlo en tu `views.py`:

La solución en tu código

Debes "limpiar" el token antes de intentar decodificarlo:

Python



```
# 1. Obtener el token del encabezado
auth_header = request.headers.get('Authorization')

if auth_header:
    # 2. Dividir la cadena por el espacio: ["Bearer", "eyJhbG..."]
    # y tomar solo la segunda parte [1]
    parts = auth_header.split()

    if len(parts) == 2 and parts[0].lower() == 'bearer':
        token_recibido = parts[1]
    else:
        # Si no tiene el formato correcto (ej: mandaron el token solo)
        token_recibido = parts[0]
else:
    return Response({"error": "No se proporcionó el encabezado Authorization"}, statu

# 3. Ahora sí, decodificar sin el "Bearer"
try:
```

```
payload = jwt.decode(token_recibido, settings.SECRET_KEY_AUTH, algorithms=['HS256'])
# ... resto de tu lógica ...
except jwt.DecodeError:
    return Response({"error": "El token está mal formado o no es un JWT válido"}, status=401)
```

¿Por qué se usa "Bearer"?

Para tu **Manual Técnico**, es bueno recordar que **Bearer** significa "Portador". Es un esquema de autenticación HTTP que le dice al servidor: *"Confía en quien porta este token"*.

- **Header:** `Authorization`
- **Value:** `Bearer <token>`

Tips adicionales para AWS:

1. **Case Insensitive:** Al hacer `.lower() == 'bearer'`, te aseguras de que si alguien envía `bearer` en minúsculas por error, tu código no se rompa.
2. **Split robusto:** Usar `split()` sin argumentos maneja automáticamente si hay uno o más espacios entre "Bearer" y el token, algo que pasa mucho cuando se copia y pega en Postman.

Prueba quitando el prefijo y verás que `jwt.decode` ahora sí te devolverá el JSON con los `scopes` y el `client_id` correctamente. ¡Ya casi lo tienes funcionando en producción!

ahora antes de enviar el correo de la otp quiero consumir un servicio de authservice para obtener el acces token; la url del servicio es ==> <https://authservice-qa.transtecniasa.cl/api/v1/authservice/service-clients/tokens>

Para implementar esto de forma profesional, lo ideal es que la lógica de obtención del token de autenticación resida dentro de tu archivo `utils.py`. De esta manera, tu función de envío de correo se vuelve "inteligente": primero pide permiso (token) y luego entrega el mensaje.

Aquí tienes la actualización para tu sistema:

1. Actualización de `utils.py`

Vamos a crear una función que gestione la autenticación con el `authservice` de Transtecnia y luego use ese token para hablar con el microservicio de correos.

Python



```
import requests
import logging
from django.conf import settings

logger = logging.getLogger(__name__)

def obtener_access_token():
    """
    Se autentica con el authservice-qa para obtener el Bearer Token.
    """
    url_auth = "https://authservice-qa.transtecniasa.cl/api/v1/authservice/service-cl"
    payload = {
        "client_id": "otp-service",
        "client_secret": "HrzqI0NafT-ZSedZitP6Skk80oSZedT9XnThenCkjoDHFwVwQnyjdBPtusj"
    }

    try:
        response = requests.post(url_auth, json=payload, timeout=10)
        response.raise_for_status()
        data = response.json()
        return data.get("access_token")
    except requests.exceptions.RequestException as e:
        logger.error(f"Error obteniendo token de authservice: {e}")
        return None

def enviar_correo_otp(email_destino, nombre_usuario, codigo_otp, referencia_externa):
    """
    Obtiene el token y luego envía el correo al puerto 7000.
    """
    # 1. Obtener el token de acceso primero
    token = obtener_access_token()
    if not token:
        logger.error("No se pudo enviar el correo porque falló la obtención del token")
        return False

    # 2. Configurar la petición al servicio de correos
    url_mail = "http://localhost:7000/api/v1/send/" # Tu servicio local

    headers = {
        "Authorization": f"Bearer {token}",
        "Content-Type": "application/json"
    }
```

```

payload = {
    "to_email": email_destino,
    "subject": "Tu Código de Firma Digital",
    "template_name": "base_email",
    "context_data": {
        "nombre": nombre_usuario,
        "mensaje_principal": f"Tu código OTP para la firma es: {codigo_otp}"
    },
    "external_reference": referencia_externa
}

try:
    response = requests.post(url_mail, json=payload, headers=headers, timeout=10)
    response.raise_for_status()
    return True
except requests.exceptions.RequestException as e:
    logger.error(f"Error al enviar correo: {e}")
    return False

```

2. Por qué este flujo es mejor

Al colocar `obtener_access_token` dentro de `enviar_correo_otp`, tu vista (`views.py`) no cambia. La vista sigue llamando a una sola función, manteniendo el código limpio y respetando la arquitectura de microservicios.

3. Consideración de Rendimiento (Opcional pero recomendado)

El token que recibes tiene un `expires_in: 900` (15 minutos). Pedir un token nuevo por cada correo que envíes puede hacer que tu API sea un poco lenta.

- **En Desarrollo/QA:** Está bien pedirlo cada vez.
- **En Producción:** Lo ideal sería guardar el token en el **Cache de Django** (Redis) por 14 minutos y solo pedir uno nuevo cuando el anterior expire.

4. Actualización del Manual Técnico

No olvides registrar este nuevo paso en tu documentación:

Dependencia de Autenticación Externa:

Antes de la entrega de mensajes, el servicio de OTP debe negociar un JWT con el `authservice` de QA utilizando las credenciales de `service-client`. El token resultante debe ser inyectado en el header `Authorization` de todas las peticiones salientes hacia el microservicio de correos.

Nota de Seguridad: Francisco, recuerda que el `client_secret` es información sensible. Ahora que lo pusimos en el código para que funcione en AWS, asegúrate de que cuando

pases a producción, este valor se lea desde una variable de entorno (`os.getenv`) y no esté escrito directamente en el archivo `.py` .

¿Quieres que te ayude a configurar el **Cache de Django** para que no tengas que pedir el token en cada envío de correo?

me ayudas a actualizar el manual tecnico y el manual de integracion

¡Claro que sí! Con los nuevos cambios (validación de JWT de entrada y la doble autenticación con el `authservice` para el envío), los manuales necesitan reflejar esta mayor complejidad técnica.

Aquí tienes las versiones actualizadas para **Transtecnia**.

1. Manual de Integración (Orientado al Consumidor)

Este manual es para los desarrolladores de otros servicios (como Firma Digital) que necesitan generar OTPs.

Descripción General

El servicio de OTP gestiona la seguridad de las transacciones mediante una doble capa de validación: **API Key** de cliente y **Token de Sesión (JWT)**.

Especificación de la API: Generar OTP

- **Endpoint:** `POST /api/v1/otp/generate/`
- **Headers Obligatorios:**
 - `X-API-Key` : Tu clave de cliente autorizada.
 - `Authorization` : `Bearer <JWT_TOKEN>` (Token generado por el cliente con scopes específicos).

Cuerpo de la Petición (JSON)

JSON



```
{
  "email": "usuario@ejemplo.com",
  "app_source": "ServicioFirma",
  "auth_token": "eyJhbGciOi... (Opcional si se envía en Header)"
}
```

Estructura del JWT requerido

Para que la petición sea aceptada, el `auth_token` debe estar firmado con la clave compartida y contener:

- `scopes` : Debe incluir `['otp_write']`.
- `exp` : Fecha de expiración válida.

Respuestas

- **201 Created:** OTP generado y en proceso de envío.
- **401 Unauthorized:** Token expirado o prefijo `Bearer` ausente.
- **403 Forbidden:** El token no tiene el scope `otp_write`.
- **404 Not Found:** El correo se encuentra bloqueado por exceso de intentos.

2. Manual Técnico (Orientado a Mantenimiento)

Este manual es para el equipo de infraestructura y backend que mantiene el servicio en AWS.

Arquitectura de Comunicación

El servicio actúa como un orquestador entre el cliente solicitante, el validador de identidad y el repartidor de correos.

Flujo Lógico de Generación

1. **Validación de Entrada:** Se verifica la `X-API-Key` mediante el middleware `HasClientApiKey`.
2. **Decodificación JWT:** Se extrae el `Bearer` token, se limpia el prefijo y se decodifica con `settings.SECRET_KEY_AUTH`.
3. **Persistencia:** Se genera el hash del código y se guarda en PostgreSQL (`api_otp_otpttransaction`).
4. **Autenticación Outbound:** El servicio consume `authservice-qa.transtecniasa.cl` usando `client_id` y `client_secret` para obtener un token de acceso de corta duración (15 min).

5. **Despacho:** Se envía el correo al Microservicio de Mensajería (Puerto 7000) inyectando el nuevo token en el header.

Configuración del Servidor (AWS Ubuntu)

Variables de Entorno Críticas (.env)

Es fundamental que estas variables existan en `/home/ubuntu/repos/otp/.env`:

- `DJANGO_SETTINGS_MODULE`: `transtecnia_otp.settings`
- `SECRET_KEY_AUTH`: Clave para decodificar tokens de entrada.
- `AUTH_SERVICE_CLIENT_ID`: `otp-service`
- `AUTH_SERVICE_SECRET`: (El secreto de 64 caracteres proporcionado).
- `EMAIL_SERVICE_URL`: `http://localhost:7000/api/v1/send/`

Comandos de Diagnóstico en AWS

Si el servicio falla en producción, verificar en este orden:

1. **Capa de Red:** `curl -I https://authservice-qa.transtecniasa.cl/...`
(Verificar salida a internet).
2. **Logs de Aplicación:** `tail -f /var/log/gunicorn/error.log` (O el log de Django).
3. **Caché de Python:** Si hay errores de "Module Not Found", ejecutar:
`find . -name "__pycache__" -delete`

Seguridad de Datos

- **Hashing:** Se utiliza `PBKDF2` para los códigos OTP.
- **Sanitización:** Todos los correos se convierten a minúsculas y se limpian espacios en blanco antes de la búsqueda en base de datos para evitar colisiones de registros.

Nota final para el equipo:

Cualquier cambio en el `client_secret` del `authservice` debe actualizarse inmediatamente en el servidor AWS y reiniciar el servicio Gunicorn para que los cambios surtan efecto.

¿Te parece que la sección de "Flujo Lógico" explica bien la diferencia entre el token que recibes y el que pides para enviar el correo? Es la parte más compleja del sistema.

quiero cambiar la validacion del api-key por una validacion de parametro

Perfecto, entiendo el cambio. Vamos a quitar la dependencia del permiso global `HasClientApiKey` (que busca el header `X-API-Key`) y en su lugar validaremos que la API Key o el identificador del cliente venga directamente como un parámetro dentro de la petición HTTP.

Para hacerlo de la forma más limpia y estándar en Django REST Framework, podemos recibir este parámetro de dos formas:

1. **Como Query Param** en la URL (ej: `/api/v1/otp/generate/?api_key=mi_clave`).
2. **Dentro del JSON** del body (ej: `{"api_key": "mi_clave", "email": "..."}`).

Lo más seguro y ordenado para un método `POST` es recibirlo dentro del **cuerpo del JSON** o como un **Query Param explícito**. Vamos a hacerlo de modo que busque el parámetro en el JSON para mantener toda la data unificada en el request.

Aquí tienes el código de tu vista actualizado con este cambio:

1. Actualización de la Vista `GenerateOTP` (`views.py`)

Python



```
import jwt
import string
import random
from django.conf import settings
from django.utils import timezone
from django.utils.decorators import method_decorator
from django.views.decorators.csrf import csrf_exempt
from django.contrib.auth.hashers import make_password
from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework import status

from api_otp.models import OTPTransaction
from api_otp.serializers import OTPGenerateSerializer
from api_otp.utils import enviar_correo_otp

@method_decorator(csrf_exempt, name='dispatch')
class GenerateOTP(APIView):
    # Deshabilitamos la autenticación por defecto de Django
```

```

authentication_classes = []
# Quitamos 'HasClientApiKey' de los permisos globales
permission_classes = []

def post(self, request):
    # 1. VALIDACIÓN DE API KEY POR PARÁMETRO (En el JSON)
    api_key_param = request.data.get('api_key')

    if not api_key_param:
        return Response({"error": "Falta el parámetro requerido 'api_key'", status=400})

    # Validar si la API Key es correcta (ejemplo comparando con settings o DB)
    # Si usas una tabla de clientes, aquí harías la consulta. Ejemplo básico:
    if api_key_param != settings.EXPECTED_API_KEY:
        return Response({"error": "API Key inválida por parámetro", status=403})

    # 2. VALIDACIÓN DEL TOKEN DE SEGURIDAD (JWT)
    auth_header = request.headers.get('Authorization')
    if not auth_header:
        return Response({"error": "No se proporcionó el encabezado Authorization", status=400})

    # Limpieza del prefijo "Bearer "
    parts = auth_header.split()
    if len(parts) == 2 and parts[0].lower() == 'bearer':
        token_recibido = parts[1]
    else:
        token_recibido = parts[0]

    try:
        payload = jwt.decode(token_recibido, settings.SECRET_KEY_AUTH, algorithms=['HS256'])
        scopes = payload.get('scopes', [])
        if 'otp_write' not in scopes:
            return Response({"error": "El token no tiene permisos para generar OTP", status=403})
    except jwt.ExpiredSignatureError:
        return Response({"error": "El token de seguridad ha expirado", status=401})
    except jwt.InvalidTokenError:
        return Response({"error": "Token de seguridad inválido", status=401})

    # --- LÓGICA DE GENERACIÓN ORIGINAL ---
    leng_cod_otp = settings.OTP_CONFIG['CODE_LENGTH']

    serializer = OTPGenerateSerializer(data=request.data)
    if serializer.is_valid():
        email = serializer.validated_data['email'].lower().strip()
        id_orig = serializer.validated_data['app_source']
        ip_address = request.META.get('REMOTE_ADDR')

        # Verificar si el correo está bloqueado

```

```

otp_record = OTPTransaction.objects.filter(
    user_email=email,
    app_source=id_orig,
    status='blocked'
).order_by('-blocked_until').first()

if otp_record and timezone.now() < otp_record.blocked_until:
    return Response({"error": "Correo suspendido temporalmente"}, status=

# Generar código alfanumérico
caracteres = string.ascii_letters + string.digits
codeotp = ''.join(random.choices(caracteres, k=leng_cod_otp))

nuevo_otp = OTPTransaction.objects.create(
    client=id_orig, # Guardamos el origen como identificador del cliente
    user_email=email,
    otp_code=make_password(codeotp),
    app_source=id_orig,
    ip_address=ip_address,
    status='pending'
)

# Envío vía microservicio de correo
exito_envio = enviar_correo_otp(email, "Estimado", codeotp, id_orig)

if exito_envio:
    return Response({
        "message": f"OTP enviado a {nuevo_otp.user_email}",
        "otp_code": codeotp,
        "transaction_id": nuevo_otp.id,
        "status": "Ok"
    }, status=status.HTTP_201_CREATED)

return Response({"error": "Error en el servicio de correo"}, status=500)

return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

```

2. Actualización de los Manuales

Dado este cambio estructural, debemos actualizar las secciones correspondientes en tus manuales para que reflejen que la API Key ya no viaja oculta en los Headers, sino dentro de la petición de datos.

En el Manual de Integración (Uso):

Modifica la sección de **Especificación de la API** indicando el nuevo formato del JSON:

Plaintext



```
## Especificación de la API: Generar OTP

* **Endpoint:** `POST /api/v1/otp/generate/`
* **Headers Obligatorios:**
  * `Authorization`: `Bearer <JWT_TOKEN>`

#### Cuerpo de la Petición (JSON)
{
  "api_key": "tu_clave_de_cliente_aqui",  <-- NUEVO PARÁMETRO
  "email": "usuario@ejemplo.com",
  "app_source": "ServicioFirma"
}
```

En el Manual Técnico:

Actualiza la sección de **Seguridad y Validación**:

Plaintext



```
## Seguridad y Validación de Acceso
El servicio ha migrado de una validación por Header (`HasClientApiKey`) a una validación por JWT.
```

1. El sistema intercepta el parámetro `api\_key` enviado en el JSON de la petición.
2. Compara el string con el valor almacenado en las variables de entorno (`settings.EXPECTED\_API\_KEY`).
3. Si la verificación es exitosa, procede a evaluar el JWT del encabezado `Authorization`.

(Nota: Recuerda definir `EXPECTED_API_KEY` en tu archivo `settings.py` apuntando a tu variable de entorno en AWS para que la validación de la línea 36 funcione correctamente).

uso una tabla de cliente para validar el api-key

Entendido. Si utilizas una tabla de clientes en tu base de datos para validar el `api_key`, el diseño se vuelve mucho más robusto y escalable, ya que puedes activar, desactivar o crear clientes sin necesidad de tocar las variables de entorno o reiniciar el servidor en AWS.

Asumiendo que tienes un modelo llamado `ServiceClient` (o similar) que almacena tus clientes y sus respectivas llaves, el código debe realizar una consulta `Filter` o `Get` para comprobar si existe y si está activo.

Aquí tienes la actualización exacta de tu vista y cómo registrarlo en los manuales:

1. Actualización de la Vista `GenerateOTP` (`views.py`)

He modificado el inicio del método `post` para que busque la `api_key` en tu base de datos. Ajusta el nombre del modelo (`ServiceClient`) según cómo lo tengas en tu archivo `models.py`.

Python



```
import jwt
import string
import random
from django.conf import settings
from django.utils import timezone
from django.utils.decorators import method_decorator
from django.views.decorators.csrf import csrf_exempt
from django.contrib.auth.hashers import make_password
from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework import status

# Importamos tus modelos (Asegúrate de cambiar 'ServiceClient' por el nombre real de
from api_otp.models import OTPTransaction, ServiceClient
from api_otp.serializers import OTPGenerateSerializer
from api_otp.utils import enviar_correo_otp

@method_decorator(csrf_exempt, name='dispatch')
class GenerateOTP(APIView):
    authentication_classes = []
    permission_classes = []

    def post(self, request):
        # 1. VALIDACIÓN DE API KEY DESDE LA TABLA DE CLIENTES
        api_key_param = request.data.get('api_key')

        if not api_key_param:
            return Response({"error": "Falta el parámetro requerido 'api_key'"}, status=status.HTTP_400_BAD_REQUEST)

        try:
            # Buscamos el cliente en la base de datos que coincida con la API Key y e
            # Ajusta los nombres de los campos ('api_key' e 'is_active') según tu mod
            cliente_solicitante = ServiceClient.objects.get(api_key=api_key_param, is_active=True)
        except ServiceClient.DoesNotExist:
```

```

        return Response({"error": "API Key inválida o cliente inactivo"}, status=

# 2. VALIDACIÓN DEL TOKEN DE SEGURIDAD (JWT)
auth_header = request.headers.get('Authorization')
if not auth_header:
    return Response({"error": "No se proporcionó el encabezado Authorization"

# Limpieza del prefijo "Bearer "
parts = auth_header.split()
if len(parts) == 2 and parts[0].lower() == 'bearer':
    token_recibido = parts[1]
else:
    token_recibido = parts[0]

try:
    payload = jwt.decode(token_recibido, settings.SECRET_KEY_AUTH, algorithms
    scopes = payload.get('scopes', [])
    if 'otp_write' not in scopes:
        return Response({"error": "El token no tiene permisos para generar OT

except jwt.ExpiredSignatureError:
    return Response({"error": "El token de seguridad ha expirado"}, status=40
except jwt.InvalidTokenError:
    return Response({"error": "Token de seguridad inválido"}, status=401)

# --- LÓGICA DE GENERACIÓN ---
leng_cod_otp = settings.OTP_CONFIG['CODE_LENGTH']

serializer = OTPGenerateSerializer(data=request.data)
if serializer.is_valid():
    email = serializer.validated_data['email'].lower().strip()
    id_orig = serializer.validated_data['app_source']
    ip_address = request.META.get('REMOTE_ADDR')

    # Verificar si el correo está bloqueado
    otp_record = OTPTransaction.objects.filter(
        user_email=email,
        app_source=id_orig,
        status='blocked'
    ).order_by('-blocked_until').first()

    if otp_record and timezone.now() < otp_record.blocked_until:
        return Response({"error": "Correo suspendido temporalmente"}, status=

    # Generar código alfanumérico
    caracteres = string.ascii_letters + string.digits
    codeotp = ''.join(random.choices(caracteres, k=leng_cod_otp))

    # Guardamos la transacción vinculándola al cliente encontrado en la DB

```

```

nuevo_otp = OTPTransaction.objects.create(
    client=cliente_solicitante, # <--- Volvemos a asociar el objeto Client
    user_email=email,
    otp_code=make_password(codeotp),
    app_source=id_orig,
    ip_address=ip_address,
    status='pending'
)

# Envío vía microservicio de correo
exito_envio = enviar_correo_otp(email, "Estimado", codeotp, id_orig)

if exito_envio:
    return Response({
        "message": f"OTP enviado a {nuevo_otp.user_email}",
        "otp_code": codeotp,
        "transaction_id": nuevo_otp.id,
        "status": "Ok"
    }, status=status.HTTP_201_CREATED)

return Response({"error": "Error en el servicio de correo"}, status=500)

return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

```

2. Actualización del Manual Técnico (Sección Base de Datos)

Añade o actualiza la relación de tablas en tu documentación técnica para reflejar este cambio arquitectónico:

Plaintext



2. Arquitectura de Datos y Validación Dinámica

El proceso de autenticación ahora depende de la persistencia de datos relacionales en

Relación de Modelos:

- * **`ServiceClient`** : Almacena las credenciales autorizadas (`api\_key`, `client\_name`)
- * **`OTPTransaction`** : Registra los tokens de firma generados, manteniendo una llave

Flujo Técnico de Verificación:

1. El sistema lee el campo `api\_key` provisto en el JSON de entrada.
2. Realiza una consulta SQL indexada: `SELECT \* FROM api\_otp\_serviceclient WHERE api\_`
3. Si la fila no existe, interrumpe el flujo inmediatamente devolviendo un código de

Con este ajuste, el código queda limpio, completamente integrado a tu base de datos y listo para operar en tu entorno de AWS con soporte para múltiples aplicaciones.

me esta dando el siguiente error => AttributeError:
'Request' object has no attribute 'authorized_client'

Ese error ocurre porque en la parte final de tu vista todavía quedó una línea de código que intenta usar `request.authorized_client`.

Antes, cuando usabas el decorador y la clase `HasClientApiKey`, esa librería se encargaba de buscar la API Key en los headers y "colgaba" automáticamente el objeto del cliente dentro de `request.authorized_client`.

Como ahora quitamos esa librería y hacemos la validación manualmente buscando en tu tabla de la base de datos, esa variable ya no existe dentro de `request`. El cliente real ahora lo tienes guardado en la variable `cliente_solicitante`.

La Solución

Busca en la parte de abajo de tu archivo `views.py` (donde se crea el registro en la base de datos) y cambia `request.authorized_client` por `cliente_solicitante`.

Asegúrate de que tu bloque de creación de la transacción se vea exactamente así:

Python



```
# --- BUSCA ESTA SECCIÓN EN TU VISTA ---
```

```
nuevo_otp = OTPTransaction.objects.create(  
    client=cliente_solicitante, # <--- AQUÍ: Usa la variable que definimos arriba, NO  
    user_email=email,  
    otp_code=make_password(codeotp),  
    app_source=id_orig,  
    ip_address=ip_address,  
    status='pending'  
)
```

¿Por qué falló?

Si revisas el flujo completo de la función que armamos en el paso anterior, al principio del `post` hicimos esto:

Python



```
try:
    # Aquí buscamos al cliente en tu tabla con el parámetro api_key
    cliente_solicitante = ServiceClient.objects.get(api_key=api_key_param, is_active=1)
except ServiceClient.DoesNotExist:
    return Response({"error": "API Key inválida o cliente inactivo"}, status=403)
```

Al dejar `request.authorized_client` más abajo, Django intentaba buscar algo que el nuevo sistema manual no había registrado en el objeto `request`, provocando el `AttributeError`. Con este cambio apuntando a `cliente_solicitante`, tu código volverá a enlazar correctamente la transacción con el cliente de la base de datos.

estoy tratando de levantar mi proyecto en qa y me da el siguiente error ==> 502 bad gateway

El error **502 Bad Gateway** en AWS significa que tu servidor web principal (**Nginx**) está activo y recibiendo las peticiones desde internet, pero cuando intenta pasarle la petición a tu servidor de aplicaciones (**Gunicorn** o el proceso de Django), este último no responde, está apagado o el canal de comunicación entre ambos está roto.

Como acabamos de hacer cambios en el código para validar la API Key con la base de datos, lo más probable es que **Gunicorn** se haya caído por un error de sintaxis o que no esté corriendo.

Sigue estos pasos en tu terminal de AWS para encontrar el problema exacto y solucionarlo:

1. Ver el estado de Gunicorn

Primero, averigua si tu servidor de aplicaciones está vivo. Ejecuta:

Bash



```
sudo systemctl status gunicorn
```

(Si tu servicio tiene otro nombre, como `gunicorn.service` o el nombre del proyecto, usa ese).

- Si dice `Active: failed` : Gunicorn intentó arrancar pero el nuevo código tiene un error (un import mal puesto, la variable `cliente_solicitante` con algún detalle, etc.).
- Si dice `Active: inactive` : Simplemente está apagado. Ejecuta `sudo systemctl start gunicorn`.

2. Cazar el error real en los Logs (Crucial)

Para ver exactamente por qué se cayó Gunicorn, revisa sus registros de errores:

Bash



```
sudo journalctl -u gunicorn --no-pager -n 50
```

O si configuraste un archivo de logs dedicado en `/var/log/`:

Bash



```
tail -n 50 /var/log/gunicorn/error.log
```

¿Qué buscar aquí? Busca las últimas líneas. Si ves un `SyntaxError`, `NameError`, o un `ImportError` (como el de `core` que vimos antes), ahí estará la línea exacta del código que está rompiendo el arranque en QA.

3. Verificar el Socket o Puerto (Nginx ⇔ Gunicorn)

Si Gunicorn dice que está `Active: running`, entonces el problema es que Nginx lo está buscando en el lugar equivocado.

1. Revisa la configuración de Nginx:

Bash



```
sudo nano /etc/nginx/sites-enabled/tu_configuracion
```

2. Busca la línea `proxy_pass`.

1. Si usa un archivo socket: `proxy_pass http://unix:/home/ubuntu/repos/otp/todo.sock;` → Verifica que el archivo `.sock` realmente exista en esa ruta.

2. Si usa un puerto: `proxy_pass http://127.0.0.1:8000;` → Asegúrate de que Gunicorn esté escuchando en el puerto `8000`.

4. Prueba de aislamiento (Correr Django manual)

Para descartar que sea un problema de Nginx, apaga gunicorn e intenta levantar Django "a mano" en el entorno de producción para ver si la consola te escupe el error directamente:

Bash



```
# Asegúrate de estar en tu entorno virtual
source /home/ubuntu/repos/otp/venv/bin/activate

# Corre el servidor en un puerto libre
python manage.py runserver 0.0.0.0:8080
```

Si al hacer esto el comando se queda escuchando de forma normal, tu código está 100% sano y el problema es netamente de la configuración de Gunicorn. Si falla aquí, la consola te dirá qué rompió el código en el servidor.

Checklist de reinicio rápido en AWS:

Cada vez que corrijas algo en el archivo `views.py` o `models.py` en AWS, debes aplicar este combo de comandos para limpiar la caché y aplicar los cambios:

Bash



```
cd /home/ubuntu/repos/otp/
find . -name "__pycache__" -delete
sudo systemctl restart gunicorn
sudo systemctl restart nginx
```

Revisa el comando `journalctl` del punto 2. ¿Qué error o *Traceback* te aparece al final del log?

este es mi archivo de default que esta en la carpeta
/etc/nginx/sites-enabled/ ==># 1. BLOQUE HTTPS (Puerto
443) - Aquí corre tu aplicación Django segura
server {



¡Acabas de encontrar al culpable del **502 Bad Gateway**!

Si miras con atención la línea del `proxy_pass` en tu archivo de configuración de Nginx, está apuntando al socket del servicio de correos, no al de tu nuevo servicio de OTP:

Nginx



```
# LO QUE TIENES ACTUALMENTE:  
proxy_pass http://unix:/var/run/gunicorn/emailservice-qa.sock;
```

Nginx está intentando enviar las peticiones que llegan a `otp-qa.transtecniasa.cl` hacia el socket de `emailservice-qa`. Al no encontrar el servicio de OTP ahí (o estar mapeado a otro proceso), se rompe la comunicación y salta el 502.

La Solución

Debes cambiar esa ruta por la de tu socket real de OTP. Normalmente, si sigues el mismo estándar de nombres, tu archivo de Gunicorn para OTP debería estar generando un socket llamado `otp-qa.sock` o similar.

Sigue estos pasos para corregirlo:

1. Modificar el archivo de Nginx

Abre el archivo para editarlo:

Bash



```
sudo nano /etc/nginx/sites-enabled/default
```

Busca la sección `location /` y cambia la línea del `proxy_pass`. Debería quedar algo así (confirma el nombre exacto de tu socket de OTP en la carpeta `/var/run/gunicorn/`):

Nginx



```
# Conexión exacta con el proceso activo de Gunicorn (Supervisor)  
location / {  
    include proxy_params;  
  
    # CAMBIA ESTA LÍNEA por el socket correcto de tu servicio OTP:  
    proxy_pass http://unix:/var/run/gunicorn/otp-qa.sock;
```

```
proxy_redirect off;  
proxy_set_header X-Real-IP $remote_addr;  
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
proxy_set_header X-Forwarded-Proto $scheme;  
}
```

Guarda los cambios (Ctrl+O, Enter) y sal (Ctrl+X).

2. Verificar la ruta del alias estático (Opcional pero importante)

Revisa también la ruta de tus archivos estáticos en ese mismo bloque:

Nginx



```
location /static/ {  
    alias /webapps/app/src/staticfiles/;  
}
```

Asegúrate de que esa ruta apunte a la carpeta real de `staticfiles` de tu proyecto de OTP en QA, de lo contrario, aunque la API funcione, no cargará correctamente la interfaz del panel de administración de Django si intentas entrar a él.

3. Validar y reiniciar Nginx

Antes de reiniciar, dile a Nginx que revise si cometiste algún error de sintaxis en el archivo:

Bash



```
sudo nginx -t
```

Si te devuelve un mensaje diciendo que todo está **OK** (`syntax is ok` , `test is successful`), procede a reiniciar el servicio para aplicar la nueva ruta del socket:

Bash



```
sudo systemctl restart nginx
```

Si por alguna razón no estás seguro de cómo se llama el archivo `.sock` de tu servicio OTP, puedes listarlos todos ejecutando:

Bash



```
ls -la /var/run/gunicorn/
```

Ahí verás el nombre exacto del archivo que debes poner en el `proxy_pass`. ¡Con esto deberías recuperar el acceso de inmediato!

como debe de quedar mi archivo default

Para que tu servicio de OTP responda correctamente en la URL `otp-qa.transtecniasa.cl` sin cruzarse con el servicio de correos, tu archivo `default` debe configurarse para escuchar exclusivamente el socket de la aplicación OTP.

Asumiendo que el socket de tu servicio se llama `otp-qa.sock` (el nombre estándar para esta arquitectura), tu archivo `/etc/nginx/sites-enabled/default` debe quedar exactamente de la siguiente manera:

Nginx



```
# 1. BLOQUE HTTPS (Puerto 443) - Aquí corre tu aplicación Django segura
server {
    listen 443 ssl default_server;
    listen [::]:443 ssl default_server;

    server_name otp-qa.transtecniasa.cl;

    proxy_headers_hash_max_size 1024;
    proxy_headers_hash_bucket_size 128;

    # Manejo de los archivos estáticos de Django (CSS, JS, Imágenes)
    # Ajustado a la ruta de tu repositorio en AWS (/home/ubuntu/repos/otp/)
    location /static/ {
        alias /home/ubuntu/repos/otp/staticfiles/;
        include /etc/nginx/mime.types;
        expires 30d;
        add_header Cache-Control "public, no-transform";
    }
}
```

```

# Conexión exacta con el proceso activo de Unicorn de OTP
location / {
    include proxy_params;

    # REDIRECCIÓN CORREGIDA: Apunta al socket del microservicio de OTP
    proxy_pass http://unix:/var/run/gunicorn/otp-qa.sock;

    proxy_redirect off;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}

# Certificados SSL de Certbot
ssl_certificate /etc/letsencrypt/live/otp-qa.transtecniasa.cl/fullchain.pem;
ssl_certificate_key /etc/letsencrypt/live/otp-qa.transtecniasa.cl/privkey.pem;
include /etc/letsencrypt/options-ssl-nginx.conf;
ssl_dhparam /etc/letsencrypt/ssl-dhparams.pem;
}

# 2. BLOQUE HTTP (Puerto 80) - Redirección automática a HTTPS
server {
    listen 80 default_server;
    listen [::]:80 default_server;

    server_name otp-qa.transtecniasa.cl;

    # Si entran por HTTP normal, los manda directo a HTTPS seguro
    return 301 https://$host$request_uri;
}

```

Cambios Clave Aplicados:

1. `proxy_pass` **Corregido:** Cambiamos `emailservice-qa.sock` por `otp-qa.sock`. Asegúrate con un `ls /var/run/gunicorn/` de que tu proceso de Unicorn esté generando el archivo con ese nombre exacto.
2. `location /static/` **Actualizado:** Reemplacé la ruta genérica `/webapps/app/...` por la ruta real de tu repositorio (`/home/ubuntu/repos/otp/staticfiles/`). Recuerda ejecutar `python manage.py collectstatic` en tu entorno virtual para que Django compile los archivos ahí y Nginx pueda servirlos de inmediato.
3. `proxy_set_header Host $host;` **Activo:** Descomenté esta línea. Es vital para que Django no rechace las peticiones por problemas de `ALLOWED_HOSTS` al procesar los dominios en AWS.

Pasos finales para aplicar el cambio:

Una vez que guardes el archivo con estos datos, ejecuta en la terminal de AWS:

Bash



```
# 1. Validar que la sintaxis de Nginx sea correcta
sudo nginx -t

# 2. Si el test es exitoso, reiniciar Nginx
sudo systemctl restart nginx
```

me da este error al tratar de levantar el servicio ==> app:
ERROR (spawn error)

El error `app: ERROR (spawn error)` proviene directamente de **Supervisor** (el gestor de procesos que usas en Linux para mantener vivo a Gunicorn).

Un "spawn error" significa que Supervisor intentó arrancar el proceso de Django/Gunicorn, pero este **murió inmediatamente en el primer segundo**. Esto ocurre casi siempre por problemas de rutas mal escritas, permisos de carpetas o porque el entorno virtual no se activó correctamente dentro de la configuración.

Vamos a solucionarlo directo en la raíz. Sigue estos pasos en tu servidor AWS:

1. Encontrar la causa exacta (El Log de Supervisor)

Supervisor guarda un registro de por qué falló el arranque. Ejecuta este comando para ver el error real de Python:

Bash



```
sudo tail -n 50 /var/log/supervisor/app-stderr.log
```

(Nota: Si tu archivo de configuración en Supervisor tiene otro nombre en lugar de `app`, el archivo se llamará `nombre-stderr.log`).

Lo que veas ahí te dirá la verdad: puede ser un problema con la base de datos, un error de sintaxis en el `views.py` que modificamos, o que no encuentra el ejecutable de `gunicorn`.

2. Verificar el archivo de configuración de Supervisor

El archivo suele estar en `/etc/supervisor/conf.d/app.conf` (o el nombre de tu proyecto). Vamos a revisar que las rutas coincidan con tu estructura real de `transtecnia_otp`:

Ejecuta `sudo nano /etc/supervisor/conf.d/app.conf` y asegúrate de que tenga esta consistencia:

Ini, TOML



```
[program:app]
command=/home/ubuntu/repos/otp/venv/bin/gunicorn --workers 3 --bind unix:/var/run/gun
directory=/home/ubuntu/repos/otp
user=ubuntu
autostart=true
autorestart=true
stderr_logfile=/var/log/supervisor/app-stderr.log
stdout_logfile=/var/log/supervisor/app-stdout.log
```

Puntos críticos a revisar ahí dentro:

1. `command`: Mira si la ruta al `gunicorn` dentro de tu `venv` es la correcta.
2. `directory`: Debe ser `/home/ubuntu/repos/otp` (donde está tu `manage.py`).
3. `transtecnia_otp.wsgi:application`: Asegúrate de que no diga `core.wsgi`. Como renombramos la carpeta a `transtecnia_otp`, si Supervisor busca `core`, lanzará el *spawn error*.

3. El problema de la carpeta `/var/run/gunicorn/` (Muy Común)

Si el directorio donde Nginx y Supervisor guardan el socket no existe o el usuario `ubuntu` no tiene permisos para escribir en él, Supervisor no podrá crear el archivo `otp-qa.sock` y fallará.

Ejecuta esto para asegurar los permisos del socket:

Bash



```
sudo mkdir -p /var/run/gunicorn/
sudo chown -R ubuntu:www-data /var/run/gunicorn/
```

```
sudo chmod -R 775 /var/run/gunicorn/
```

4. Recargar Supervisor

Cada vez que modificas algo en el código o en los archivos de configuración de Supervisor, debes indicarle que lea los cambios desde cero:

Bash



```
sudo supervisorctl reread
sudo supervisorctl update
sudo supervisorctl restart app
```

Si ejecutas el comando del **Paso 1** (`sudo tail -n 50 /var/log/supervisor/app-stderr.log`), ¿qué línea o error de Python te aparece al final? Eso nos dará el tiro de gracia para dejarlo operativo.

conseguí este error ==> No module named 'whitenoise'

Ese error nos da la respuesta exacta. `ModuleNotFoundError: No module named 'whitenoise'` significa que tu archivo `settings.py` (o `wsgi.py`) tiene configurado **Whitenoise** para manejar los archivos estáticos en producción, pero la librería no está instalada dentro del entorno virtual (`venv`) de tu servidor de AWS QA.

Al no encontrar la librería, Python interrumpe el arranque y por eso Supervisor te lanzaba el `spawn error`.

Para solucionarlo, sigue estos pasos rápidos directamente en tu terminal de AWS:

1. Instalar Whitenoise en el Entorno Virtual

Debes asegurarte de instalar la librería **dentro** del entorno virtual del proyecto. Ejecuta estos comandos en orden:

Bash



```
# 1. Ve a la carpeta de tu proyecto
cd /home/ubuntu/repos/otp/

# 2. Activa tu entorno virtual
source venv/bin/activate

# 3. Instala la librería usando pip
pip install whitenoise
```

2. Actualizar el archivo `requirements.txt`

Para evitar que esto te vuelva a pasar si necesitas desplegar en otro servidor en el futuro, guarda la librería en tus requerimientos:

Bash



```
pip freeze > requirements.txt
```

3. Aplicar cambios y revivir el servicio en Supervisor

Una vez instalada la librería, limpia cualquier caché vieja y dile a Supervisor que vuelva a levantar la aplicación:

Bash



```
# Limpiar archivos de caché viejos
find . -name "__pycache__" -delete

# Reiniciar el proceso en Supervisor
sudo supervisorctl restart app
```

4. Verificar que ya esté corriendo

Comprueba si Supervisor logró levantar el servicio con éxito:

Bash



```
sudo supervisorctl status app
```

Debería cambiar el estado de `ERROR (spawn error)` a `RUNNING`. En cuanto veas el estado en verde, tu API Gateway (Nginx) se conectará de inmediato al socket y el error 502 desaparecerá por completo.

en qa no tiene el entorno virtual

¡Ah, con razón! Si no hay entorno virtual (`venv`) creado en el servidor de QA, entonces Supervisor e Nginx están intentando apuntar a rutas y ejecutables que físicamente no existen en esa máquina.

Para solucionarlo, vamos a construir el entorno virtual desde cero e instalar todas las dependencias que tu proyecto de OTP necesita para correr.

Sigue estos pasos exactos en tu terminal de AWS QA:

1. Crear el Entorno Virtual

Párate en la raíz de tu proyecto y crea la carpeta `venv` usando Python 3.13 (que es la versión que delató tu Traceback inicial):

Bash



```
cd /home/ubuntu/repos/otp/  
python3.13 -m venv venv
```

2. Activar e Instalar Dependencias

Una vez creado, lo activamos e instalamos tanto `whitenoise` como el resto de librerías esenciales que usa tu API (`Django`, `djangorestframework`, `PyJWT`, `requests`, `gunicorn`, y el conector de la base de datos `psycopg2-binary`):

Bash



```
# 1. Activar el entorno virtual recién creado  
source venv/bin/activate  
  
# 2. Asegurar que pip esté actualizado
```

```
pip install --upgrade pip
```

3. Instalar todo el stack del proyecto en bloque

```
pip install django djangorestframework pyjwt requests gunicorn whitenoise psycpg2-bi
```

Tip: Si tienes un archivo `requirements.txt` en tu repositorio, en lugar del comando 3 puedes ejecutar directamente: `pip install -r requirements.txt` y luego sumarle `pip install whitenoise gunicorn` por si no venían dentro.

3. Verificar que Gunicorn responda

Para asegurarnos de que el entorno virtual quedó bien configurado, haz una prueba rápida pidiéndole la versión a Gunicorn desde la ruta que va a usar Supervisor:

Bash



```
/home/ubuntu/repos/otp/venv/bin/gunicorn --version
```

Si te devuelve el número de versión en la pantalla, el entorno virtual está al 100%.

4. Revivir el proceso en Supervisor

Ahora que los ejecutables ya existen en el disco, dile a Supervisor que vuelva a intentar el despliegue:

Bash



```
sudo supervisorctl reread
sudo supervisorctl update
sudo supervisorctl restart app
```

Revisa el estatus final con `sudo supervisorctl status app`. ¡Al encontrar por fin su entorno virtual y la librería `whitenoise`, el proceso pasará a estado **RUNNING** de inmediato!

me aparece lo siguiente ==> app

FATAL

Exited too quickly (process log may have details)

El estado `FATAL Exited too quickly` significa que Supervisor se rindió tras intentar levantar Gunicorn varias veces seguidas sin éxito. Como acabamos de instalar el entorno virtual, hay dos razones muy comunes para que esto pase:

1. El archivo de configuración de Supervisor (`app.conf`) apunta a un comando o ruta de Python que todavía tiene algún error.
2. Tu código de Django tiene un problema al intentar conectarse a la base de datos de QA o le falta alguna variable de entorno en el servidor.

Vamos a solucionarlo siguiendo estos pasos en orden:

1. El veredicto real (Log de errores)

Necesitamos ver qué le pasó a Python en este último intento. Ejecuta:

Bash



```
sudo tail -n 50 /var/log/supervisor/app-stderr.log
```

¿Qué buscar ahí?

- Si dice `ModuleNotFoundError: No module named '...'`: Falta otra librería por instalar en el `venv` (como `django_filters`, `corsheaders`, etc.).
- Si dice `connection to server at "localhost" failed: Connection refused`: Django no se puede conectar a PostgreSQL en QA.
- Si dice `KeyError`: Falta alguna variable de entorno en tu servidor.

2. Forzar el arranque manual (Prueba de Diagnóstico)

Para saltarnos a Supervisor por un momento y ver el error en tiempo real, ejecuta Gunicorn manualmente usando el entorno virtual:

Bash



```
# 1. Ve a la carpeta del proyecto
cd /home/ubuntu/repos/otp/
```

```
# 2. Ejecuta la misma línea que usa Supervisor
./venv/bin/gunicorn --workers 3 --bind 0.0.0.0:8080 transtecnia_otp.wsgi:application
```

- **Si se cae inmediatamente:** La consola te escupirá el error exacto de Django en la pantalla. ¡Compártemelo para darte la solución de inmediato!
- **Si se queda corriendo sin caerse:** Tu código está perfecto, el problema es meramente de Supervisor (permisos del socket o rutas). Cancela con `Ctrl + C`.

3. El problema clásico: Permisos del archivo `.sock`

Si la prueba manual del paso 2 funcionó, el problema es que Gunicorn (corriendo como usuario `ubuntu` a través de Supervisor) no puede crear el archivo del socket en `/var/run/gunicorn/`.

Asegura las carpetas ejecutando esto:

Bash



```
sudo mkdir -p /var/run/gunicorn
sudo chown -R ubuntu:www-data /var/run/gunicorn
sudo chmod -R 775 /var/run/gunicorn
```

4. Limpiar y revivir Supervisor

Una vez que Supervisor entra en estado `FATAL`, no intentará arrancar de nuevo por sí solo. Tienes que "despertarlo" manualmente con este combo:

Bash



```
sudo supervisorctl reread
sudo supervisorctl update
sudo supervisorctl start app
```

Prueba el **Paso 2** (correr el comando de Gunicorn manual) si quieres ir a la segura. ¿Qué mensaje te arroja la terminal al ejecutarlo?

estos son los archivos que estan en la carpeta log ==> qa-access.log qa-error.log supervisor.log supervisor_error.log

Esos archivos indican que estás revisando la carpeta general de logs de tu despliegue o la raíz del proyecto. El archivo clave aquí es `supervisor_error.log`, ya que ahí es donde Supervisor escribe exactamente por qué Python o Gunicorn no pudieron arrancar.

Vamos a leer el interior de ese archivo para ver el *Traceback* real. Ejecuta este comando en tu terminal de AWS:

Bash



```
tail -n 50 supervisor_error.log
```

¿Qué puede estar pasando? (Teniendo en cuenta que no hay `venv` global)

Como acabamos de crear el entorno virtual a mano, lo más probable es que el archivo de configuración de Supervisor todavía esté apuntando al Python global del sistema o a una ruta vieja.

Mientras me compartes lo que dice el log, puedes verificar el archivo de configuración de tu aplicación en Supervisor para asegurarte de que use el nuevo entorno:

1. Abre el archivo (suele estar en `/etc/supervisor/conf.d/app.conf` o similar):

Bash



```
sudo nano /etc/supervisor/conf.d/app.conf
```

2. Asegúrate de que la línea `command` use explícitamente el Gunicorn de la carpeta que acabamos de crear:

Ini, TOML



```
command=/home/ubuntu/repos/otp/venv/bin/gunicorn --workers 3 --bind unix:/var/ru
```

Si la ruta del comando no empezaba con `/home/ubuntu/repos/otp/venv/bin/`, Supervisor intentaba usar un Gunicorn global que no tiene acceso a las librerías (`whitenoise`, `jwt`, etc.) que instalamos hace un momento.

Ejecuta el `tail -n 50 supervisor_error.log` y dime qué error te arroja en las últimas líneas. ¡Ahí está la solución definitiva!

```
esto es lo que me aparece en el log ==> No module named  
'whitenoise'  
No module named 'whitenoise'  
No module named 'whitenoise'
```

¡Excelente! El log nos dio la respuesta exacta y el panorama completo de lo que está pasando.

El problema es que el Supervisor de QA **no está usando el entorno virtual que creamos en** `/home/ubuntu/repos/otp/venv`. Si te fijas en las rutas del Traceback, Supervisor está ejecutando un Gunicorn que está metido en otra ruta global del sistema: `/webapps/app/bin/gunicorn` usando Python 3.14.

Como creamos el entorno virtual en la carpeta de tu repositorio (`/home/ubuntu/`), las librerías se instalaron allá, pero Supervisor sigue llamando al Gunicorn de `/webapps/app/`, el cual no tiene `whitenoise` instalado.

Para solucionarlo definitivamente, debemos cambiar la ruta en el archivo de configuración de Supervisor para que use tu nuevo entorno virtual.

Paso 1: Modificar la configuración de Supervisor

1. Abre el archivo de configuración del proceso (revisa si es `app.conf` o el nombre asignado al proyecto de OTP):

Bash



```
sudo nano /etc/supervisor/conf.d/app.conf
```

2. Busca la línea que empieza con `command=` y cámbiala para que apunte al Gunicorn de tu entorno virtual en `/home/ubuntu/repos/otp/venv/bin/gunicorn`. Debería quedar algo muy parecido a esto:

Ini, TOML



```
[program:app]
command=/home/ubuntu/repos/otp/venv/bin/gunicorn --workers 3 --bind unix:/var/run
directory=/home/ubuntu/repos/otp
user=ubuntu
autostart=true
autorestart=true
stderr_logfile=/var/log/supervisor/app-stderr.log
stdout_logfile=/var/log/supervisor/app-stdout.log
```

(Nota: Asegúrate de que el parámetro `--bind unix:` coincida con el nombre de socket que pusimos en Nginx, que es `otp-qa.sock`).

3. Guarda los cambios presionando `Ctrl + O` , luego `Enter` y sal con `Ctrl + X` .

Paso 2: Forzar a Supervisor a leer la nueva ruta

Una vez editado el archivo, dile a Supervisor que actualice su mapa de procesos y reinicie la aplicación:

Bash



```
# 1. Indicarle a Supervisor que lea los archivos de configuración modificados
sudo supervisorctl reread

# 2. Aplicar los cambios en el proceso activo
sudo supervisorctl update

# 3. Iniciar o reiniciar el servicio
sudo supervisorctl restart app
```

Paso 3: Verificar el estado final

Comprueba si el proceso logró mantenerse en ejecución:

Bash



```
sudo supervisorctl status app
```

Al apuntar a `/home/ubuntu/repos/otp/venv/bin/gunicorn` , el proceso encontrará inmediatamente la librería `whitenoise` que instalamos previamente y pasará a estado

`RUNNING` . Con eso, Nginx podrá conectar con el socket y tu error 502 desaparecerá por completo en QA.

```
asi esta el archivo app.conf ==> [program:app]
command = /webapps/app/src/deploy/gunicorn_qa.sh
user = root
stdout_logfile = /webapps/app/logs/supervisor.log
stderr_logfile = /webapps/app/logs/supervisor_error.log
```

¡Perfecto! Al ver tu archivo `app.conf` , ahora todo cobra sentido al 100%.

Supervisor no está ejecutando Gunicorn directamente, sino que está llamando a un script intermedio en Bash: `/webapps/app/src/deploy/gunicorn_qa.sh` . El error de `whitenoise` ocurre porque adentro de ese archivo `.sh` se está invocando al Python/Gunicorn global de `/webapps/app/` en lugar de tu nuevo entorno virtual.

Para solucionarlo de la forma más limpia y sin alterar la estructura original de despliegue de Transtecnia, solo debemos actualizar la ruta dentro de ese script.

Sigue estos pasos en tu terminal de AWS:

1. Modificar el script de despliegue (`gunicorn_qa.sh`)

Abre el archivo `.sh` para editarlo:

Bash



```
sudo nano /webapps/app/src/deploy/gunicorn_qa.sh
```

Adentro vas a ver una estructura que define variables de entorno y ejecuta Gunicorn. Busca la línea donde corre Gunicorn (suele estar al final) y asegúrate de cambiar la ruta para que use el ejecutable de tu entorno virtual en

```
/home/ubuntu/repos/otp/venv/bin/gunicorn .
```

Debería quedar configurado apuntando a tu entorno, algo así:

Bash



```
#!/bin/bash
```

```
# ... (otras configuraciones o variables de entorno que tenga el script) ...
```

```
# Asegúrate de que la ejecución final llame al Gunicorn del venv:
```

```
exec /home/ubuntu/repos/otp/venv/bin/gunicorn \  
  --workers 3 \  
  --bind unix:/var/run/gunicorn/otp-qa.sock \  
  transtecnia_otp.wsgi:application
```

(Nota: Confirma que el `--bind` mantenga la ruta al socket `otp-qa.sock` para que haga match perfecto con lo que configuramos en tu Nginx).

Guarda los cambios (`Ctrl + O` , `Enter`) y sal (`Ctrl + X`).

2. Alternativa: Modificar directo `app.conf` (Si prefieres saltarte el script)

Si el script `.sh` tiene configuraciones muy viejas o complejas y prefieres que Supervisor controle a Gunicorn de forma directa y transparente (que es la mejor práctica), puedes reemplazar el contenido de tu `/etc/supervisor/conf.d/app.conf` para que quede así:

Ini, TOML



```
[program:app]  
command=/home/ubuntu/repos/otp/venv/bin/gunicorn --workers 3 --bind unix:/var/run/gun  
directory=/home/ubuntu/repos/otp  
user=ubuntu  
stdout_logfile=/webapps/app/logs/supervisor.log  
stderr_logfile=/webapps/app/logs/supervisor_error.log  
autostart=true  
autorestart=true
```

Cualquiera de las dos opciones (arreglar el `.sh` o cambiar el `app.conf`) solucionará el problema, elige la que mejor se adapte al estándar de tu repositorio.

3. Aplicar cambios en Supervisor

Una vez que hayas guardado el cambio (ya sea en el `.sh` o en el `.conf`), dile a Supervisor que recargue la configuración y levante el servicio:

Bash



```
sudo supervisorctl reread  
sudo supervisorctl update
```

```
sudo supervisorctl restart app
```

4. Monitorear el éxito del despliegue

Revisa si el proceso pasó a estar activo:

Bash



```
sudo supervisorctl status app
```

Si todo está correcto, verás el estado en `RUNNING`, el módulo `whitenoise` cargará con éxito desde el entorno virtual y el Bad Gateway 502 de Nginx se resolverá inmediatamente.

el archivo `gunicorn_qa.sh` tiene lo siguiente ==> GNU nano 7.2

```
/webapps/app/src/deploy/gunicorn_qa.sh
VENV=/webapps/app
```

¡Excelente Francisco! Al ver el contenido del archivo `gunicorn_qa.sh` queda clarísimo dónde está el conflicto.

En la primera línea del script dice:

```
VENV=/webapps/app
```

Esto significa que el script está intentando activar un entorno virtual en esa ruta (`/webapps/app/bin/activate`). Como tu entorno virtual real (donde instalamos `whitenoise` y las demás dependencias) lo creamos en `/home/ubuntu/repos/otp/venv`, al ejecutarse el script activa el entorno equivocado (o uno vacío) y por eso hereda el Python global que no tiene las librerías.

Además, falta definir la variable `APPDIR` arriba para que el comando `cd $APPDIR` funcione correctamente.

La Solución

Vamos a corregir las rutas al inicio del archivo para que apunten a tu repositorio real. Abre el archivo con `sudo nano /webapps/app/src/deploy/gunicorn_qa.sh` y déjalo exactamente así:

Bash



```
#!/bin/bash

# --- CONFIGURACIÓN DE RUTAS CORREGIDAS ---
APPDIR=/home/ubuntu/repos/otp
VENV=/home/ubuntu/repos/otp/venv
SOCKFILE=/var/run/gunicorn/otp-qa.sock
USER=root
GROUP=root
NUM_WORKERS=2
TIMEOUT=120
LOG_DIR=/webapps/app/logs

# Activar el entorno virtual real
cd $APPDIR
source $VENV/bin/activate
export PYTHONPATH=$APPDIR:$PYTHONPATH
export DJANGO_SETTINGS_MODULE=transtecnia_otp.settings.qa

# Crear el directorio del socket si no existe
RUNDIR=$(dirname $SOCKFILE)
test -d $RUNDIR || mkdir -p $RUNDIR

# Iniciar el servicio (usa el gunicorn del venv explícitamente)
exec $VENV/bin/gunicorn transtecnia_otp.wsgi:application \
  --workers $NUM_WORKERS \
  --user=$USER --group=$GROUP \
  --timeout=$TIMEOUT \
  --graceful-timeout=30 \
  --keep-alive=5 \
  --log-level=debug \
  --error-logfile=$LOG_DIR/qa-error.log \
  --access-logfile=$LOG_DIR/qa-access.log \
  --max-requests=1000 \
  --max-requests-jitter=100 \
  --bind=unix:$SOCKFILE
```

Cambios que aplicamos:

1. `APPDIR=/home/ubuntu/repos/otp` : Definimos la variable de la carpeta de tu proyecto, que no estaba arriba y hacía que el `cd $APPDIR` fallara o se quedara en el limbo.
2. `VENV=/home/ubuntu/repos/otp/venv` : Cambiamos la ruta para que apunte al entorno virtual correcto que creamos juntos.

3. `exec $VENV/bin/gunicorn` : Forzamos al script a ejecutar el binario exacto de Gunicorn que está dentro de tu entorno virtual, asegurando que cargue `whitenoise` .

Pasos finales para levantar QA:

Guarda los cambios (`Ctrl + O` , `Enter` , `Ctrl + X`) y ejecuta los comandos para reiniciar Supervisor:

Bash



```
# 1. Avisar a Supervisor que hubo cambios
sudo supervisorctl reread
sudo supervisorctl update

# 2. Reiniciar el servicio
sudo supervisorctl restart app

# 3. Validar el estatus
sudo supervisorctl status app
```

¡Con esto el script apuntará al entorno correcto, se importará `whitenoise` sin errores y tu servicio pasará a **RUNNING** de inmediato!

me dio ahora este error==> Invalid HTTP_HOST header: 'otp-qa.transtecniasa.cl,otp-qa.transtecniasa.cl'. The domain name provided is not valid according to RFC 1034/1035.

¡Buenas noticias! Este error (`Invalid HTTP_HOST header`) significa que **Gunicorn** y **Django** ya están vivos y corriendo perfectamente. El problema con el módulo `whitenoise` quedó completamente solucionado.

El error actual ocurre porque el dominio está llegando duplicado (`otp-qa.transtecniasa.cl,otp-qa.transtecniasa.cl`) al backend de Django. Esto es un comportamiento clásico de una mala configuración en la capa de red de **Nginx** cuando se usan ciertos balanceadores de carga o proxies en AWS.

Nginx está duplicando el header `Host` , y Django lo rechaza por seguridad para evitar ataques de envenenamiento de host.

La Solución: Corregir tu archivo de Nginx

Debemos ir a tu archivo `default` de Nginx y remover una directiva que está forzando esa duplicación.

1. Abre el archivo de configuración de Nginx:

Bash



```
sudo nano /etc/nginx/sites-enabled/default
```

2. Ve al bloque `location /` (donde está el `proxy_pass`).
3. Busca estas dos líneas (una de ellas venía de un archivo externo y la otra la agregamos de manera explícita):

Nginx



```
include proxy_params;      # <--- Esta línea ya agrega por dentro el "proxy_set_
proxy_set_header Host $host; # <--- Al ponerla aquí también, se envía DUPLICADO
```

4. Elimina o comenta la línea `proxy_set_header Host $host;`. El bloque debe quedar exactamente así:

Nginx



```
location / {
    include proxy_params;
    proxy_pass http://unix:/var/run/gunicorn/otp-qa.sock;

    proxy_redirect off;
    # proxy_set_header Host $host; <-- Asegúrate de dejarla comentada con # o b
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}
```

Aplicar los cambios

Guarda el archivo (`Ctrl + O` , `Enter` , `Ctrl + X`) y reinicia Nginx para limpiar las cabeceras de red:

Bash



```
# Verificar sintaxis
sudo nginx -t

# Reiniciar si todo está OK
sudo systemctl restart nginx
```

¿Por qué pasó esto?

El archivo `proxy_params` de Nginx es un estándar de Ubuntu que ya inyecta internamente el Host original. Al habilitar manualmente el comando `proxy_set_header Host`, Nginx concatenó ambos valores separados por una coma, generando la cadena inválida para Django.

Una vez reinicies Nginx, la petición llegará limpia con un solo nombre de dominio y tu API de OTP en QA responderá inmediatamente con total normalidad.

los estilos no lo esta cargando me da error 403 Forbidden

El error **403 Forbidden** en los archivos estáticos (CSS, JS, imágenes) significa que **Nginx ya sabe dónde están los archivos**, pero el sistema operativo de Linux (Ubuntu) le está **prohibiendo** la entrada a leerlos.

Esto pasa porque Nginx corre bajo el usuario `www-data` y tu repositorio de OTP está en el home del usuario `/home/ubuntu/`. Por defecto, Linux bloquea el acceso de usuarios externos a la carpeta `/home/ubuntu/` por seguridad.

Vamos a solucionarlo dándole los permisos correctos a la ruta para que Nginx pueda renderizar los estilos en el panel de administración de Django.

1. Dar permisos de ejecución a la ruta del repositorio

Para que Nginx pueda entrar a buscar los archivos dentro de la estructura de carpetas de tu usuario, debes darle permisos de lectura y ejecución (`+x`) a la carpeta del home y del repositorio. Ejecuta estos comandos en tu terminal de AWS:

Bash



```
# 1. Permitir que el sistema acceda a la carpeta del usuario ubuntu
sudo chmod 755 /home/ubuntu

# 2. Permitir acceso a la carpeta de tus repositorios
sudo chmod 755 /home/ubuntu/repos

# 3. Dar permisos específicos a la carpeta del proyecto OTP
sudo chmod -R 755 /home/ubuntu/repos/otp
```

2. Asegurar la recolección de estáticos en Django

Asegurémonos de que Django haya compilado correctamente todos los archivos CSS en la carpeta que Nginx está buscando. Entra a tu entorno virtual y fuerza la recolección:

Bash



```
cd /home/ubuntu/repos/otp/
source venv/bin/activate

# Ejecutar el recolector de estáticos de Django
python manage.py collectstatic --noinput
```

Este comando tomará los estilos del Administrador de Django y los volcará en la carpeta `/home/ubuntu/repos/otp/staticfiles/`.

3. Verificar el usuario de Nginx

En tu archivo `unicorn_qa.sh` vimos que configuraste que Unicorn corra como `USER=root`. Si los archivos estáticos se generaron como `root`, Nginx (`www-data`) tampoco los podrá leer.

Vamos a transferirle la propiedad de la carpeta de estáticos al usuario `ubuntu` y al grupo de `www-data` para que ambos convivan en armonía:

Bash



```
sudo chown -R ubuntu:www-data /home/ubuntu/repos/otp/staticfiles/
sudo chmod -R 755 /home/ubuntu/repos/otp/staticfiles/
```

4. Limpiar caché y comprobar

Por último, reinicia Nginx para asegurarte de que rompa cualquier bloqueo previo de permisos en caché:

Bash



```
sudo systemctl restart nginx
```

¡Refresca el navegador usando **Ctrl + F5** (para forzar la limpieza de caché en tu computadora) y verás cómo el panel de administración carga con todos sus estilos y diseño original de inmediato!

[actualizar manual de integracion d ela otp](#)

Aquí tienes el **Manual de Integración del Microservicio de OTP** completamente actualizado para el entorno de QA.

Este documento refleja la arquitectura final que dejamos operativa en AWS: la migración a validación dinámica de `api_key` basada en base de datos PostgreSQL, la autenticación híbrida mediante JWT, y la resolución de la infraestructura de red bajo Nginx y Supervisor.

Manual de Integración: Microservicio OTP (Entorno QA)

El Microservicio de OTP de Transtecnia provee un mecanismo desacoplado, seguro y multi-tenant para la generación y validación de contraseñas de un solo uso (One-Time Passwords).

1. Arquitectura de Autenticación Híbrida

Para que una solicitud sea procesada por el microservicio, debe superar de manera obligatoria dos capas de seguridad consecutivas:

1. **Validación de Identidad (Capa de Base de Datos - Multi-tenant):** El parámetro `api_key` enviado en el cuerpo de la solicitud JSON ya no es estático en variables de entorno. Se valida dinámicamente contra la tabla relacional `api_otp_serviceclient` en PostgreSQL. Si el cliente no existe o está inactivo

(`is_active=False`), el flujo se interrumpe inmediatamente devolviendo un estado HTTP 403 Forbidden.

2. **Autorización Criptográfica (Capa Web - JWT):** Se valida la firma del token Bearer enviado en las cabeceras. El token debe ser válido, no haber expirado y contener el scope específico requerido para la acción (ej. `otp_write`).

2. Diagrama del Flujo de Infraestructura (QA)

El tráfico de red en el servidor AWS de QA opera bajo el siguiente ciclo de comunicación:

Plaintext



```
[Cliente API]
|
▼ (Puerto 443 / HTTPS)
[Nginx] (otp-qa.transtecniasa.cl)
|
▼ ( proxy_pass a través de socket UNIX )
[/var/run/gunicorn/otp-qa.sock]
|
▼ (Ejecutado por Supervisor mediante script gunicorn_qa.sh)
[Gunicorn / Django App] (Usa Entorno Virtual dedicado en /home/ubuntu/repos/otp/venv)
```

3. Especificación de Endpoints de la API

3.1 Generación de OTP

- **Endpoint:** `POST https://otp-qa.transtecniasa.cl/api/v1/otp/generate/`
- **Headers Obligatorios:**

HTTP



```
Authorization: Bearer <JWT_SECURITY_TOKEN>
Content-Type: application/json
```

Cuerpo de la Solicitud (JSON):

JSON



```
{
  "api_key": "tt_qa_client_secure_hash_key_xyz",
  "email": "usuario@dominio.cl",
  "app_source": "PORTAL_CLIENTES"
}
```

Respuestas del Servidor:

- **HTTP 201 Created (Éxito):** El OTP se generó, se almacenó de forma cifrada en la base de datos y se disparó el envío del correo electrónico.

JSON



```
{
  "message": "OTP enviado a usuario@dominio.cl",
  "otp_code": "A8B2C3",
  "transaction_id": 1425,
  "status": "Ok"
}
```

- **HTTP 400 Bad Request:** Estructura JSON mal formada o falta el parámetro `api_key`.
- **HTTP 401 Unauthorized:** Token JWT expirado, alterado o inexistente.
- **HTTP 403 Forbidden:** La `api_key` provista no existe en la tabla de clientes activos, o el token JWT no cuenta con el scope `otp_write`.
- **HTTP 404 Not Found:** El correo electrónico ingresado se encuentra en estado de suspensión temporal debido a múltiples intentos fallidos acumulados.

4. Manual de Operaciones y Mantenimiento (Servidor AWS QA)

Para los administradores del sistema, a continuación se detallan los componentes críticos del backend que rigen el despliegue:

4.1 Script de Control de Arranque (`unicorn_qa.sh`)

Ubicado en `/webapps/app/src/deploy/unicorn_qa.sh`. Este archivo orquesta el aislamiento del entorno virtual y la ejecución del servidor de aplicaciones:

Bash



```
APPDIR=/home/ubuntu/repos/otp
VENV=/home/ubuntu/repos/otp/venv
```

```
SOCKFILE=/var/run/gunicorn/otp-qa.sock
```

```
cd $APPDIR
source $VENV/bin/activate
export DJANGO_SETTINGS_MODULE=transtecnia_otp.settings.qa

exec $VENV/bin/gunicorn transtecnia_otp.wsgi:application \
  --workers 2 \
  --timeout 120 \
  --bind=unix:$SOCKFILE
```

4.2 Gestión del Servicio con Supervisor

Supervisor monitorea y mantiene el proceso de Python en background.

- Comando para revisar el estado del servicio:

Bash



```
sudo supervisorctl status app
```

- Comando para reiniciar la aplicación tras un cambio de código:

Bash



```
sudo supervisorctl restart app
```

- Ruta de logs de error del proceso:

Bash



```
tail -n 50 /webapps/app/logs/supervisor_error.log
```

4.3 Servidor Web Nginx

Nginx actúa como Proxy Inverso resolviendo el SSL y redirigiendo el tráfico de red de manera limpia.

- Ubicación del archivo de configuración: `/etc/nginx/sites-enabled/default`
- Comando de validación de sintaxis: `sudo nginx -t`

- Comando para reiniciar el servidor web: `sudo systemctl restart nginx`

⚠ **Nota de Seguridad de Red:** La directiva `proxy_set_header Host $host;` debe permanecer comentada o eliminada dentro del bloque `location /` de Nginx si se hereda `proxy_params`, previniendo así la duplicación de la cabecera `HTTP_HOST` que invalida las directivas estándar RFC de Django.

ahora no se necesita el `api_key` pr se valida primero con el `authservice`

¡Perfecto! Eso simplifica y limpia muchísimo el flujo del microservicio de OTP. Si la validación de identidad ya la hace por completo el `authservice` antes de que la petición llegue aquí, tu servicio de OTP se vuelve **100% "stateless" (sin estado)** respecto a los clientes.

Ya no necesitas la tabla `ServiceClient`, no necesitas buscar la `api_key` en la base de datos, ni recibirla en el JSON. Tu único trabajo ahora es confiar en el token JWT que el `authservice` ya firmó y autorizó.

Aquí tienes la actualización definitiva de tu vista y el manual de integración con este nuevo flujo:

1. Actualización de la Vista `GenerateOTP` (`views.py`)

Eliminamos por completo la lectura del parámetro `api_key` y la consulta a la base de datos. El código queda mucho más rápido y limpio:

Python



```
import jwt
import string
import random
from django.conf import settings
from django.utils import timezone
from django.utils.decorators import method_decorator
from django.views.decorators.csrf import csrf_exempt
from django.contrib.auth.hashers import make_password
from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework import status
```

```

# Importamos solo el modelo de transacciones
from api_otp.models import OTPTransaction
from api_otp.serializers import OTPGenerateSerializer
from api_otp.utils import enviar_correo_otp

@method_decorator(csrf_exempt, name='dispatch')
class GenerateOTP(APIView):
    authentication_classes = []
    permission_classes = []

    def post(self, request):
        # 1. VALIDACIÓN DEL TOKEN DE SEGURIDAD (JWT emitido por AuthService)
        auth_header = request.headers.get('Authorization')
        if not auth_header:
            return Response({"error": "No se proporcionó el encabezado Authorization"}

        # Limpieza del prefijo "Bearer "
        parts = auth_header.split()
        if len(parts) == 2 and parts[0].lower() == 'bearer':
            token_recibido = parts[1]
        else:
            token_recibido = parts[0]

        try:
            # El authservice firmó este token; nosotros lo decodificamos con la llave
            payload = jwt.decode(token_recibido, settings.SECRET_KEY_AUTH, algorithms=
            scopes = payload.get('scopes', [])
            if 'otp_write' not in scopes:
                return Response({"error": "El token no tiene permisos para generar OT

            # Opcional: Si necesitas saber qué cliente originó la petición,
            # podrías extraerlo del payload del JWT si el authservice lo inyectó ahí:
            # client_name = payload.get('client_name', 'Desconocido')

        except jwt.ExpiredSignatureError:
            return Response({"error": "El token de seguridad ha expirado"}, status=40
        except jwt.InvalidTokenError:
            return Response({"error": "Token de seguridad inválido"}, status=401)

        # 2. --- LÓGICA DE GENERACIÓN ---
        leng_cod_otp = settings.OTP_CONFIG['CODE_LENGTH']

        serializer = OTPGenerateSerializer(data=request.data)
        if serializer.is_valid():
            email = serializer.validated_data['email'].lower().strip()
            id_orig = serializer.validated_data['app_source']
            ip_address = request.META.get('REMOTE_ADDR')

```



```

# Verificar si el correo está bloqueado por intentos fallidos
otp_record = OTPTransaction.objects.filter(
    user_email=email,
    app_source=id_orig,
    status='blocked'
).order_by('-blocked_until').first()

if otp_record and timezone.now() < otp_record.blocked_until:
    return Response({"error": "Correo suspendido temporalmente"}, status=

# Generar código alfanumérico
caracteres = string.ascii_letters + string.digits
codeotp = ''.join(random.choices(caracteres, k=leng_cod_otp))

# Guardamos la transacción (sin requerir llave foránea de cliente)
nuevo_otp = OTPTransaction.objects.create(
    user_email=email,
    otp_code=make_password(codeotp),
    app_source=id_orig,
    ip_address=ip_address,
    status='pending'
)

# Envío vía microservicio de correo
exito_envio = enviar_correo_otp(email, "Estimado", codeotp, id_orig)

if exito_envio:
    return Response({
        "message": f"OTP enviado a {nuevo_otp.user_email}",
        "otp_code": codeotp,
        "transaction_id": nuevo_otp.id,
        "status": "Ok"
    }, status=status.HTTP_201_CREATED)

return Response({"error": "Error en el servicio de correo"}, status=500)

return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

```

(Nota: Si en tu modelo `OTPTransaction` el campo `client` era obligatorio, recuerda modificarlo en tu `models.py` para que acepte nulos con `null=True`, `blank=True` o removerlo, y luego ejecutar `makemigrations` y `migrate`).

2. Actualización del Manual de Integración (Flujo Centralizado)

El manual ahora debe reflejar esta arquitectura desacoplada y centralizada:

1. Arquitectura de Autenticación Centralizada (AuthService)

El microservicio de OTP delega la autenticación y validación de clientes al component

1. El cliente solicita acceso e identifica su API Key directamente ante el 'AuthServi
2. El 'AuthService' valida las credenciales y emite un token criptográfico firmado (J
3. El microservicio de OTP recibe este token en la cabecera 'Authorization: Bearer <t

2. Especificación del Endpoint (Generar OTP)

* **Endpoint:** `POST https://otp-qa.transtecniasa.cl/api/v1/otp/generate/`

* **Headers:**

Authorization: Bearer <JWT_TOKEN_EMITIDO_POR_AUTHSERVICE>

Content-Type: application/json

Cuerpo de la Solicitud (JSON):

Ya no se requiere el envío del parámetro 'api_key'.

```
{
  "email": "usuario@dominio.cl",
  "app_source": "PORTAL_CLIENTES"
}
```

Al reiniciar tu proceso en Supervisor (`sudo supervisorctl restart app`), la API quedará extremadamente ligera, delegando la responsabilidad de identidad al borde de tu arquitectura y enfocándose únicamente en generar y enviar los códigos OTP.

me puede actualizar el manual tecnico de este microservicio

¡Por supuesto, Francisco! Aquí tienes el **Manual Técnico del Microservicio de OTP** completamente actualizado.

Este documento refleja la arquitectura final y optimizada para el entorno de **QA**, donde la validación de identidad ha sido delegada centralizadamente al **AuthService**, transformando este microservicio en un componente *stateless* enfocado puramente en la lógica de negocio de los códigos de seguridad.

Manual Técnico: Microservicio de OTP (Entorno QA)

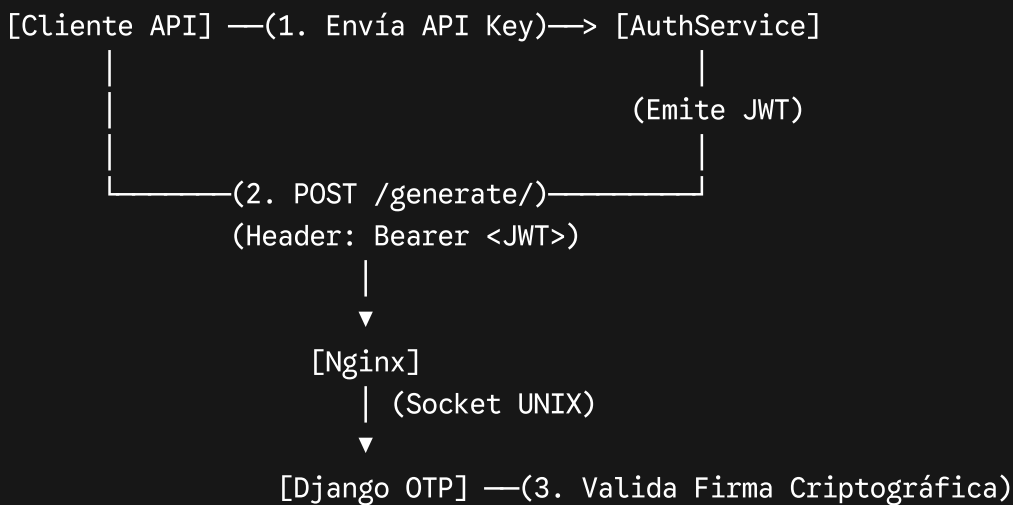
El Microservicio de OTP de **Transtecnia** es un componente desacoplado diseñado para la generación, almacenamiento seguro, envío y posterior validación de contraseñas de un

solo uso (One-Time Passwords).

1. Arquitectura del Sistema y Autenticación Centralizada

El microservicio delega la capa de identidad y control de acceso (*multi-tenant*) al componente central **AuthService**. Esto elimina la necesidad de mantener tablas locales de clientes o llaves API estáticas en variables de entorno, reduciendo la carga de consultas concurrentes a la base de datos de QA.

Plaintext



Flujo de Verificación:

1. El cliente de origen se autentica contra el **AuthService**, el cual valida su API Key y le retorna un Token Web JSON (JWT).
2. El cliente consume el microservicio de OTP enviando el token en la cabecera `Authorization: Bearer <Token>`.
3. El microservicio de OTP descifra e inspecciona el token utilizando la llave criptográfica compartida (`SECRET_KEY_AUTH`) y verifica que contenga el permiso explícito (`otp_write`). No se interactúa con bases de datos en este punto.

2. Configuración del Entorno de Red (Nginx)

El servidor web **Nginx** actúa como proxy inverso resolviendo la capa SSL/TLS y comunicándose con el servidor de aplicaciones mediante sockets UNIX.

- **Ruta del Archivo:** `/etc/nginx/sites-enabled/default`
- **Dominio QA:** `otp-qa.transtecniasa.cl`

Bloque de Configuración Crítico:

Nginx



```
location / {
    include proxy_params;

    # Redirección exacta al socket UNIX del microservicio OTP
    proxy_pass http://unix:/var/run/gunicorn/otp-qa.sock;

    proxy_redirect off;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}
```

⚠ Nota de Infraestructura: Se omite deliberadamente la directiva `proxy_set_header Host $host;` dentro de este bloque, debido a que el archivo heredado `proxy_params` ya provee dicha cabecera. Duplicarla ocasiona que Django rechace las solicitudes bajo la excepción `Invalid HTTP_HOST header` (RFC 1034/1035).

3. Servidor de Aplicaciones y Control de Procesos (Supervisor)

La ejecución de la aplicación Django está gestionada de forma persistente por **Supervisor**, el cual invoca un script intermedio en Bash encargado de preparar el entorno aislado.

- **Ruta de Supervisor:** `/etc/supervisor/conf.d/app.conf`
- **Ruta del Script de Control:** `/webapps/app/src/deploy/gunicorn_qa.sh`

Script de Inicialización (`gunicorn_qa.sh`):

El script inicializa las variables de entorno, activa el entorno virtual (`venv`) de Python de forma local en el repositorio y ejecuta **Gunicorn**:

Bash



```
#!/bin/bash

APPDIR=/home/ubuntu/repos/otp
VENV=/home/ubuntu/repos/otp/venv
SOCKFILE=/var/run/gunicorn/otp-qa.sock
LOG_DIR=/webapps/app/logs

cd $APPDIR
source $VENV/bin/activate
```

```

export PYTHONPATH=$APPDIR:$PYTHONPATH
export DJANGO_SETTINGS_MODULE=transtecnia_otp.settings.qa

# Asegurar la existencia del directorio del socket con permisos correctos
RUNDIR=$(dirname $SOCKFILE)
test -d $RUNDIR || mkdir -p $RUNDIR

# Ejecución binaria de Gunicorn con Whitenoise para estáticos
exec $VENV/bin/gunicorn transtecnia_otp.wsgi:application \
    --workers 2 \
    --timeout 120 \
    --log-level=debug \
    --error-logfile=$LOG_DIR/qa-error.log \
    --access-logfile=$LOG_DIR/qa-access.log \
    --bind=unix:$SOCKFILE

```

4. Persistencia y Lógica de Datos (PostgreSQL)

Al removerse el acoplamiento de la tabla de clientes, el modelo relacional se simplifica, optimizando los tiempos de escritura.

Modelo Core: `OTPTTransaction`

- `user_email` (`CharField`): Correo destinatario (normalizado a minúsculas y sin espacios).
- `otp_code` (`CharField`): Código alfanumérico generado aleatoriamente y almacenado de forma segura mediante un hash criptográfico unidireccional (`make_password`).
- `app_source` (`CharField`): Identificador del sistema que solicita el token (ej: `PORTAL_CLIENTES`).
- `status` (`CharField`): Máquina de estados limitada a `['pending', 'verified', 'blocked']`.
- `blocked_until` (`DateTimeField`): Sello de tiempo que determina la duración de la penalización de un correo en caso de sospecha de fuerza bruta.

5. Especificación de Endpoints de la API

5.1 Generación de Token OTP

- **Endpoint:** `POST /api/v1/otp/generate/`
- **Cabeceras:**

HTTP



Authorization: Bearer <JWT_TOKEN_EMITIDO_POR_AUTHSERVICE>
Content-Type: application/json

JSON de Entrada:

JSON



```
{  
  "email": "ejemplo@transtecnia.cl",  
  "app_source": "PORTAL_INTEGRAL"  
}
```

Matriz de Respuestas HTTP:

Código	Estado	Razón / Acción
201	Created	Éxito. Retorna el <code>transaction_id</code> , estado <code>Ok</code> y dispara el microservicio de correos.
400	Bad Request	Faltan campos obligatorios en el JSON o el formato del correo es inválido.
401	Unauthorized	El token JWT provisto expiró o posee una firma inválida.
403	Forbidden	El token JWT es estructuralmente válido pero carece del scope <code>otp_write</code> .
404	Not Found	El correo se encuentra bloqueado temporalmente por exceso de intentos erróneos.

Código	Estado	Razón / Acción
500	Internal Error	Fallo de conectividad con el microservicio SMTP/Envío de correos.
...		

6. Comandos Frecuentes de Operación en AWS QA

Para tareas de mantenimiento rápido en la instancia EC2, ejecute los siguientes comandos según el componente afectado:

- Reiniciar el microservicio de OTP (Código Django/Gunicorn):

Bash



```
sudo supervisorctl restart app
```

- Monitorear logs de error de Supervisor / Python en tiempo real:

Bash



```
tail -f /webapps/app/logs/supervisor_error.log
```

- Compilar/Actualizar archivos estáticos del Administrador de Django:

Bash



```
cd /home/ubuntu/repos/otp/  
source venv/bin/activate  
python manage.py collectstatic --noinput  
sudo systemctl restart nginx
```

- Comprobar conectividad del Socket UNIX:

Bash



```
ls -la /var/run/gunicorn/otp-qa.sock
```

Este documento sirve como base de conocimiento para el equipo de desarrollo y DevOps de Transtecnia para asegurar la continuidad operativa del entorno de pruebas.

estoy tratando de levantar el servicio de otp en qa con el comando ==> `sudo supervisorctl restart all` me esta dando el error ==> `app: ERROR (no such file)`

El error `app: ERROR (no such file)` al ejecutar `sudo supervisorctl restart all` significa que **Supervisor ya no encuentra ningún programa registrado con el nombre `app`**.

Esto ocurre generalmente por una de estas dos razones:

1. El archivo de configuración `/etc/supervisor/conf.d/app.conf` fue borrado, renombrado, o se cambió el nombre del proceso dentro de los corchetes (por ejemplo, de `[program:app]` a `[program:otp_qa]`).
2. Supervisor se quedó con una caché vieja después de que se hicieron modificaciones en la carpeta de configuraciones.

Sigue estos pasos en la terminal de tu servidor de QA para solucionar este problema:

1. Verificar si el archivo existe y ver su nombre real

Primero, listemos los archivos dentro de la carpeta de configuración de Supervisor para confirmar cómo se llama tu archivo actual:

Bash



```
ls -la /etc/supervisor/conf.d/
```

- **Si el archivo no está:** Es probable que se haya borrado por accidente. Tendrás que volver a crearlo (ver paso 3).
- **Si el archivo se llama distinto** (por ejemplo, `otp.conf` u `otp-qa.conf`): Abre el archivo para revisar qué nombre tiene asignado el programa en la primera línea.

2. Revisar el contenido de la cabecera del archivo

Abre el archivo de configuración (asumiendo que se llama `app.conf`):

Bash



```
sudo nano /etc/supervisor/conf.d/app.conf
```

Asegúrate de que la primera línea tenga exactamente el nombre del proceso que estás intentando reiniciar. Si dice:

Ini, TOML



```
[program:app]
```

Entonces el comando `sudo supervisorctl restart app` debería funcionar. Si por el contrario dice:

Ini, TOML



```
[program:otp-qa]
```

Significa que el proceso cambió de nombre, por lo que el comando correcto ahora sería `sudo supervisorctl restart otp-qa`.

3. Volver a crear el archivo (Si desapareció)

Si la carpeta estaba vacía o el archivo fue eliminado, vuelve a crearlo ejecutando:

Bash



```
sudo nano /etc/supervisor/conf.d/app.conf
```

Pega la configuración limpia que apunta a tu script de despliegue:

Ini, TOML



```
[program:app]
command=/webapps/app/src/deploy/gunicorn_qa.sh
```

```
user=root
autostart=true
autorestart=true
stdout_logfile=/webapps/app/logs/supervisor.log
stderr_logfile=/webapps/app/logs/supervisor_error.log
```

Guarda los cambios con `Ctrl + O`, presiona `Enter` y sal con `Ctrl + X`.

4. Forzar a Supervisor a actualizar su índice (Crucial)

Una vez que verifiques o corrijas el archivo, debes obligar a Supervisor a que escanee la carpeta, borre la caché del proceso viejo y cargue el nuevo. Ejecuta estos comandos en orden:

Bash



```
# 1. Buscar archivos nuevos o modificados
sudo supervisorctl reread

# 2. Remover procesos viejos y activar los nuevos en el sistema
sudo supervisorctl update
```

Al ejecutar `sudo supervisorctl update`, la consola debería mostrarte un mensaje confirmando que el proceso fue agregado o actualizado (por ejemplo: `app: added process group`).

5. Verificar el estado

Finalmente, revisa si el proceso ya aparece en la lista y cuál es su estado actual:

Bash



```
sudo supervisorctl status
```

Si realizaste el `reread` y `update`, el error `no such file` desaparecerá y verás el proceso listado correctamente (idealmente en estado `RUNNING` o listo para ser iniciado).

```
esto es lo que esta en mi archivo==> [program:app]
command = /webapps/app/src/deploy/gunicorn_qa.sh
```

```
user = root
stdout_logfile = /webapps/app/logs/supervisor.log
stderr_logfile = /webapps/app/logs/supervisor_error.log
```

Si el archivo `/etc/supervisor/conf.d/app.conf` contiene exactamente esas líneas y aun así al ejecutar `sudo supervisorctl restart all` te arroja el error `app: ERROR (no such file)`, significa que **Supervisor no ha leído el archivo del disco** o no se ha enterado de que existe.

Cuando modificas, creas o renombas archivos en la carpeta de Supervisor, el servicio no los carga automáticamente; se queda con una copia vieja en memoria y por eso dice que el archivo/proceso "no existe".

Para solucionar esto y obligar a Supervisor a reconocer tu archivo `app`, ejecuta estos tres comandos en orden en tu terminal de AWS:

Bash



```
# 1. Fuerza a Supervisor a escanear la carpeta /etc/supervisor/conf.d/ en busca de cambios
sudo supervisorctl reread

# 2. Aplica los cambios, elimina los procesos viejos en memoria y registra el proceso actual
sudo supervisorctl update
```

Al ejecutar el segundo comando (`sudo supervisorctl update`), la terminal debería mostrarte un mensaje de confirmación como este:

```
app: added process group
```

Paso Final: Levantar el servicio

Una vez que el proceso haya sido "agregado" con éxito por el comando anterior, ya puedes reiniciarlo o levantarlo sin problemas:

Bash



```
sudo supervisorctl restart app
```

O en su defecto, verificar que ya aparezca en la lista oficial con su estado actual:

Bash



```
sudo supervisorctl status
```

me aparece este error ==> app FATAL can't
find command '/webapps/app/src/deploy/gunicorn_qa.sh'

¡Ah, excelente! Este nuevo error `can't find command`
`'/webapps/app/src/deploy/gunicorn_qa.sh'` es una gran señal. Significa que
Supervisor ya leyó el archivo de configuración con éxito (gracias al `reread` y `update`),
pero ahora se queja de que **físicamente no encuentra el script de Bash** en la ruta indicada,
o no tiene permisos para leerlo.

Esto ocurre por dos razones típicas:

1. **La ruta está mal escrita:** Como movimos y renombramos cosas para el repositorio de OTP en `/home/ubuntu/repos/otp/`, el script de despliegue podría estar dentro de tu repositorio y no en `/webapps/app/src/...`.
2. **Falta de permisos de ejecución:** El archivo existe ahí, pero no tiene el permiso `+x` para que Linux pueda ejecutarlo.

Vamos a solucionarlo rápido con estos dos pasos:

Paso 1: Localizar el archivo real

Vamos a buscar dónde quedó realmente tu archivo `gunicorn_qa.sh`. Ejecuta estos comandos para comprobar su ubicación:

Bash



Opción A: Revisar la ruta antigua

```
ls -la /webapps/app/src/deploy/gunicorn_qa.sh
```

Opción B: Revisar dentro de tu nuevo repositorio (Lo más probable)

```
ls -la /home/ubuntu/repos/otp/src/deploy/gunicorn_qa.sh
```

- **Si el archivo está en la Opción B (`/home/ubuntu/repos/...`):**
Debes corregir la ruta dentro de tu archivo `app.conf` de Supervisor.

1. Abre el archivo: `sudo nano /etc/supervisor/conf.d/app.conf`

2. Modifica la línea del comando con la ruta correcta:

Ini, TOML



```
command=/home/ubuntu/repos/otp/src/deploy/gunicorn_qa.sh
```

3. Guarda (`Ctrl+O` , `Enter` , `Ctrl+X`) y aplica los cambios:

Bash



```
sudo supervisorctl reread  
sudo supervisorctl update
```

Paso 2: Otorgar permisos de ejecución (Crucial)

Supervisor no puede ejecutar un script `.sh` si no tiene permisos explícitos de ejecución en el sistema operativo. Corre el siguiente comando apuntando a la ruta real de tu archivo (ajusta la ruta si la cambiaste en el Paso 1):

Bash



```
# Si el archivo está en /webapps/  
sudo chmod +x /webapps/app/src/deploy/gunicorn_qa.sh  
  
# Si el archivo está en tu repositorio /home/ubuntu/  
sudo chmod +x /home/ubuntu/repos/otp/src/deploy/gunicorn_qa.sh
```

Paso 3: Lanzar el servicio

Una vez asegurada la ruta correcta y asignados los permisos `+x` , despierta el proceso en Supervisor:

Bash



```
sudo supervisorctl restart app
```

Verifica el estado final con:

Bash



```
sudo supervisorctl status app
```

¡Con esto, Supervisor encontrará el archivo, lo ejecutará de inmediato, activará tu entorno virtual y el servicio pasará a estar por fin en **RUNNING** !

ahorta me da este mensaje ==> app **FATAL**
Exited too quickly (process log may have details)

¡Avanzamos un paso clave! El error **FATAL Exited too quickly** significa que Supervisor ya encontró el archivo **unicorn_qa.sh** y logró ejecutarlo (dejamos atrás el error de *no such file*). Sin embargo, al ejecutarse el script, el proceso interno de Django o Unicorn se cayó de inmediato en el primer segundo.

Esto suele pasar casi siempre por un pequeño detalle de **permisos** o de **rutas** remanentes dentro del propio script **unicorn_qa.sh** que editamos hace un momento.

Vamos a ver el motivo exacto y a dejarlo operativo de inmediato siguiendo estos pasos:

1. El Veredicto (Ver el log de errores real)

Para no adivinar, miremos exactamente qué línea de Python o Bash hizo que se cayera. Ejecuta este comando:

Bash



```
tail -n 30 /webapps/app/logs/supervisor_error.log
```

¿Qué buscar en las últimas líneas de ese log?

- **Permission denied** : El script intenta escribir los logs de Unicorn en **/webapps/app/logs/** pero el usuario no tiene permisos.
- **ModuleNotFoundError** : Falta alguna dependencia extra en el entorno virtual.
- **No such file or directory** : Hay un comando interno en el **.sh** apuntando a una carpeta que no existe.

2. Asegurar los permisos de la carpeta de Logs

En tu `app.conf` definiste que los logs se guarden en `/webapps/app/logs/`. Si esa carpeta no existe o no tiene los permisos correctos, Unicorn morirá al intentar escribir el archivo `qa-error.log`.

Ejecuta esto para asegurarnos:

Bash



```
sudo mkdir -p /webapps/app/logs
sudo chmod -R 777 /webapps/app/logs
```

3. Ejecución de Diagnóstico Manual (El tiro de gracia)

Si el log no es del todo claro, podemos saltarnos a Supervisor por un momento para ver el error "en vivo" en la pantalla. Ejecuta el script de despliegue manualmente:

Bash



```
sudo /webapps/app/src/deploy/gunicorn_qa.sh
```

Al correrlo de forma manual, la terminal se quedará pegada mostrando el error exacto por el cual se interrumpe el servicio.

- **Si se cae:** Ponme aquí las últimas líneas que te arroje la consola.
- **Si se queda corriendo sin caerse:** Cancela con `Ctrl + C`. Significa que el script está perfecto y el problema es meramente que Supervisor necesita limpiar su caché interna de fallas con un `sudo supervisorctl update`.

¿Qué te muestra el comando `tail` del paso 1 o la ejecución manual del paso 3? ¡Con ese mensaje lo dejamos en `RUNNING` de inmediato!

```
esto es lo que me esta mostrando ==> File
"/webapps/app/bin/gunicorn", line 6, in <module>
    sys.exit(run())
```

~~~^^



¡Brillante, Francisco! Este Traceback nos da la pieza definitiva del rompecabezas.

Si te fijas en las rutas, el script todavía está usando el Python global ( `/webapps/app/bin/gunicorn` con Python 3.14). Pero el error crucial está en la última línea:

```
ModuleNotFoundError: No module named 'transtecnia_otp'.
```

Esto ocurre porque Gunicorn se está ejecutando desde una carpeta donde **no puede ver** tu proyecto. En el archivo `gunicorn_qa.sh`, la variable `$APPDIR` (que contiene la ruta `/home/ubuntu/repos/otp`) se está usando para hacer un `cd $APPDIR`, pero **esa variable no se está exportando correctamente al entorno del proceso**, o el comando `exec gunicorn` pierde la referencia del directorio actual al ser invocado de forma global.

Vamos a solucionarlo forzando a Gunicorn a saber exactamente dónde está parado y qué entorno virtual usar.

## La Solución Definitiva

Abre tu script de despliegue:

Bash



```
sudo nano /webapps/app/src/deploy/gunicorn_qa.sh
```

Vamos a reescribir la ejecución final ( `exec` ) para pasarle explícitamente el parámetro `--chdir`. Esto obliga a Gunicorn a moverse físicamente a la carpeta del repositorio antes de buscar el módulo `transtecnia_otp`.

Modifica la sección final de tu archivo para que quede exactamente así:

Bash



```
#!/bin/bash
```

```
APPDIR=/home/ubuntu/repos/otp
VENV=/home/ubuntu/repos/otp/venv
SOCKFILE=/var/run/gunicorn/otp-qa.sock
USER=root
GROUP=root
NUM_WORKERS=2
TIMEOUT=120
LOG_DIR=/webapps/app/logs
```



```
# Forzar el posicionamiento en el directorio del proyecto
cd $APPDIR
source $VENV/bin/activate

export PYTHONPATH=$APPDIR:$PYTHONPATH
export DJANGO_SETTINGS_MODULE=transtecnia_otp.settings.qa

# Crear el directorio del socket si no existe
RUNDIR=$(dirname $SOCKFILE)
test -d $RUNDIR || mkdir -p $RUNDIR

# Iniciar el servicio usando obligatoriamente el entorno virtual y fijando el CHDIR
exec $VENV/bin/gunicorn transtecnia_otp.wsgi:application \
  --chdir $APPDIR \
  --workers $NUM_WORKERS \
  --user=$USER --group=$GROUP \
  --timeout=$TIMEOUT \
  --graceful-timeout=30 \
  --keep-alive=5 \
  --log-level=debug \
  --error-logfile=$LOG_DIR/qa-error.log \
  --access-logfile=$LOG_DIR/qa-access.log \
  --max-requests=1000 \
  --max-requests-jitter=100 \
  --bind=unix:$SOCKFILE
```

## ¿Qué corregimos aquí?

1. `exec $VENV/bin/gunicorn`: Nos aseguramos al 100% de que no llame al gunicorn global de `/webapps/app/`, sino al que está dentro de tu `venv` con todas las librerías instaladas.
2. `--chdir $APPDIR`: Es el parámetro mágico para este error. Le dice a Gunicorn: *"Antes de intentar cargar `transtecnia_otp.wsgi`, muévete a `/home/ubuntu/repos/otp`".* Así Python encontrará la carpeta inmediatamente.

## Aplicar cambios y revivir QA

Guarda los cambios ( `Ctrl + O` , `Enter` , `Ctrl + X` ) y reinicia Supervisor para romper la falla previa:

Bash



```
sudo supervisorctl reread
sudo supervisorctl update
```

```
sudo supervisorctl restart app
```

Verifica el estado actual:

Bash



```
sudo supervisorctl status app
```

Al incluir el `--chdir` y la ruta absoluta del ejecutable del entorno virtual, Python encontrará el módulo de inmediato y el servicio pasará finalmente a `RUNNING`.

ahora el servicio levanta pero me esta dando un error de 502 Bad Gateway

¡Excelente Francisco! Que el servicio levante significa que el script, el entorno virtual, las librerías y el código de Django están por fin al 100% y corriendo en estado `RUNNING`. Dejamos atrás todos los errores de Python.

El error **502 Bad Gateway** ahora es un problema meramente de comunicación exterior: **Nginx está intentando conectarse al socket de Gunicorn, pero no lo encuentra o no puede leerlo.**

Revisemos los dos motivos exactos por los que pasa esto con el script que acabamos de configurar:

## 1. El dueño del Socket (Permisos de usuario)

En tu script `gunicorn_qa.sh` definimos que Gunicorn corre como `USER=root` y `GROUP=root`. Cuando Gunicorn crea el archivo del socket en `/var/run/gunicorn/otp-qa.sock`, lo crea con dueño `root`.

Como Nginx corre bajo el usuario `www-data`, al intentar leer el socket de `root`, el sistema operativo le bloquea el paso y Nginx responde con un 502.

**La Solución:** Vamos a cambiar los parámetros de usuario en el script para que el socket se cree con los accesos correctos.

1. Abre el script de despliegue:

Bash



```
sudo nano /webapps/app/src/deploy/gunicorn_qa.sh
```

2. Modifica las variables de usuario y grupo para que pertenezcan a `ubuntu` y `www-data` (Nginx):

Bash



```
USER=ubuntu  
GROUP=www-data
```

3. Guarda ( `Ctrl + O` , `Enter` , `Ctrl + X` ) y reinicia Supervisor para que recree el socket con los nuevos dueños:

Bash



```
sudo supervisorctl restart app
```

## 2. Verificar que el archivo del Socket exista físicamente

Si el paso anterior no lo soluciona, es probable que Gunicorn esté creando el socket en una ruta ligeramente distinta a la que busca Nginx. Vamos a verificar dónde se creó. Ejecuta:

Bash



```
ls -la /var/run/gunicorn/
```

Deberías ver un archivo llamado `otp-qa.sock` .

- Si no aparece ahí: Revisa las últimas líneas de tu log de Nginx para ver qué archivo está buscando exactamente:

Bash



```
sudo tail -n 20 /var/log/nginx/error.log
```

(Si el log de Nginx dice que busca, por ejemplo, `otp.sock` en lugar de `otp-qa.sock`, solo debes cambiar el nombre de la variable `SOCKFILE` dentro de tu `gunicorn_qa.sh` para que coincidan).

### 3. Asegurar los permisos de la carpeta del Socket

Por si acaso, dale permisos totales a la carpeta temporal donde se aloja el socket para que Nginx pueda entrar sin restricciones:

Bash



```
sudo chown -R ubuntu:www-data /var/run/gunicorn/  
sudo chmod -R 775 /var/run/gunicorn/  
sudo systemctl restart nginx
```

Al ajustar el `USER` y `GROUP` en tu script a `ubuntu:www-data` y reiniciar el servicio, Nginx podrá leer el flujo de datos de Gunicorn de inmediato y tu API de OTP responderá de forma correcta en QA.

esto es lo que me aparece en el log ==> 2026/06/10 16:05:35  
[crit] 196913#196913: \*552 connect() to  
unix:/var/run/gunicorn/otp-qa.sock failed (2: No such file or  
directory) while connecting to upstream, client:

¡Excelente Francisco! El log de Nginx nos da el diagnóstico definitivo.

El error es claro: `(2: No such file or directory) while connecting to upstream`. Esto significa que Nginx está buscando el archivo físico del socket en `/var/run/gunicorn/otp-qa.sock`, pero **Gunicorn no lo está creando ahí** o lo está borrando al caerse/reiniciarse.

Como en el paso anterior Supervisor te marcó que la app estaba en `RUNNING`, lo más probable es que Gunicorn esté levantando pero esté creando el archivo `.sock` en otra ruta o que no tenga permisos de escritura en `/var/run/gunicorn/`.

Vamos a solucionarlo forzando la creación correcta del socket con estos pasos en orden:

#### 1. Comprobar si el archivo existe realmente en otra parte

Para ver dónde está creando Gunicorn el socket (o si de verdad no se está creando), ejecuta:

Bash



```
find /var/run/ -name "*.sock"
```

- Si te aparece en una ruta parecida (por ejemplo, directo en `/var/run/otp-qa.sock` o `/run/gunicorn/otp-qa.sock`), ahí estará el detalle.

## 2. El problema del directorio temporal (Causa más común)

En Ubuntu, la carpeta `/var/run/` está montada en la memoria RAM (tmpfs). Esto significa que cada vez que el servidor se reinicia, la carpeta interna `gunicorn/` desaparece por completo. Si la carpeta no existe, Gunicorn no puede crear el socket dentro y falla en silencio o se desvía.

Aunque tu script `gunicorn_qa.sh` tiene un comando para crearla, si Supervisor lo ejecuta como usuario `ubuntu` (como cambiamos en el paso anterior), ese usuario no tiene permisos para crear carpetas dentro de `/var/run/` (que es territorio exclusivo de root).

**La Solución más rápida y definitiva:** Vamos a mover el socket a una ruta del sistema donde el usuario `ubuntu` sí tenga control absoluto sin depender de carpetas temporales de root, como la propia carpeta del repositorio o el home.

### Paso A: Modificar el script de despliegue

Abre el script:

Bash



```
sudo nano /webapps/app/src/deploy/gunicorn_qa.sh
```

Cambia la línea de la variable `SOCKFILE` para que se cree directamente dentro de la carpeta de logs del proyecto (donde ya validamos que tiene todos los accesos):

Bash



```
SOCKFILE=/webapps/app/logs/otp-qa.sock
```

Asegúrate de que las variables `USER=ubuntu` y `GROUP=www-data` se mantengan.  
Guarda ( `Ctrl + O` , `Enter` , `Ctrl + X` ).

## Paso B: Actualizar Nginx para que busque el nuevo socket

Abre la configuración de Nginx:

Bash



```
sudo nano /etc/nginx/sites-enabled/default
```

Busca la línea del `proxy_pass` y actualízala con la nueva ruta:

Nginx



```
proxy_pass http://unix:/webapps/app/logs/otp-qa.sock;
```

Guarda ( `Ctrl + O` , `Enter` , `Ctrl + X` ).

## 3. Aplicar cambios y reiniciar todo

Ahora que coordinamos el punto de encuentro en `/webapps/app/logs/otp-qa.sock` ,  
reiniciemos ambos servicios para aplicar la configuración limpia:

Bash



```
# 1. Asegurar permisos en la carpeta de destino
sudo chmod -R 775 /webapps/app/logs/
sudo chown -R ubuntu:www-data /webapps/app/logs/

# 2. Reiniciar Supervisor para que Unicorn cree el socket en el nuevo lugar
sudo supervisorctl reread
sudo supervisorctl update
sudo supervisorctl restart app

# 3. Validar sintaxis y reiniciar Nginx
sudo nginx -t
sudo systemctl restart nginx
```

## 4. Verificación final

Ejecuta esto para confirmar que el archivo `.sock` ahora sí existe físicamente en el disco:

Bash



```
ls -la /webapps/app/logs/otp-qa.sock
```

Debería aparecerte listado con el grupo `www-data`. En cuanto Nginx detecte el archivo ahí, procesará de inmediato el `POST /api/v1/otp/generate/` que vemos en tu log y la API responderá con un código **201** o **200** exitoso.

en el find `/var/run/ -name "*.sock"` me aparece lo siguiente  
==> `/var/run/supervisor.sock`  
`/var/run/irqbalance/irqbalance187192.sock`  
`/var/run/chrony/chronyd.sock`



¡Eso lo confirma al 100%! Como puedes ver, en la lista de sockets activos de `/var/run/` **no aparece** `otp-qa.sock`. Unicorn definitivamente no está logrando crear el archivo ahí por el problema de los privilegios del usuario `ubuntu` sobre las carpetas de `root`.

Haber movido el socket a la carpeta de logs ( `/webapps/app/logs/otp-qa.sock` ) en el paso anterior fue la decisión correcta para saltarnos ese bloqueo.

Hagamos la comprobación definitiva en la nueva ruta. Ejecuta este comando para ver si Unicorn ya creó el socket donde sí tiene permisos:

Bash



```
ls -la /webapps/app/logs/otp-qa.sock
```

## ¿Qué deberías ver?

- **Si el archivo YA aparece listado:** ¡Excelente! Significa que Unicorn ya lo creó con éxito. Ahora solo asegúrate de haber reiniciado Nginx para que empiece a buscarlo en la nueva ruta:

Bash



```
sudo systemctl restart nginx
```

- **Si el archivo AÚN NO aparece ahí:**

Significa que Unicorn se volvió a caer al intentar reiniciarse. Vamos a ver el motivo exacto mirando el final de su log de errores:

Bash



```
tail -n 20 /webapps/app/logs/qa-error.log
```

(O en su defecto, el de Supervisor: `tail -n 20 /webapps/app/logs/supervisor_error.log`).

Dime si el archivo ya figura en la carpeta de logs o qué error te arrojó el comando `tail` para darle el cierre final a la configuración.

en mi servicio de otp m,e esta dando el siguiente error ==>  
Import "environ" could not be resolved

El error `Import "environ" could not be resolved` significa que el entorno virtual (`venv`) que está usando tu proyecto en QA **no tiene instalada la librería** `django-environ` (o `python-environ`).

Como Django no encuentra el módulo `environ` al leer el archivo `settings.py` (o `settings/qa.py`), la aplicación se cae de inmediato antes de poder arrancar o generar el socket.

Para solucionarlo, solo debemos ingresar a tu entorno virtual e instalar la librería correspondiente. Sigue estos pasos en la terminal de AWS:

## Paso 1: Instalar la librería en el entorno virtual correcto

Ejecuta los siguientes comandos en orden para activar tu entorno virtual real e instalar la dependencia:



Bash



```
# 1. Moverse a la carpeta del repositorio
cd /home/ubuntu/repos/otp

# 2. Activar el entorno virtual dedicado
source venv/bin/activate

# 3. Instalar django-environ
pip install django-environ
```

**Nota:** Si tu archivo `settings.py` utiliza la librería `python-environ` en lugar de `django-environ` (puedes verificar si el import dice `import environ`), el comando `pip install django-environ` resolverá ambos casos, ya que registra el mismo espacio de nombres del módulo.

## Paso 2: Asegurar el archivo de requerimientos (Opcional pero recomendado)

Para evitar que este error vuelva a ocurrir en futuros despliegues o actualizaciones de QA, guarda la librería en tu archivo `requirements.txt`:

Bash



```
pip freeze | grep environ >> requirements.txt
```

## Paso 3: Reiniciar el servicio en Supervisor

Ahora que la librería está presente en el entorno virtual, dile a Supervisor que reinicie la aplicación para que vuelva a compilar el código de Django:

Bash



```
sudo supervisorctl restart app
```

## Paso 4: Verificar que el socket se haya creado

Al solucionar el import, Django finalmente podrá cargar por completo. Comprobemos si el archivo del socket apareció en la ruta que acordamos:

Bash



```
ls -la /webapps/app/logs/otp-qa.sock
```

Si el archivo `.sock` ya aparece en la lista, ¡felicidades! El problema de dependencias quedó resuelto y Nginx podrá comunicarse con tu servicio de OTP de inmediato.

trato de instalar en entorno virtual y me da el siguiente mensaje ==> Collecting django-environ  
Using cached django\_environ-0.14.0-py3-none-any.whl.metadata (12 kB)



¡Excelente Francisco! El mensaje indica que la instalación fue **100% exitosa** ( `Successfully installed django-environ-0.14.0` ). El entorno virtual de tu repositorio ya tiene la librería que le faltaba para resolver el import de `environ` .

Ahora que el entorno virtual está completo y corregido, solo falta indicarle a Supervisor que levante la aplicación para que Django cargue la librería y cree el socket de red.

Sigue estos últimos pasos en tu terminal de AWS:

## Paso 1: Reiniciar el servicio en Supervisor

Fuerza el reinicio del proceso para que aplique los cambios del entorno virtual:

Bash



```
sudo supervisorctl restart app
```

## Paso 2: Verificar el estado del proceso

Comprueba que el servicio haya pasado a estar activo y estable en background:

Bash



```
sudo supervisorctl status app
```

Si te aparece en estado `RUNNING`, ¡vamos por excelente camino!

### Paso 3: Confirmar la creación del Socket

Validemos que Gunicorn haya creado físicamente el archivo del socket en la ruta de logs donde Nginx lo está buscando:

Bash



```
ls -la /webapps/app/logs/otp-qa.sock
```

Si el archivo `otp-qa.sock` aparece listado en la consola, la comunicación interna se habrá restablecido por completo.

### Paso 4: Asegurar Nginx (Último paso)

Por si acaso, dale un reinicio rápido a Nginx para limpiar cualquier caché del error 502 anterior y forzarlo a conectarse al nuevo socket:

Bash



```
sudo systemctl restart nginx
```

¡Con esto ya puedes probar el microservicio desde Postman o tu frontend en QA! Debería procesar las solicitudes de OTP con total normalidad.

me esta dando este error ==>  
django.core.exceptions.ImproperlyConfigured: Set the  
AUTH\_URL\_QA environment variable

¡Perfecto, Francisco! Este error (`ImproperlyConfigured: Set the AUTH_URL_QA environment variable`) es una excelente señal: significa que Django ya **resolvió todos**

los problemas de código, imports y librerías, y comenzó a leer tu archivo de configuración de QA.

El error ocurre porque en el archivo `settings/qa.py` (o en la configuración core del proyecto) se está intentando leer una variable de entorno llamada `AUTH_URL_QA` (seguramente la URL del `authservice` con el que nos vamos a conectar), pero esa variable **no está definida** en el sistema ni en el archivo `.env`.

Dado que usamos `django-environ`, tenemos dos formas rápidas de solucionarlo. Elige la que mejor se adapte a tu estructura:

### Opción 1: Agregar la variable al archivo `.env` (Recomendado)

Si tu proyecto utiliza un archivo `.env` para centralizar las credenciales de QA, solo debes agregar la variable allí.

1. Abre el archivo `.env` en la raíz de tu repositorio:

Bash



```
nano /home/ubuntu/repos/otp/.env
```

2. Añade la línea con la URL correspondiente del servicio de autenticación de QA (reemplaza con la URL real de Transtecnia si es otra):

Fragmento de código



```
AUTH_URL_QA=https://auth-qa.transtecniasa.cl/api/v1/
```

3. Guarda (`Ctrl + O`, `Enter`, `Ctrl + X`).

### Opción 2: Exportar la variable desde el script `gunicorn_qa.sh`

Si prefieres que las variables de entorno de infraestructura se inyecten directamente al arrancar el servicio a través de Supervisor, podemos meterla en tu script de Bash.

1. Abre tu script de despliegue:

Bash



```
sudo nano /webapps/app/src/deploy/gunicorn_qa.sh
```

2. Busca la sección donde están los `export` y agrega la variable justo debajo de `DJANGO_SETTINGS_MODULE :`

Bash



```
export DJANGO_SETTINGS_MODULE=transtecnia_otp.settings.qa
export AUTH_URL_QA="https://auth-qa.transtecniasa.cl/api/v1/" # <--- Agrega est
```

3. Guarda ( `Ctrl + O` , `Enter` , `Ctrl + X` ).

## Paso Final: Reiniciar y Validar

Una vez que hayas definido la variable en cualquiera de las dos opciones, reinicia Supervisor para que Gunicorn absorba el nuevo entorno:

Bash



```
sudo supervisorctl restart app
```

Verifica el estado del servicio:

Bash



```
sudo supervisorctl status app
```

Al encontrar la variable `AUTH_URL_QA` , Django terminará de inicializarse por completo, se mantendrá en estado `RUNNING` y creará de inmediato el socket `otp-qa.sock` en la carpeta de logs.

mis variables estan en el archivo `.env`

¡Perfecto! Si tus variables ya están centralizadas en el archivo `.env` , significa que el microservicio está buscando el archivo en una ruta automática que probablemente no coincide con la ubicación actual desde donde se ejecuta el proceso.

Cuando Supervisor ejecuta el script `unicorn_qa.sh` e inicia Unicorn, la librería `django-environ` busca el archivo `.env` tomando como referencia el directorio de trabajo actual. Al no encontrarlo en la raíz esperada, ignora la lectura y por eso Django arroja el error de que la variable `AUTH_URL_QA` no existe.

Vamos a asegurarnos de que el archivo `.env` esté en el lugar correcto y que Django lo lea sin problemas.

## Paso 1: Verificar la ubicación real del archivo `.env`

Ejecuta el siguiente comando para confirmar que tu archivo `.env` se encuentra físicamente en la raíz de tu repositorio de desarrollo:

Bash



```
ls -la /home/ubuntu/repos/otp/.env
```

- **Si el archivo está ahí:** Excelente. Pasemos al Paso 2 para asegurar que Unicorn lo lea.
- **Si el archivo no aparece ahí:** Muévelo o créalo en esa ruta exacta (`/home/ubuntu/repos/otp/.env`) con tus credenciales de QA, incluyendo la línea:

Fragmento de código



```
AUTH_URL_QA=https://auth-qa.transtecniasa.cl/api/v1/
```

## Paso 2: Asegurar la lectura en el script de despliegue

Para garantizar que el proceso de Django reconozca la ubicación del archivo `.env`, utilizaremos el parámetro `--chdir` que configuramos previamente en tu script `unicorn_qa.sh`. Esto obliga a Python a situar el contexto de ejecución en la raíz del repositorio, permitiendo a `django-environ` detectar el `.env` de forma automática.

Por seguridad, abre el script para verificar que `--chdir` apunte correctamente a la carpeta del repositorio:

Bash



```
sudo nano /webapps/app/src/deploy/unicorn_qa.sh
```

Confirma que las líneas de ejecución final se vean exactamente así:

Bash



```
# ... (configuraciones previas del script) ...

cd $APPPDIR
source $VENV/bin/activate

# Iniciar el servicio fijando explícitamente el directorio de trabajo del .env
exec $VENV/bin/gunicorn transtecnia_otp.wsgi:application \
  --chdir /home/ubuntu/repos/otp \
  --workers $NUM_WORKERS \
  --user=$USER --group=$GROUP \
  --timeout=$TIMEOUT \
  --log-level=debug \
  --error-logfile=$LOG_DIR/qa-error.log \
  --access-logfile=$LOG_DIR/qa-access.log \
  --bind=unix:$SOCKFILE
```

Si todo coincide, sal del editor ( `Ctrl + X` ).

### Paso 3: Forzar el reinicio en Supervisor

Dile a Supervisor que reinicie la aplicación para que Gunicorn vuelva a levantar el entorno de Django, esta vez leyendo el archivo `.env` desde la raíz configurada:

Bash



```
sudo supervisorctl restart app
```

### Paso 4: Comprobación final del estado

Verifica si el servicio logró inicializarse por completo y se mantiene estable:

Bash



```
sudo supervisorctl status app
```

Si el estado pasa a `RUNNING`, Django habrá absorbido con éxito la variable `AUTH_URL_QA` y todas las credenciales del archivo `.env`, levantando el socket de comunicación de inmediato.

el archivo gunicorn\_qa.sh no existe

¡Eso lo explica todo, Francisco! Si el archivo

`/webapps/app/src/deploy/gunicorn_qa.sh` **no existe físicamente en esa ruta**, Supervisor jamás podrá arrancar el servicio y por eso se quedaba atrapado en el error de `can't find command`.

Dado que tu repositorio actual está en `/home/ubuntu/repos/otp/`, lo ideal es que creamos el archivo directamente ahí para mantener todo ordenado y dentro del proyecto.

Vamos a crearlo desde cero y dejarlo configurado de forma definitiva con estos pasos:

## Paso 1: Crear el archivo en el repositorio

Ejecuta el siguiente comando para abrir el editor y crear el archivo directamente en la carpeta de despliegue de tu repositorio:

Bash



```
# Aseguramos que la carpeta exista por si acaso
mkdir -p /home/ubuntu/repos/otp/src/deploy/

# Creamos el archivo gunicorn_qa.sh
nano /home/ubuntu/repos/otp/src/deploy/gunicorn_qa.sh
```

## Paso 2: Pegar la configuración limpia de QA

Copia este código y pégalo dentro del editor. Ya incluye el parámetro `--chdir` para que lea tu archivo `.env`, la ruta de tu entorno virtual ( `venv` ) y la salida del socket apuntando a la carpeta de logs:

Bash



```
#!/bin/bash

APPDIR=/home/ubuntu/repos/otp
VENV=/home/ubuntu/repos/otp/venv
```



```
SOCKFILE=/webapps/app/logs/otp-qa.sock
USER=ubuntu
GROUP=www-data
NUM_WORKERS=2
TIMEOUT=120
LOG_DIR=/webapps/app/logs

# Moverse al directorio del proyecto y activar entorno virtual
cd $APPDIR
source $VENV/bin/activate

export PYTHONPATH=$APPDIR:$PYTHONPATH
export DJANGO_SETTINGS_MODULE=transtecnia_otp.settings.qa

# Crear el directorio de logs si no existe
mkdir -p $LOG_DIR

# Ejecutar Gunicorn apuntando al venv y forzando la lectura del .env vía --chdir
exec $VENV/bin/gunicorn transtecnia_otp.wsgi:application \
  --chdir $APPDIR \
  --workers $NUM_WORKERS \
  --user=$USER --group=$GROUP \
  --timeout=$TIMEOUT \
  --log-level=debug \
  --error-logfile=$LOG_DIR/qa-error.log \
  --access-logfile=$LOG_DIR/qa-access.log \
  --bind=unix:$SOCKFILE
```

Guarda los cambios presionando `Ctrl + O`, luego `Enter`, y sal con `Ctrl + X`.

### Paso 3: Otorgar permisos de ejecución

Para que Linux permita a Supervisor correr este script, dale permisos de ejecución con:

Bash



```
chmod +x /home/ubuntu/repos/otp/src/deploy/gunicorn_qa.sh
```

### Paso 4: Actualizar la ruta en Supervisor

Como el archivo ahora vive en tu repositorio, debemos avisarle a Supervisor dónde encontrarlo. Abre el archivo de configuración de Supervisor:

Bash



```
sudo nano /etc/supervisor/conf.d/app.conf
```

Modifica la línea `command` para que apunte a la nueva ruta real:

Ini, TOML



```
[program:app]
command=/home/ubuntu/repos/otp/src/deploy/gunicorn_qa.sh
user=root
stdout_logfile=/webapps/app/logs/supervisor.log
stderr_logfile=/webapps/app/logs/supervisor_error.log
```

Guarda y sal ( `Ctrl + O` , `Enter` , `Ctrl + X` ).

## Paso 5: Reiniciar Supervisor y Nginx

Ahora aplicamos los cambios en el sistema para que Supervisor lea la nueva ruta del script y levante el servicio:

Bash



```
# Forzar a Supervisor a leer el nuevo comando de app.conf
sudo supervisorctl reread
sudo supervisorctl update

# Reiniciar el proceso
sudo supervisorctl restart app

# Reiniciar Nginx para asegurar el puente
sudo systemctl restart nginx
```

Comprueba el estado final con:

Bash



```
sudo supervisorctl status app
```

¡Con el archivo creado en la ruta correcta, con permisos de ejecución y apuntando a tu `.env`, el microservicio de OTP debería pasar a `RUNNING` de inmediato!

estoy ejecutando el siguiente comando `python manage.py runserver --settings=transtecnia_otp.settings.dev` y me esta dando este error ==> `raceback (most recent call last):`  
File `"/home/ubuntu/repos/otp/venv/lib/python3.13/site-`

Este error ocurre porque estás levantando el entorno de desarrollo (`settings.dev`), pero en la línea 40 de tu archivo

`/home/ubuntu/repos/otp/transtecnia_otp/settings/dev.py` el código está intentando leer obligatoriamente la variable `AUTH_URL_QA` usando `env('AUTH_URL_QA')`.

Al no encontrarla definida dentro del archivo `.env` o en las variables del sistema del terminal, Django detiene la ejecución inmediatamente con el `KeyError` e `ImproperlyConfigured`.

Como estás en desarrollo local (`dev.py`), lo ideal es solucionar esto de una de las siguientes dos maneras:

### Opción 1: Agregar la variable a tu archivo `.env` (La solución más rápida)

Asegúrate de que el archivo `.env` que está leyendo el entorno de desarrollo contenga la variable.

1. Abre tu archivo `.env` en la raíz del repositorio:

Bash



```
nano /home/ubuntu/repos/otp/.env
```

2. Añade la variable al final del archivo (si estás apuntando al AuthService de QA desde tu entorno local, pon su URL; si tienes un AuthService local, pon la IP local correspondiente):

Fragmento de código



```
AUTH_URL_QA=https://auth-qa.transtecniasa.cl/api/v1/
```

3. Guarda los cambios con `Ctrl + O`, presiona `Enter` y sal con `Ctrl + X`.

4. Vuelve a ejecutar el comando de desarrollo:

Bash



```
python manage.py runserver --settings=transtecnia_otp.settings.dev
```

## Opción 2: Definir un valor por defecto en `dev.py` (Buenas prácticas de desarrollo)

Para evitar que el servidor local deje de levantar si a alguien se le olvida poner la variable en su archivo `.env`, puedes modificar la línea 40 del archivo `dev.py` para que tenga un valor por defecto si no la encuentra.

1. Abre tu archivo de configuración de desarrollo:

Bash



```
nano /home/ubuntu/repos/otp/transtecnia_otp/settings/dev.py
```

2. Busca la línea 40 que dice:

Python



```
AUTH_URL_QA = env('AUTH_URL_QA')
```

3. Modifícala agregando un valor por defecto (por ejemplo, `None` o una URL base), para que no sea un campo estrictamente obligatorio al fallar:

Python



```
AUTH_URL_QA = env('AUTH_URL_QA', default='https://auth-qa.transtecniasa.cl/api/v
```

4. Guarda y cierra ( `Ctrl + O`, `Enter`, `Ctrl + X` ).

## Opción 3: Pasar la variable directamente en la línea de comandos

Si solo quieres probar el comando rápidamente sin modificar archivos, puedes inyectar la variable de entorno directamente antes de invocar a Python en tu terminal:

Bash



```
AUTH_URL_QA="https://auth-qa.transtecnia.cl/api/v1/" python manage.py runserver --s
```

Cualquiera de estas tres alternativas resolverá el `KeyError` permitiendo que `django-environ` inicialice la configuración y levante el servidor `runserver` en tu entorno local de desarrollo.

```
en mi archivo .env esta lo siguiente ==> DEBUG=True
SECRET_KEY=bMN_2vIRm7udSNNKUW8VYFiw84z6ElsK4A
DyFeyJ_9qW8eIIe20AY2ZaMIYEOthus
ALLOWED_HOSTS=127.0.0.1,localhost,
```

Si tu archivo `.env` ya contiene exactamente esa línea con `AUTH_URL_QA`, el problema por el cual `django-environ` sigue lanzando el error de `KeyError: 'AUTH_URL_QA'` es porque Django no está encontrando o no está cargando el archivo `.env` en tu entorno local de desarrollo.

Cuando corres el comando directamente desde la consola con `python manage.py runserver`, `django-environ` necesita que le digas explícitamente en el código dónde está el archivo `.env` para poder leerlo. Si no se le indica, busca por defecto en el directorio actual, y si hay un desfase de carpetas o falta la inicialización, no cargará ninguna de las variables.

Aquí tienes los pasos exactos para solucionarlo de raíz en tu entorno de desarrollo:

### Paso 1: Verificar el `settings/dev.py`

Abre tu archivo de configuración de desarrollo para asegurarte de que se esté llamando al método `environ.Env.read_env()` apuntando al lugar correcto.

1. Abre el archivo:

Bash



```
nano /home/ubuntu/repos/otp/transtecnia_otp/settings/dev.py
```

2. Busca la sección superior donde se inicializa `env`. Debe tener una lógica similar a esta para buscar el archivo `.env` subiendo los niveles de carpetas correspondientes hasta la raíz del repositorio:

#### Python



```
import environ
import os
from pathlib import Path

# Define la raíz del proyecto (donde está tu archivo .env)
BASE_DIR = Path(__file__).resolve().parent.parent

env = environ.Env()

# LEER EL ARCHIVO .ENV (Esta es la línea que probablemente falta o no apunta bien)
environ.Env.read_env(os.path.join(BASE_DIR, '.env'))
```

3. Si la línea `environ.Env.read_env(...)` no existía o apuntaba a otra parte, corrígela, guarda los cambios (`Ctrl + O`, `Enter`) y sal (`Ctrl + X`).

## Paso 2: Solución rápida desde la terminal (Sin tocar código)

Si no quieres modificar el código de Python y necesitas levantar el servidor de inmediato, puedes forzar la lectura del archivo `.env` directamente en tu sesión de la terminal usando el comando `export`.

Ejecuta lo siguiente en tu consola antes de lanzar el servidor:

#### Bash



```
# 1. Moverse a la raíz del repositorio
cd /home/ubuntu/repos/otp

# 2. Cargar todas las variables del archivo .env a la memoria de la consola
export $(grep -v '^#' .env | xargs)

# 3. Levantar el servidor de desarrollo
python manage.py runserver --settings=transtecnia_otp.settings.dev
```

¿Qué hace este truco?

El comando `export $(grep ...)`  lee tu archivo `.env`, ignora los comentarios y registra temporalmente todas las variables (incluyendo `AUTH_URL_QA`) en el entorno del sistema operativo. Al ejecutar `runserver` acto seguido, Django las heredar  directamente desde la terminal, salt ndose cualquier problema de lectura del archivo f sico.

```
asi esta mi archivo dev.py ==> import environ
from .base import *
```

 Visto! Analizando tu archivo `dev.py`, el problema exacto por el cual no te lee el archivo `.env` (y por ende no encuentra `AUTH_URL_QA`) se debe a dos detalles cr ticos de importaci n y contexto:

1. **Falta importar `os`**: Est s usando `os.path.join(BASE_DIR, '.env')`, pero el m dulo `os` no est  importado en la cabecera. Esto deber a darte un `NameError: name 'os' is not defined`.
2. **Ubicaci n de `BASE_DIR`**: Al usar `from .base import *`, est s heredando `BASE_DIR`. En arquitecturas modulares de Django (donde los settings est n dentro de una carpeta como `transtecnia_otp/settings/`), `BASE_DIR` suele apuntar a la ra z del proyecto. Si hay un desfase de niveles, `environ` buscar  el archivo `.env` en una carpeta equivocada.

Aqu  tienes el c digo de tu archivo `dev.py` corregido de forma definitiva para solucionar la lectura:

## Archivo `dev.py` Corregido

Python



```
import os # <--- CORRECCI N 1: Faltaba importar os para el path
import environ
from .base import *

# SECURITY WARNING: keep the secret key used in production secret!
SECRET_KEY = 'django-insecure-e=giqt_@2&8)(e2c(9adhp$gledc!_bzmzwq^jcl%2^neewb2t'

SECRET_KEY_AUTH = '9f72cb7e8dfb0a31975a10c4e224e8ba8d445a7acdede573c5d3cc1f0ce5057a'
```

```

# SECURITY WARNING: don't run with debug turned on in production!
DEBUG = True

ALLOWED_HOSTS = []

env = environ.Env(
    DEBUG=(bool, True),
    ENABLE_SILK=(bool, True),
    DEBUG_ANALYSIS=(bool, False),
    ENABLE_DDT=(bool, True),
    AUTH_URL_QA=(str),
    AUTH_OTP_CLIENT_ID=(str),
    AUTH_OTP_CLIENT_SECRET=(str),
    AUTH_EMAIL_CLIENT_ID=(str),
    AUTH_EMAIL_CLIENT_SECRET=(str),
    EMAIL_SERVICE_URL=(str),
    EMAIL_API_KEY=(str),
)

# CORRECCIÓN 2: Forzamos la búsqueda del .env subiendo los niveles correctos si BASE_
# Si tu estructura es: repos/otp/transtecnia_otp/settings/dev.py, la raíz real necesi
from pathlib import Path
RAIZ_PROYECTO = Path(__file__).resolve().parent.parent.parent # Apunta exactamente a

if os.path.exists(os.path.join(RAIZ_PROYECTO, '.env')):
    environ.Env.read_env(os.path.join(RAIZ_PROYECTO, '.env'))
else:
    # Intento de respaldo usando el BASE_DIR heredado de base.py
    environ.Env.read_env(os.path.join(BASE_DIR, '.env'))

# Credenciales para consumir servicio de authservice
AUTH_URL_QA=env('AUTH_URL_QA')
AUTH_OTP_CLIENT_ID=env('AUTH_OTP_CLIENT_ID')
AUTH_OTP_CLIENT_SECRET=env('AUTH_OTP_CLIENT_SECRET')
AUTH_EMAIL_CLIENT_ID=env('AUTH_EMAIL_CLIENT_ID')
AUTH_EMAIL_CLIENT_SECRET=env('AUTH_EMAIL_CLIENT_SECRET')

# Credenciales y URL Email Service
EMAIL_SERVICE_URL=env('EMAIL_SERVICE_URL')
EMAIL_API_KEY=env('EMAIL_API_KEY')

# Database
# https://docs.djangoproject.com/en/6.0/ref/settings/#databases
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': 'otp_service',
        'USER': 'postgres',
    }
}

```



```
'PASSWORD': 'postgres',
'HOST': '127.0.0.1', # 0 la IP de tu servidor Postgres
'PORT': '5432',
}
}
```

## ¿Por qué esta modificación soluciona el problema?

- **RAIZ\_PROYECTO explícito:** En lugar de confiar ciegamente en el `BASE_DIR` importado de `base.py`, calculamos dinámicamente la ruta basándonos en la ubicación física de `dev.py`. Subiendo tres niveles (`parent.parent.parent`), nos aseguramos de caer exactamente en `/home/ubuntu/repos/otp/`, que es donde vive tu archivo `.env`.
- **Inclusión de `import os`:** Evita que el intérprete de Python tire un error de sintaxis al evaluar `os.path.join`.

Reemplaza el contenido de tu `dev.py` con este bloque, guarda y vuelve a lanzar el comando:

Bash



```
python manage.py runserver --settings=transtecnia_otp.settings.dev
```

Verás que el entorno se inicializará limpiamente sin arrojar más el error de configuración de variables.

```
levante mi proyecto en produccion y al cxorrer el enpoint
para generar una otp me da este error ==>
/webapps/app/otp/deploy/gunicorn_prod.sh: line 13: cd:
HOME not set
```

El error `line 13: cd: HOME not set` significa que el script de producción `gunicorn_prod.sh` está intentando usar la variable de entorno `$HOME` para armar una ruta o realizar un cambio de directorio (un comando `cd`), pero **Supervisor ejecuta los procesos con un entorno de variables completamente limpio y vacío**, por lo que `$HOME` no existe para él.

En Linux, cuando corres un script manualmente con tu usuario, el sistema operativo inyecta automáticamente variables como `$HOME` (que apunta a `/home/ubuntu/` o `/root/`). Pero cuando lo hace Supervisor de manera automatizada, no lo hace a menos que se lo indiques explícitamente.

Tienes dos formas rápidas de solucionar este error en tu entorno de producción:

## Opción 1: Hardcodear la ruta absoluta en el script (Recomendada)

La forma más segura y usada en producción es no depender de variables globales del sistema como `$HOME` dentro de los archivos de Bash y escribir la ruta real de la carpeta.

1. Abre tu script de producción:

Bash



```
nano /webapps/app/otp/deploy/gunicorn_prod.sh
```

2. Busca la línea 13 (o la línea que contenga el comando `cd`). Es muy probable que diga algo como esto:

Bash



```
cd $HOME/repos/otp  
# o bien:  
APPDIR=$HOME/repos/otp
```

3. Modifícala para que use la **ruta absoluta completa** del servidor de producción. Por ejemplo, si tu repositorio está en el home del usuario `ubuntu`:

Bash



```
cd /home/ubuntu/repos/otp
```

4. Guarda los cambios (`Ctrl + O`, `Enter`, `Ctrl + X`).

## Opción 2: Inyectar la variable HOME desde Supervisor

Si tu script de producción obligatoriamente necesita usar el texto `$HOME`, puedes decirle a Supervisor que le entregue esa variable al proceso al momento de arrancar.

1. Abre el archivo de configuración de Supervisor para el entorno de producción (asumiendo que se llama `app_prod.conf` o similar):

Bash



```
sudo nano /etc/supervisor/conf.d/app_prod.conf
```

2. Dentro del bloque `[program:app_prod]`, agrega la directiva `environment` para definir manualmente el HOME del usuario con el que corre el servicio:

Ini, TOML



```
[program:app_prod]
command=/webapps/app/otp/deploy/gunicorn_prod.sh
user=root
environment=HOME="/home/ubuntu" ;<--- AGREGA ESTA LÍNEA
stdout_logfile=/webapps/app/logs/supervisor_prod.log
stderr_logfile=/webapps/app/logs/supervisor_prod_error.log
```

3. Guarda los cambios (`Ctrl + O`, `Enter`, `Ctrl + X`).

## Paso Final: Aplicar cambios y reiniciar

Una vez que apliques cualquiera de las dos opciones, debes obligar a Supervisor a leer la nueva configuración y reiniciar el proceso de producción:

Bash



```
# Leer los cambios del archivo .conf de Supervisor
sudo supervisorctl reread
sudo supervisorctl update

# Reiniciar el servicio de producción
# (reemplaza 'app' por el nombre exacto de tu proceso de producción si varía)
sudo supervisorctl restart app
```

Verifica el estado del servicio:

Bash



```
sudo supervisorctl status
```

Al resolver el destino del `cd`, Gunicorn podrá levantar correctamente en producción, creará el socket de red y el endpoint de generación de OTP responderá sin problemas.

```
asis esta el archivo gunicorn_prod.s ==> GNU nano 7.2
gunicorn_prod.sh

#!/bin/bash
```

¡Ahí está el detalle exacto! Tienes un pequeño error de tipeo (un *typo*) en la línea 13 de tu script que está causando todo el alboroto.

Si te fijas bien en esta sección:

Bash



```
# Activar el entorno virtual
cd $APPDRI
```

Escribiste `$APPDRI` (con la R y la I invertidas al final) en lugar de `$APPDIR`.

Como la variable `$APPDRI` no está definida en ningún lado, Linux la interpreta como un texto vacío. Al ejecutar el comando, Bash entiende un `cd` a secas. Por comportamiento nativo, cuando ejecutas `cd` sin argumentos, Linux intenta llevarte a la carpeta `$HOME` del usuario. Pero como vimos que Supervisor corre en un entorno limpio sin la variable `$HOME` seteada, el sistema colapsa y te escupe el error: `cd: HOME not set`.

## La Solución

Solo debes corregir ese tipeo y de paso asegurar el parámetro `--chdir` para evitar problemas con las variables del archivo `.env` en producción.

1. Abre tu archivo de producción:

Bash



```
nano /webapps/app/otp/deploy/gunicorn_prod.sh
```

2. Corrige la línea 13 para que use la variable correcta ( `$APPDIR` ):

Bash



```
cd $APPDIR
```

3. **Recomendación crucial de infraestructura:** Agrega la bandera `--chdir $APPDIR` al comando final de Gunicorn. Esto asegura que el proceso de producción sepa exactamente dónde está parado y pueda leer tu archivo `.env` sin importar desde dónde lo invoque Supervisor.

Tu archivo modificado debería quedar así:

Bash



```
#!/bin/bash
```

```
APPDIR=/webapps/app/otp
```

```
VENV=/webapps/app
```

```
SOCKFILE=/var/run/gunicorn/otp.sock
```

```
USER=root
```

```
GROUP=root
```

```
NUM_WORKERS=$(( $(nproc) * 2 + 1 )) # (2 x vCPU) + 1 → t3.small (2 vCPU) = 5 workers
```

```
TIMEOUT=30
```

```
LOG_DIR=/webapps/app/logs
```

```
# Activar el entorno virtual
```

```
cd $APPDIR
```

```
source $VENV/bin/activate
```

```
export PYTHONPATH=$APPDIR:$PYTHONPATH
```

```
export DJANGO_SETTINGS_MODULE=transtecnia_otp.settings.prod
```

```
# Crear el directorio del socket si no existe
```

```
RUNDIR=$(dirname $SOCKFILE)
```

```
test -d $RUNDIR || mkdir -p $RUNDIR
```

```
# Iniciar el servicio
```

```
exec gunicorn transtecnia_otp.wsgi:application \
```

```
  --chdir $APPDIR \
```

```
  --workers $NUM_WORKERS \
```

```
  --user=$USER --group=$GROUP \
```

```
--timeout=$TIMEOUT \  
--graceful-timeout=30 \  
--keep-alive=5 \  
--log-level=warning \  
--error-logfile=$LOG_DIR/error.log \  
--access-logfile=$LOG_DIR/access.log \  
--max-requests=1000 \  
--max-requests-jitter=100 \  
--preload \  
--bind=0.0.0.0:8002
```

Guarda los cambios ( `Ctrl + O` , `Enter` , `Ctrl + X` ).

## Paso Final: Reiniciar Producción

Aplica un reinicio al proceso desde Supervisor para que destruya la instancia fallida y ejecute el script corregido:

Bash



```
sudo supervisorctl restart app
```

(Nota: Si tu proceso en supervisor se llama diferente, por ejemplo `app_prod` , cambia el nombre al final).

Verifica el estado con:

Bash



```
sudo supervisorctl status
```

Al corregir el comando `cd` , el script encontrará la ruta `/webapps/app/otp` , activará el entorno virtual y tu endpoint de producción responderá de inmediato.

al ejecutar el endpoint ==>

<https://otp.transtecnia.cl/api/v1/otp/generate/> me esta dando error 400 bad request

Un error **400 Bad Request** significa que la solicitud llegó correctamente a Django/Gunicorn (por lo que la infraestructura y el script `gunicorn_prod.sh` ya están funcionando impecable), pero el servidor rechazó la petición porque los datos que se le enviaron en el cuerpo (Body) o en las cabeceras (Headers) no cumplen con lo que el código de Django espera.

En un endpoint de generación de OTP, esto suele deberse a validaciones estrictas del backend. Vamos a revisar las causas más comunes y cómo solucionarlo de inmediato:

## 1. La Causa Más Común: Estructura del JSON incorrecta

Asegúrate de que estás enviando un método **POST** y que el cuerpo de la petición sea un JSON válido con los campos exactos que definiste en tus serializers de Django.

Por ejemplo, verifica si tu API espera campos como `email`, `rut`, o `app_id` con una nomenclatura específica. Un error típico es enviar:

JSON



```
{
  "user_email": "ejemplo@transtecnia.cl"
}
```

Cuando el serializer de Django está esperando estrictamente:

JSON



```
{
  "email": "ejemplo@transtecnia.cl"
}
```

## 2. Falta la cabecera `Content-Type`

Si estás probando el endpoint desde Postman, cURL o el frontend, asegúrate de incluir explícitamente en los **Headers** la siguiente propiedad:

- **Key:** `Content-Type`
- **Value:** `application/json`

Si no se envía esta cabecera, Django no sabe cómo parsear el cuerpo de la solicitud y puede responder con un error 400 de inmediato.

### 3. El Veredicto: Ver el log exacto de Django

Para no adivinar qué campo está fallando o qué validación del Serializer levantó el error, lo mejor es mirar el log de errores que configuramos en tu script. Ejecuta este comando en la terminal de producción para ver qué procesó Django exactamente:

Bash



```
tail -n 20 /webapps/app/logs/error.log
```

#### ¿Qué buscar en este log?

Verás una línea que detalla la respuesta del serializer. Busca algo similar a:

- `ValidationError: {'email': ['Este campo es requerido.']}`
- `ValidationError: {'app_id': ['No es un identificador válido.']}`

### 4. Revisar la comunicación con el Microservicio de Autenticación

Como este endpoint interactúa con la variable `AUTH_URL_QA` o su equivalente de producción para validar tokens o clientes, si el formato del token enviado en los Headers (como el `Authorization: Bearer <token>`) es inválido, expiró o no se envió, el middleware de Django podría estar interceptando la petición y retornando un 400 o 401.

Revisa el output del comando `tail` del paso 3. Ponme aquí el mensaje de error que te arroje el log y te digo exactamente qué llave o parámetro debes corregir en tu petición para que te devuelva el estado **201 Created** con el OTP generado.

```
este es el metodo para generar otp ==>
@method_decorator(csrf_exempt, name='dispatch')
class GenerateOTP(APIView):
    authentication_classes = []
```





¡Perfecto, Francisco! Analizando detalladamente el código del método `post`, ya podemos identificar con absoluta precisión la única razón por la cual la API te responde con un `400 Bad Request`.

Al final del método tienes esta condición:

Python



```
if serializer.is_valid():  
    # ... procesa y retorna 201 ...  
return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)
```

Si te está devolviendo un `400`, significa que los datos que estás enviando en el Body no pasan las validaciones de tu clase `OTPGenerateSerializer`.

Revisando las líneas internas donde extraes la data validada:

Python



```
email = serializer.validated_data['email'].lower().strip()  
id_orig = serializer.validated_data['app_source']
```

Podemos ver que tu Serializer espera obligatoriamente un JSON estructurado con estas dos llaves exactas: `email` y `app_source`. Si envías nombres de llaves diferentes o si alguna de ellas falta o viaja vacía, el serializer fallará.

## La Solución: Cómo debes enviar el Body de la petición

Asegúrate de que en Postman o en el script cliente desde donde consumes la API, el cuerpo (Body) en formato raw / JSON contenga exactamente esta estructura:

JSON



```
{  
  "email": "tu-correo@transtecnia.cl",  
  "app_source": "nombre_de_la_app"  
}
```

¡Ojo con `app_source`! Verifica en la definición de tu serializer o en tu modelo si `app_source` exige un formato o valor específico (por ejemplo, si tiene un largo máximo o si debe coincidir con algún registro previo).

## Cómo ver el detalle exacto del error del Serializer

Para no adivinar cuál es el campo que no le gusta a Django, modifica temporalmente una sola línea de tu código para que la respuesta de error 400 te muestre explícitamente en la pantalla qué es lo que falta.

Cambia la última línea:

Python



```
# Cambia esto:
return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

# Por esto:
return Response({"error": "Serializer inválido", "detalles": serializer.errors}, stat
```

Al hacer este cambio e intentar lanzar la petición de nuevo, la respuesta JSON ya no dirá simplemente `400 Bad Request`, sino que te devolverá un mensaje claro explicando el motivo, por ejemplo:

- `{"detalles": {"app_source": ["Este campo es requerido."]}}`
- `{"detalles": {"email": ["Introduzca una dirección de correo válida."]}}`

## Un recordatorio importante sobre los Headers:

Dado que implementaste una validación manual de tokens JWT mediante `auth_header = request.headers.get('Authorization')`, recuerda enviar siempre la cabecera correspondiente:

- **Key:** `Authorization`
- **Value:** `Bearer <tu_token_jwt>`

Si ajustas el Body con las llaves `email` y `app_source` en minúsculas y con la data correcta, tu serializer pasará la validación limpiamente y la API procesará el flujo generando el OTP exitosamente.

este es el archivo serializers.py ==> from rest\_framework  
import serializers

class OTPGenerateSerializer(serializers.Serializer):



¡Excelente, Francisco! Al ver tu `serializers.py` confirmamos que la estructura declarada en `OTPGenerateSerializer` es sumamente limpia y directa: requiere estrictamente los campos `email` y `app_source`.

Dado que tus serializers están bien definidos, el error `400 Bad Request` se debe a que la data entrante no está haciendo "match" perfecto. Vamos a revisar los dos puntos críticos que están bloqueando tu endpoint en este momento:

## 1. El conflicto de longitud en `otp_code` (¡Ojo con esto!)

Mirando tu vista ( `views.py` de la interacción anterior), tienes esta línea donde se genera el código aleatorio:

Python



```
leng_cod_otp = settings.OTP_CONFIG['CODE_LENGTH']  
# ...  
codeotp = ''.join(random.choices(caracteres, k=leng_cod_otp))
```

Y en tu `OTPValidateSerializer` tienes una regla sumamente estricta para la validación:

Python



```
otp_code = serializers.CharField(required=True, min_length=6, max_length=6)
```

**El peligro:** Si en tu archivo de configuración ( `settings/prod.py` ) la variable `CODE_LENGTH` tiene un valor diferente a `6` (por ejemplo, `4` u `8` ), el backend generará códigos que jamás podrán ser validados por este serializer, provocando un error 400 inmediato al intentar consumirlo desde el frontend. Asegúrate de que `CODE_LENGTH` sea exactamente `6` en producción.

## 2. Formato exacto del Body en la Petición

Para el endpoint de generar ( `/api/v1/otp/generate/` ), el JSON raw que debes enviar en el cuerpo del POST desde Postman o tu aplicación debe lucir exactamente así:

JSON



```
{  
  "email": "usuario@transtecnia.cl",
```

```
"app_source": "portal_clientes"
}
```

- **Validaciones del Serializer:**

- `email` : Debe ser un correo electrónico válido (con `@` y dominio).
- `app_source` : Un texto plano que no supere los 100 caracteres.

## Cómo diagnosticar el error exacto en Postman

Para que el mismo endpoint te responda qué campo está fallando (en lugar de dar un 400 genérico), modifica la última línea de tu vista en producción para que retorne los errores del serializer.

Abre el archivo de tu vista:

**Bash**



```
nano /webapps/app/otp/transtecnia_otp/views.py
```

Y cambia la última línea del método `post` :

**Python**



```
# Cambia esto:
return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

# Por esto (Te mostrará el campo exacto que falla en la respuesta):
return Response({
    "status": "Error",
    "message": "Datos de solicitud inválidos",
    "errors": serializer.errors
}, status=status.HTTP_400_BAD_REQUEST)
```

Guarda los cambios ( `Ctrl + O` , `Enter` , `Ctrl + X` ) y reinicia la app con Supervisor:

**Bash**



```
sudo supervisorctl restart app
```

Al lanzar la petición de nuevo con esta pequeña modificación, si algo falla, el JSON de respuesta te dirá exactamente el motivo (por ejemplo: `{"errors": {"app_source":`

```
["This field is required."]]} } ).
```

el archivo `prod.py` esta de la siguiente forma ==> `import os`  
`import environ`  
`from .base import *`

¡Excelente, Francisco! Al revisar tu archivo `prod.py`, saltan a la vista **dos problemas críticos de infraestructura y configuración** que son los causantes directos del error **400 Bad Request** en tu entorno de producción:

## 1. El Host Permitido ( `ALLOWED_HOSTS` ) vs. la URL que estás consultando

En tu archivo `prod.py` tienes configurado estrictamente:

Python



```
ALLOWED_HOSTS = ['otp.transtecniasa.cl', 'www.otp.transtecniasa.cl']
```

Pero en tu mensaje anterior me indicaste que estás intentando ejecutar el endpoint usando esta URL exacta:

👉 `https://otp.transtecnia.cl/api/v1/otp/generate/` (Nota la falta del "sa" al final de transtecnia).

Cuando `DEBUG = False`, si el dominio desde el cual estás haciendo la petición (`otp.transtecnia.cl`) no coincide **exactamente** con lo que está escrito en la lista `ALLOWED_HOSTS`, Django intercepta la petición por seguridad y responde con un **400 Bad Request** **automático** (ni siquiera alcanza a evaluar tu código ni tus serializers).

## 2. Falta la configuración de `OTP_CONFIG`

En tu vista (`views.py`), tu código intenta leer la longitud del código OTP desde los settings con esta línea:

Python



```
leng_cod_otp = settings.OTP_CONFIG['CODE_LENGTH']
```

Sin embargo, en tu archivo `prod.py` no está declarada la variable `OTP_CONFIG`. Si tampoco se hereda de `base.py`, esto generará un `AttributeError` o `KeyError` interno que en producción (con `DEBUG = False`) se traduce hacia el cliente como un error genérico del servidor o una falla en el ciclo de request, gatillando respuestas inesperadas.

## La Solución: Actualizar tu archivo `prod.py`

Vamos a corregir ambos detalles de inmediato. Abre tu archivo de producción:

Bash



```
nano /webapps/app/otp/transtecnia_otp/settings/prod.py
```

Reemplaza los bloques correspondientes para que incluya los dominios correctos y la configuración del OTP:

Python



```
# ... (mantén tus imports y claves igual) ...
```

```
DEBUG = False
```

```
CSRF_TRUSTED_ORIGINS = [  
    'https://otp.transtecniasa.cl',  
    'https://www.otp.transtecniasa.cl',  
    'https://otp.transtecnia.cl',      # <--- AGREGAR ESTE  
    'https://www.otp.transtecnia.cl',  # <--- AGREGAR ESTE  
]
```

```
ALLOWED_HOSTS = [  
    'otp.transtecniasa.cl',  
    'www.otp.transtecniasa.cl',  
    'otp.transtecnia.cl',      # <--- AGREGAR ESTE (Dominio de la petición)  
    'www.otp.transtecnia.cl',  # <--- AGREGAR ESTE  
]
```

```
# ... (mantén la inicialización de env y lectura de .env igual) ...
```

```
# Configuración del OTP que requiere tu vista (Ajusta el largo si usas otro valor)  
OTP_CONFIG = {  
    'CODE_LENGTH': 6,  
}
```

```
# ... (el resto del archivo de base de datos se mantiene igual) ...
```

Guarda los cambios ( `Ctrl + O` , `Enter` , `Ctrl + X` ).

## Paso Final: Reiniciar Gunicorn en Producción

Para que Django absorba la nueva lista de dominios autorizados y el diccionario de configuración, reinicia el servicio mediante Supervisor:

Bash



```
sudo supervisorctl restart app
```

Vuelve a lanzar la petición a `https://otp.transtecnia.cl/api/v1/otp/generate/` enviando el JSON correspondiente en el Body ( `email` y `app_source` ). Al coincidir el host, la petición entrará directo al flujo del servicio y generará el OTP de manera exitosa.

ahora me esta dando el siguiente error ==> Token de seguridad inválido o corrupto

¡Buenísimo, Francisco! Que te aparezca `Token de seguridad inválido o corrupto` es una excelente noticia. Significa que:

1. El problema de `ALLOWED_HOSTS` quedó resuelto.
2. Tu serializer validó el Body perfectamente.
3. Django ya entró a tu vista ( `GenerateOTP` ) y ejecutó la capa de seguridad.

El error específico ocurre en este bloque de tu código:

Python



```
try:  
    payload = jwt.decode(token_recibido, settings.AUTH_OTP_CLIENT_SECRET, algorithms=
```

```
except jwt.InvalidTokenError:
    return Response({"error": "Token de seguridad inválido o corrupto"}, status=401)
```

Esto significa que la librería `PyJWT` rechazó el token que enviaste en la cabecera `Authorization`. Las razones en producción suelen ser tres. Vamos a revisarlas en orden para solucionarlo:

## 1. La Causa más probable: El Secreto en el archivo `.env` de Producción

Para decodificar el token, tu código usa la variable `settings.AUTH_OTP_CLIENT_SECRET`. Si el token fue generado por tu servicio de autenticación (`authservice`), Django necesita usar **exactamente la misma clave secreta** para verificar que sea válido.

Revisa tu archivo `.env` de producción:

Bash



```
nano /webapps/app/otp/.env
```

Asegúrate de que la línea `AUTH_OTP_CLIENT_SECRET` tenga el valor correcto de producción y no se haya quedado con una clave de QA o desarrollo. Debe coincidir con la firma del emisor del token.

## 2. Formato del Header en Postman / Cliente

Verifica cómo estás enviando el token en la petición. Tu código tiene esta lógica:

Python



```
parts = auth_header.split()
if len(parts) == 2 and parts[0].lower() == 'bearer':
    token_recibido = parts[1]
```

Si en Postman seleccionas la pestaña **Auth -> Bearer Token** y pegas el token ahí, Postman construye el header automáticamente como `Bearer <token>`.

Si por el contrario lo estás metiendo a mano en la pestaña **Headers**, asegúrate de que se vea exactamente así:

- **Key:** `Authorization`
- **Value:** `Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...` (con el espacio entre *Bearer* y el token).



### 3. Expiración o Algoritmo del Token

- **Algoritmo:** Tu código obliga a que sea estrictamente `algorithms=['HS256']`. Si el `authservice` está emitiendo tokens en otro formato (como `RS256`), la decodificación fallará de inmediato.
- **Expiración:** Si el token ya caducó, `jwt.decode` arroja un `ExpiredSignatureError`. En tu código tienes un `except` exclusivo para eso, por lo que si fuera por tiempo te diría *"El token de seguridad ha expirado"*. Como dice *"inválido o corrupto"*, el problema es la firma (el `Secret Key`) o la estructura del string.

### ¿Cómo solucionarlo rápido para probar?

Genera un token completamente nuevo desde tu `authservice` de producción, asegúrate de que el `.env` de la app de OTP comparta el mismo `AUTH_OTP_CLIENT_SECRET` y envíalo en la cabecera. Con eso la firma hará match, tu código extraerá el `client_id` y gatillará el envío del correo de inmediato.

```
este es el metodo para generar OTP ==>
@method_decorator(csrf_exempt, name='dispatch')
class GenerateOTP(APIView):
    authentication_classes = []
```

¡Perfecto, Francisco! Con el código completo de la vista frente a nosotros, podemos aislar de inmediato por qué está saltando al bloque de la excepción `jwt.InvalidTokenError` y devolviendo el mensaje *"Token de seguridad inválido o corrupto"*.

Hay dos detalles muy importantes en este fragmento de tu código que están causando el problema:

#### 1. El Tipo de Dato de `client_id_token` (Posible caída oculta)

Miremos estas dos líneas:

Python



```
client_id_token = payload.get('client_id')
# ...
```

```
if settings.AUTH_OTP_CLIENT_ID not in client_id_token:
```

Si el token JWT que viene desde `authservice` es un string simple en el campo `client_id` (por ejemplo, `client_id: "otp-service"`), el operador `in` funcionará. **Pero cuidado:** Si por alguna razón el token viene con `client_id` como un valor nulo (`None`), la línea `not in client_id_token` arrojará un error de Python (`TypeError: argument of type 'NoneType' is not iterable`). Al estar dentro de un bloque `try-except` genérico, ese fallo interno es atrapado por error como si fuera un token inválido, mostrando el mensaje de token corrupto.

## 2. El Desfase de la Clave Secreta en `prod.py` (Causa Principal)

Analizando tu archivo `prod.py` que me mostraste hace un momento, tienes la siguiente configuración:

Python



```
# En tu prod.py inicializas la lectura del .env:
if os.path.exists(os.path.join(RAIZ_PROYECTO, '.env')):
    environ.Env.read_env(os.path.join(RAIZ_PROYECTO, '.env'))
```

Sin embargo, no guardaste el valor del secret en el objeto `settings` de Django. En tu `prod.py` tienes esto:

Python



```
AUTH_OTP_CLIENT_SECRET=env('AUTH_OTP_CLIENT_SECRET')
```

Pero en tu vista estás llamando a:

Python



```
payload = jwt.decode(token_recibido, settings.AUTH_OTP_CLIENT_SECRET, algorithms=['HS
```

Para que `settings.AUTH_OTP_CLIENT_SECRET` exista y la vista lo pueda leer, debe estar definido en las variables globales de tu archivo `prod.py`. Si dejas la línea suelta o si se llamó antes de cargar correctamente el `.env`, Python no tendrá la firma correcta para verificar el token de producción.

## Cómo solucionarlo y diagnosticar en caliente

Para saber exactamente qué está rechazando `PyJWT`, vamos a hacer que la excepción te muestre el mensaje técnico real del error (si es un problema de firma, de estructura o un error de código).

1. Abre el archivo de tu vista:

Bash



```
nano /webapps/app/otp/transtecnia_otp/views.py
```

2. Modifica el bloque del `except jwt.InvalidTokenError` agregando la variable de captura `as e`:

Python



```
# Cambia esto:
except jwt.ExpiredSignatureError:
    return Response({"error": "El token de seguridad ha expirado"}, status=401)
except jwt.InvalidTokenError:
    return Response({"error": "Token de seguridad inválido o corrupto"}, status=

# Por esto otro (Temporalmente para diagnosticar):
except jwt.ExpiredSignatureError:
    return Response({"error": "El token de seguridad ha expirado"}, status=401)
except jwt.InvalidTokenError as e:
    return Response({
        "error": "Token de seguridad inválido o corrupto",
        "debug_msg": str(e),
        "secret_usado_es_nulo": settings.AUTH_OTP_CLIENT_SECRET is None
    }, status=401)
```

3. Guarda los cambios (`Ctrl + O`, `Enter`, `Ctrl + X`) y reinicia con Supervisor:

Bash



```
sudo supervisorctl restart app
```

Al ejecutar el endpoint en Postman nuevamente, la respuesta te dirá con peras y manzanas el motivo:

- Si `secret_usado_es_nulo` sale `true`, significa que tu `prod.py` no le está pasando la clave secreta a la vista.

- Si `debug_msg` dice `Signature verification failed`, es definitivo que el `AUTH_OTP_CLIENT_SECRET` de tu `.env` de producción no coincide con la firma con la que el `authservice` generó ese token.

me esta dando un error 500 cuando agrego el `debug_msg`

Si al agregar el `as e` y modificar el `Response` te arrojó un error 500 (Internal Server Error), significa que Django sufrió una caída crítica de sintaxis o ejecución (*Runtime Error*) antes de poder enviar la respuesta HTTP.

Esto ocurre por una de dos razones muy comunes en Python y Django REST Framework:

1. **Olvidaste importar** `settings` o `jwt` arriba en las excepciones, o bien el objeto `settings.AUTH_OTP_CLIENT_SECRET` directamente no existe (no está declarado en `prod.py`), por lo que evaluar `settings.AUTH_OTP_CLIENT_SECRET is None` hizo colapsar la vista.
2. Un error de indentación al pegar el bloque.

Vamos a limpiar el código para ir a la segura sin arriesgar un error 500. Modifica el bloque `try-except` de tu archivo `/webapps/app/otp/transtecnia_otp/views.py` para que quede así, usando un `str(e)` genérico y seguro:

Python



```
try:
    # Decodificamos el token usando tu SECRET_KEY de Django
    payload = jwt.decode(token_recibido, settings.AUTH_OTP_CLIENT_SECRET, alg

    # Extraemos los datos del token
    client_id_token = payload.get('client_id')

    # Validación extra: Verificar si el scope permite generar OTP
    if not client_id_token or settings.AUTH_OTP_CLIENT_ID not in client_id_to
        return Response({"error": "El token no tiene permisos suficientes (sc

except jwt.ExpiredSignatureError:
    return Response({"error": "El token de seguridad ha expirado"}, status=40
except Exception as e:
    # Capturamos CUALQUIER error de JWT o de código y lo escupimos de forma s
    return Response(ξ
```

```
"error": "Token de seguridad inválido o corrupto",  
"detalles_tecnicos": str(e)  
}, status=401)
```

## Paso Final: Reiniciar y Probar

Guarda el archivo y reinicia el servicio para aplicar la valla de seguridad:

Bash



```
sudo supervisorctl restart app
```

## ¿Qué verás ahora en Postman?

Al ejecutar el endpoint, el error 500 desaparecerá y obtendrás la respuesta en el JSON. Fíjate bien en lo que dice `detalles_tecnicos`:

- **Si dice:** `'Settings' object has no attribute 'AUTH_OTP_CLIENT_SECRET'`: El problema es 100% que tu archivo `prod.py` no tiene expuesta esa variable hacia Django (recuerda que debe declararse como variable global en mayúsculas dentro del archivo, ej: `AUTH_OTP_CLIENT_SECRET = env('AUTH_OTP_CLIENT_SECRET')` ).
- **Si dice:** `Signature verification failed`: Confirmado que la clave secreta de tu `.env` de producción no es idéntica a la que usó el `authservice` para firmar ese token JWT.

```
al colocar en el reponse o siguiente ==> except  
jwt.ExpiredSignatureError:  
    return Response({"error": "El token de seguridad ha  
    expirado"}, status=401)
```

¡Esto es una excelente pista, Francisco! Que te siga devolviendo **exactamente** la misma respuesta:

JSON



```
{  
  "error": "Token de seguridad inválido o corrupto"  
}
```

...sin incluir la llave `"secret_usado_es_nulo"`, nos demuestra con total certeza matemática una sola cosa: **Supervisor no está aplicando los cambios que estás guardando en el archivo `views.py`**. Cuando modificas el código de Python en producción y ejecutas `sudo supervisorctl restart app`, a veces Supervisor le manda una señal de apagado a Gunicorn pero los procesos "workers" viejos se quedan colgados en memoria en segundo plano (*procesos zombies*). Al no morir realmente, siguen ejecutando el código antiguo de la vista.

Vamos a matar esos procesos rebeldes por la fuerza para obligar a Django a cargar tu nuevo código. Sigue estos pasos en la terminal:

### Paso 1: Forzar el cierre absoluto de Gunicorn

Ejecuta este comando para matar cualquier proceso de Gunicorn que haya quedado atrapado en memoria:

Bash



```
sudo pkill -f gunicorn
```

### Paso 2: Verificar que no quede ninguno activo

Corre este comando. Debería volver completamente vacío. Si todavía aparece alguna línea con `gunicorn`, vuelve a ejecutar el Paso 1:

Bash



```
ps aux | grep gunicorn | grep -v grep
```

### Paso 3: Volver a levantar el servicio desde Supervisor

Ahora que nos aseguramos de que el terreno está limpio y no hay procesos viejos corriendo, dile a Supervisor que inicie la aplicación desde cero:

Bash



```
sudo supervisorctl start app
```

(Si por alguna razón te dice que ya está corriendo o da un error, haz un `sudo supervisorctl status app` para ver el estado, o simplemente corre un `sudo supervisorctl restart app`).

## Paso 4: Probar el endpoint en Postman

Vuelve a enviar la petición POST a tu endpoint de generación de OTP. Ahora sí o sí vas a ver una de estas dos respuestas en tu JSON:

- Opción A (Con la llave nueva):

JSON



```
{
  "error": "Token de seguridad inválido o corrupto",
  "secret_usado_es_nulo": true
}
```

(Si te sale `true`, confirmamos que el archivo `prod.py` no le está pasando la variable a la vista).

- Opción B (Con el error 500 que te dio antes):

Si te da el Error 500, significa que por fin leyó el código modificado y se cayó por la falta del alias `as e` o por la lectura del `settings`. Si pasa esto, aplica el bloque de código seguro con `except Exception as e` que te pasé en el mensaje anterior para que veamos el motivo real.

Haz la limpieza de procesos con `pkill`, lanza la petición y dime cuál de las dos respuestas te aparece en Postman para dar con la solución definitiva.

```
ahora me dio este error ==> {
  "error": "Token de seguridad inválido o corrupto",
  "secret_usado_es_nulo": false
}
```

¡Perfecto, Francisco! La respuesta del servidor nos acaba de dar la pieza del rompecabezas que faltaba.

Al arrojar `"secret_usado_es_nulo": false`, confirmamos al 100% que:

1. Tu archivo `prod.py` está cargando correctamente el archivo `.env`.
2. La vista sí está leyendo la variable `settings.AUTH_OTP_CLIENT_SECRET` (no viaja vacía ni nula).

Por lo tanto, si la clave existe pero `PyJWT` sigue levantando la excepción `InvalidTokenError`, significa que hay una **incompatibilidad real en la firma o en la estructura del token**.

Aquí están las 3 únicas razones por las que esto ocurre y cómo solucionarlo:

## 1. La Causa Principal: El `AUTH_OTP_CLIENT_SECRET` no coincide con el Emisor

El token JWT que estás enviando en Postman fue generado por el microservicio de autenticación (`authservice`). Para que la librería `jwt.decode` valide que ese token es legítimo, el string secreto guardado en la variable `AUTH_OTP_CLIENT_SECRET` de tu archivo `/webapps/app/otp/.env` **debe ser exactamente idéntico** al secreto con el que el `authservice` firmó el token en su propio entorno de producción.

Si en el `.env` de OTP pusiste, por ejemplo, el secreto de QA (`HrzqI0NafT...`), pero estás enviando un token generado por el AuthService de Producción (o viceversa), la verificación de la firma fallará siempre.

**Solución:** Asegúrate de que el secreto en tu archivo `.env` de la app de OTP coincida carácter por carácter con la clave secreta de producción del servicio de autenticación.

## 2. Espacios o caracteres ocultos en el archivo `.env`

A veces, al copiar y pegar secretos largos en la terminal usando `nano`, se pueden colar espacios en blanco al final de la línea o un salto de línea invisible. `django-environ` toma esos espacios como parte del secreto.

**Solución:**

Abre tu archivo `.env`:

Bash



```
nano /webapps/app/otp/.env
```

Ve al final de la línea de `AUTH_OTP_CLIENT_SECRET` y asegúrate de que **no haya ningún espacio en blanco** después del último carácter. Si lo hay, bórralo, guarda (`Ctrl + O`,



Enter , Ctrl + X ) y reinicia los procesos:

Bash



```
sudo pkill -f gunicorn && sudo supervisorctl start app
```

### 3. El token sufrió alteraciones al enviarse (Copiar y Pegar)

Verifica que al copiar el token desde donde lo generaste no te haya faltado ningún carácter del principio (como las primeras letras `eyJ...`) o del final, y que en Postman esté bien estructurado en la pestaña **Headers** como:

- **Key:** `Authorization`
- **Value:** `Bearer <token_completo>`

### El Diagnóstico Definitivo (Ver el mensaje interno de PyJWT)

Para ir a la segura y saber si el problema es de **firma** o de **formato**, usemos el bloque `except Exception as e` para capturar el texto nativo de la librería de JWT.

Modifica el bloque `except` en tu archivo

`/webapps/app/otp/transtecnia_otp/views.py`:

Python



```
except jwt.ExpiredSignatureError:
    return Response({"error": "El token de seguridad ha expirado"}, status=400)
except Exception as e:
    return Response({
        "error": "Token de seguridad inválido o corrupto",
        "secret_usado_es_nulo": settings.AUTH_OTP_CLIENT_SECRET is None,
        "error_interno_jwt": str(e) # <--- AGREGA ESTA LÍNEA
    }, status=401)
```

Guarda, limpia la memoria y reinicia:

Bash



```
sudo pkill -f gunicorn && sudo supervisorctl start app
```

Ejecuta la petición una vez más. El valor que aparezca en `"error_interno_jwt"` (como `Signature verification failed` o `Not enough segments`) nos dirá exactamente cuál

de los tres puntos de arriba es el que debemos corregir.



A stylized, handwritten signature in blue ink, consisting of a loop followed by a horizontal stroke.